Bachelorarbeit

KI-gestütztes Liveticker-Redaktionssystem für den ProfifuSSball

Automatisierte Textgenerierung für Echtzeit-Sportjournalismus mit Human-in-the-Loop-Kontrolle

Autor

Jonas Kimmer

Hochschule

Technische Hochschule Aschaffenburg

Fachbereich Informatik

Betreuung

Prof. Dr. [Name]

Firmenbetreuung

Stackwork GmbH, Eintracht Frankfurt

April 2026

Abstract

Die manuelle Erstellung von Liveticker-Einträgen im ProfifuSSball ist ein kognitiv anspruchsvoller, zeitkritischer Prozess. Das entwickelte System kombiniert eine FastAPI-Backend-Architektur mit einer mehrsprachigen LLM-Pipeline und einem React-Frontend. Drei Betriebsmodi ermöglichen einen graduellen Übergang zwischen autonomer KI-Generierung und redaktioneller Kontrolle.

Die Evaluation umfasst 391 automatisierte Tests, eine TypeScript-Typenabdeckung von 95,84% sowie eine qualitative Bewertung von 16 KI-generierten Ticker-Einträgen aus 9 Bundesliga-Spielen. Die KI-generierten Texte erzielen einen Gesamtdurchschnitt von 4,3/5 Punkten. Die geschätzte Time-to-Publish im Auto-Modus beträgt 3,4–5,9 Sekunden gegenüber 30–120 Sekunden im manuellen Modus.
Schlüsselwörter: Large Language Models, Liveticker, Natural Language Generation, Human-in-the-Loop, Few-Shot Prompting, Sportjournalismus, FastAPI, React, TypeScript

Inhaltsverzeichnis

0.1 Kapitel 1 – Einleitung

0.1.1 1.1 Problemstellung und Zielsetzung

"Ein vernünftiger Live-Ticker ist fast unmöglich zu schreiben. Denn sein Konzept ist die komplette Überforderung. Des Autors. Des Schreibens. Und der Wirklichkeit." Mit diesen Worten beschreibt der Journalist Constantin Seibt das Grundproblem der Liveticker-Berichterstattung — eine Einschätzung, die im modernen digitalen Sportjournalismus bis heute nichts an Aktualität verloren hat (zitiert nach Beils 2023, S. 56). Der Liveticker hat sich zur dominanten Darstellungsform der Echtzeit-Berichterstattung im ProfifuSSball entwickelt und steht dabei exemplarisch für einen Zielkonflikt zwischen **Geschwindigkeit und Qualität**, der mit rein manuellen Mitteln nicht auflösbar ist.

Die vorliegende Arbeit identifiziert drei Problemdimensionen, die den Einsatz neuer technologischer Lösungen notwendig machen:

- Der **operative Zeitdruck** bei der manuellen Texterstellung: Redakteure müssen Spielereignisse in Echtzeit beobachten, einordnen und in stilistisch kohärente Kurztexte überführen — typischerweise innerhalb von 30 bis 120 Sekunden pro Eintrag.

- Die Herausforderungen der **Internationalisierung** durch Mehrsprachigkeit: ProfifuSSballvereine bedienen zunehmend internationale Fanbases; deutschsprachige Redaktionen stehen vor der Anforderung, Inhalte parallel auf Englisch oder weiteren Sprachen bereitzustellen, ohne ausreichend mehrsprachiges Redaktionspersonal vorzuhalten.

- Der **strukturelle Wandel** der Vereine zu eigenständigen Medienproduzenten mit White-Label-Anforderungen: Vereine betreiben eigene digitale Kanäle und Medienmarken, die Redaktionswerkzeuge mit konfigurierbarem vereinsspezifischem Ton und CI-konformer Gestaltung erfordern.

Diese Dimensionen werden in Kapitel 2 wissenschaftlich fundiert und in konkrete Systemanforderungen überführt.

Das übergeordnete Ziel der vorliegenden Arbeit ist daher die Konzeption und Implementierung eines produktionsfähigen Redaktionssystems, das diesen Konflikt durch den gezielten Einsatz von **Large Language Models** auflöst, während die finale Entscheidungshoheit und publizistische Verantwortung beim Menschen verbleiben. Das System wird dabei in zwei Ausprägungen realisiert: einer generischen Instanz für beliebige Vereine sowie einer vereinsspezifischen White-Label-Instanz am Beispiel **Eintracht Frankfurt**.

Um dieses Ziel zu operationalisieren, definiert und evaluiert die Arbeit drei Betriebsmodi:

1. **Status quo (`manual`):** Vollmanuelle Erstellung als Vergleichsbasis.
2. **Vollautonom (`auto`):** KI-basierte Texterstellung und Publikation ohne menschliches Korrektiv.
3. **Hybrid (`coop`):** Kombination aus KI-generierten Textvorschlägen mit redaktioneller Prüfung und Freigabe.

0.1.2 1.2 Forschungsfrage

Aus der in Kapitel 1.1 entwickelten Zielsetzung ergibt sich folgende Forschungsfrage, die der vorliegenden Arbeit zugrunde liegt:

> Inwiefern reduziert ein hybrides KI-gestütztes Redaktionssystem die Time-to-Publish bei der Liveticker-Erstellung im ProfifuSSball im Vergleich zur rein manuellen Erstellung, ohne die journalistische Qualität hinsichtlich Korrektheit, Tonalität und Verständlichkeit zu beeinträchtigen?

Die Frage ist bewusst zweiteilig formuliert: Sie erfasst einerseits eine **zeitliche Dimension** — die Reduktion der Publikationslatenz — und andererseits eine **qualitative Dimension**, die sicherstellt, dass Geschwindigkeit nicht auf Kosten journalistischer Standards erzielt wird. Beide Dimensionen sind für die Praxis gleichermaSSen relevant, da Schnelligkeit zwar als oberstes Qualitätskriterium gilt (Beils 2023, S. 57), sprachliche und inhaltliche Korrektheit jedoch selbst unter hohem Zeitdruck nicht verhandelbar sind (Bluhm & Schäfer 2023, S. 35). Die Operationalisierung beider Dimensionen wird in Kapitel 1.3 methodisch vorbereitet und in Kapitel 6 vollständig durchgeführt.

0.1.3 1.3 Methodisches Vorgehen

Die Arbeit folgt einem konstruktiven Forschungsansatz im Sinne des **Design Science Research** (Hevner et al. 2004): Ein softwarebasiertes Artefakt wird als Antwort auf ein identifiziertes Praxisproblem entworfen, implementiert und evaluiert. Der Erkenntnisgewinn entsteht dabei nicht nur aus der Evaluation, sondern auch aus dem Entwurfs- und Implementierungsprozess selbst.

Das methodische Vorgehen gliedert sich in fünf Phasen:

1. **Problemanalyse und Anforderungserhebung:** Literaturgestützte Vertiefung der Problemdimensionen und Ableitung funktionaler sowie nicht-funktionaler Anforderungen.

2. **Technologierecherche:** Aufarbeitung des Stands der Technik in den relevanten Bereichen und Bewertung hinsichtlich der Eignung für das identifizierte Problem.

3. **Systemkonzeption:** Entwurf einer dreischichtigen Systemarchitektur mit Datenmodell, LLM-Pipeline und Prompt-Design.

4. **Implementierung:** Umsetzung als produktionsfähige Webanwendung mit vollständiger API, browserbasiertem Frontend, automatisiertem ETL-System und Cloud-Deployment.

5. **Evaluation:** Technische Qualitätssicherung, Evaluation der KI-Textgenerierung sowie systematischer Anforderungsabgleich.

Die Evaluationsmethodik operationalisiert die Forschungsfrage entlang zweier Dimensionen: Die **zeitliche Dimension** wird über die Time-to-Publish-Metrik (TTP) gemessen, die den Zeitraum zwischen Spielereignis und Veröffentlichung des Ticker-Eintrags erfasst. Die **qualitative Dimension** wird über eine manuelle Bewertung auf den Skalen Korrektheit, Tonalität und Verständlichkeit operationalisiert, ergänzt durch ein **strukturiertes Experteninterview** mit einem professionellen Sportredakteur zur Validierung der Systemeignung im redaktionellen Arbeitskontext. Für den statistischen Vergleich stehen nicht-parametrische Verfahren (Cliff's Delta, Bootstrap-Konfidenzintervalle) bereit; aufgrund der Stichprobengröße (n = 16) kommen diese im vorliegenden Evaluationsdesign nicht zur Anwendung, stehen jedoch für eine Folgestudie mit größerer Datenbasis zur Verfügung.

1.3.1 Abgrenzung

Das System geht bewusst über den Umfang eines typischen akademischen Projekts hinaus und zielt auf Produktionsfähigkeit: Eine umfassende automatisierte Testsuite, ein Cloud-Deployment und die Integration realer Datenquellen dokumentieren diesen Anspruch. Diese Praxisorientierung ist durch den beruflichen Kontext des Autors motiviert: Als Mitarbeiter der **Stackwork GmbH** im IT-Bereich von Eintracht Frankfurt entstand das System

in direkter Kooperation mit den fachlichen Anforderungen einer professionellen Redaktion. Das Ergebnis ist ein vollständiges, eigenständig lauffähiges Redaktionssystem, das als Cloud-Service alle Spiele der konfigurierten Wettbewerbe verarbeitet. Zusätzlich werden publizierte Inhalte über Export-Workflows an die bestehende **Stackwork Demo App** übertragen, wodurch das Projekt zwei unabhängige Zielsysteme über nahezu identische Workflows bedient. Die technischen Details der Architektur und Implementierung werden in Kapitel 4 und 5 beschrieben.

Dennoch bestehen Einschränkungen hinsichtlich Authentifizierung, Evaluationsunabhängigkeit und thematischer Reichweite, die in Kapitel 7 kritisch reflektiert werden.

0.1.4 1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in acht Kapitel:

- **Kapitel 1 – Einleitung** *(vorliegendes Kapitel)*: Problemstellung, Forschungsfrage, methodisches Vorgehen und Aufbau der Arbeit.
- **Kapitel 2 – Motivation und Anforderungen:** Vertiefung der drei Problemdimensionen (redaktioneller Aufwand, Mehrsprachigkeit, White-Label-Bedarf), Ableitung der Systemanforderungen am Beispiel Eintracht Frankfurt, Experteninterview-Leitfaden und formale Anforderungsdefinition.
- **Kapitel 3 – Stand der Technik:** Large Language Models, Prompt Engineering, Natural Language Generation im Sport sowie Echtzeit-Technologien und ETL-Prozesse.
- **Kapitel 4 – Systemkonzeption:** Dreischichtige Architektur, Datenmodell, LLM-Pipeline und Prompt-Design.
- **Kapitel 5 – Implementierung:** Umsetzung als produktionsfähige Webanwendung mit über 70 API-Endpunkten, TypeScript-Frontend und n8n-ETL-System; ergänzt durch vollständige TypeScript-Typisierung, automatisierte Testsuite und Cloud-Deployment.
- **Kapitel 6 – Evaluation:** Technische Qualitätssicherung (Tests, Coverage, Typsicherheit), Evaluation der KI-Textgenerierung und systematischer Anforderungsabgleich.
- **Kapitel 7 – Diskussion:** Einordnung in den Stand der Technik, kritische Reflexion der Limitationen und Implikationen für den Sportjournalismus.
- **Kapitel 8 – Fazit und Ausblick:** Zusammenfassung der Erkenntnisse, Beantwortung der Forschungsfrage und Ausblick.

0.2 Kapitel 2 – Motivation und Anforderungen

0.2.1 2.1 Redaktioneller Aufwand bei Livetickern

Die manuelle Erstellung von Livetickern zählt zu den kognitiv anspruchsvollsten Tätigkeiten im Journalismus (Beils 2023, S. 10). Was von auSSen wie eine schlichte Abfolge kurzer Texteinheiten wirkt, erweist sich bei näherer Betrachtung als hochkomplexer Produktionsprozess, der mehrere journalistische Kompetenzen simultan abverlangt. Bluhm und Schäfer (2023, S. 27) beschreiben unter Rückgriff auf Ojomo und Olomojobi (2021) die strukturelle Komposition von Liveticker-Einträgen als einen Vierschritt aus:

1. **Narration:** Beschreibung von Aktionen.
2. **Evaluation:** ÄuSSerung von Meinungen über Szenen.
3. **Elaboration:** Vermittlung von Hintergrundinformationen.
4. **Zusammenfassung:** Erstellung eines Überblicks über den Wettbewerbsverlauf.

In ihrer Konstruktion sind Liveticker damit als hybride Form zu verstehen, in der sich Meinungsbeiträge, Berichte und Unterhaltungselemente verschmelzen (Bluhm & Schäfer 2023, S. 27). Diese inhaltliche Vielschichtigkeit unterscheidet den Liveticker grundlegend von anderen journalistischen Kurzformaten. Liveticker werden dabei in Abständen von einer bis drei Minuten aktualisiert (Hauser 2008, S. 3–4; Bluhm & Schäfer 2023, S. 32), wobei Schnelligkeit und Fehlerfreiheit in einem permanenten Widerspruch stehen: Trotz der extremen Publikationsfrequenz gilt sprachliche und inhaltliche Korrektheit als oberste Priorität (Bluhm & Schäfer 2023, S. 35).

Aus den genannten Merkmalen ergibt sich folgender konkreter Workflow: Ein Ereignis findet auf dem Spielfeld statt — etwa ein Tor, eine Auswechslung oder ein Foul. Die Autorin oder der Autor muss das Geschehen wahrnehmen, in Sekundenbruchteilen die journalistische Relevanz bewerten, den Eintrag mental strukturieren und unmittelbar in eine sprachlich angemessene Form bringen (vgl. Bluhm & Schäfer 2023, S. 27; Walter 2020, S. 29). Gleichzeitig sind Kontextinformationen abzurufen: Wie steht es? Wer hat das letzte Tor erzielt? In welcher Minute war der letzte Eintrag? Diese kognitive Parallelverarbeitung aus Beobachtung, Bewertung, Kontextualisierung und Textproduktion macht den Liveticker zu einer Echtzeit-Textgattung, die sich fundamental von retrospektiven Formaten wie dem Spielbericht unterscheidet (vgl. Hauser 2008, S. 2–3).

Erschwerend kommt der extreme Zeitdruck hinzu. Unmittelbar nach einer relevanten Spielszene haben die Autorinnen und Autoren nur wenige Augenblicke Zeit, das Geschehen konzise, anschaulich und vor allem spannend-unterhaltsam zu beschreiben (Meier 2018, zitiert nach Bluhm & Schäfer 2023, S. 27). Da zum Gegenlesen keine Zeit bleibt, müssen Verfassen und Redigieren in einem einzigen Prozess erfolgen (Walter 2020, S. 29). Verschärft

wird die Situation dadurch, dass Ticker-Autoren häufig parallel weitere Aufgaben übernehmen müssen — wie etwa die Erstellung des abschlieSSenden Spielberichts (Bluhm & Schäfer 2023, S. 35). Die Fehleranfälligkeit unter diesen Bedingungen ist systemimmanent: Zahlendreher bei Spielminuten, falsch zugeordnete Spielernamen oder stilistische Brüche innerhalb eines Tickers sind keine Ausnahme, sondern eine vorhersehbare Konsequenz des Produktionsdrucks (vgl. Walter 2020, S. 29; Bluhm & Schäfer 2023, S. 35).

Das Skalierungsproblem verschärft sich an Spieltagen mit Parallelspielen erheblich. An einem typischen Bundesliga-Samstag finden bis zu sechs Partien zeitgleich statt (vgl. DFL 2024, Rahmenspielplan Bundesliga). Redaktionen, die für mehrere Spiele Ticker anbieten, müssen entweder entsprechend viele Autoren gleichzeitig einsetzen oder einzelne Autoren mehrere Ticker parallel betreuen lassen — mit absehbaren QualitätseinbuSSen (vgl. Bluhm & Schäfer 2023, S. 32, 35). Für Vereinsredaktionen, die zusätzlich zu Pflichtspielen auch Jugend-, Frauen- oder Freundschaftsspiele abdecken wollen, ist eine rein manuelle Skalierung personell und wirtschaftlich kaum darstellbar.

Die personellen Konsequenzen sind in der Praxis spürbar. Liveticker-Stellen werden häufig mit freien Mitarbeitern oder studentischen Hilfskräften besetzt, die teils räumlich isoliert arbeiten (Bluhm & Schäfer 2023, S. 32). Der Aufwand bleibt hoch, die Ressourcen begrenzt, und die Qualität unterliegt einer strukturellen Inkonsistenz: Wechselndes Personal, unterschiedliche Erfahrungsstufen und variierende Arbeitsbedingungen — Stadion, Pressebereich oder Home-Office — führen dazu, dass Tonalität und sprachliches Niveau von Spiel zu Spiel schwanken. Diese Inkonsistenz ist kein individuelles Versagen, sondern eine systemische Folge des manuellen Produktionsprozesses (vgl. Bluhm & Schäfer 2023, S. 32, 35). Eine technologische Unterstützung ist daher keine optionale Ergänzung, sondern eine strukturelle Notwendigkeit.

Die empirische Validierung dieser Problemdimension erfolgt über das Experteninterview (Kap. 2.4) mit folgenden Fragen:

- *IF1: Wie läuft die Liveticker-Produktion bei Eintracht Frankfurt aktuell ab? Beschreiben Sie bitte einen typischen Spieltag.*
- *IF2: Wie viele Personen sind an einem Spieltag für den Liveticker eingeteilt?*
- *IF3: Welche Aufgaben muss ein Ticker-Autor neben dem Tickern parallel erledigen?*
- *IF4: Was sind die häufigsten Fehlerquellen unter Zeitdruck? Passieren zum Beispiel Zahlendreher, falsche Spielernamen oder stilistische Brüche?*
- *IF5: Wie wird die Ticker-Abdeckung bei Parallelspielen oder Jugend-/Frauenspielen gehandhabt?*

(IF6 ist thematisch in Kap. 2.2 eingebettet. IF7–IF9 sind thematisch in Kap. 2.3 eingebettet.)

0.2.2 2.2 Mehrsprachigkeit als Herausforderung

Die Globalisierung des ProfifuSSballs hat die Anforderungen an die Reichweite digitaler Inhalte grundlegend verändert. Internationale Fangemeinschaften erwarten heute eine mediale Begleitung in ihrer Landessprache. Am Beispiel der Premier League lässt sich dieser Wandel besonders deutlich illustrieren: Der internationale Markt ist für die Liga längst bedeutender als der heimische englische (Beils 2023, S. 221). Dies betrifft zunehmend auch Bundesliga-Vereine mit zweistelligen Millionen-Reichweiten auf globalen Plattformen (vgl. Beils 2023, S. 200). Am Beispiel von Eintracht Frankfurt zeigt sich dies konkret: Durch Spieler wie Makoto Hasebe und Daichi Kamada verfügte der Verein in zurückliegenden Spielzeiten über eine beachtliche internationale Anhängerschaft, insbesondere in Japan, wie sich an den fremdsprachigen Interaktionen auf den offiziellen Social-Media-Kanälen des Vereins beobachten lässt (eigene Beobachtung). Eine rein deutschsprachige Berichterstattung wird dieser Reichweite nicht mehr gerecht.

Die Herausforderung beschränkt sich dabei nicht auf eine reine Übersetzung existierender Inhalte. Smuczyski (2020, S. 57) zeigt am Beispiel der mehrsprachigen Berichterstattung von *Die Zeit*, dass mehrsprachige Echtzeitinhalte ein journalistisches Qualitätsmerkmal darstellen, das über wörtliche Translation hinausgeht. Im Kontext von Sport-Livetickern kommt hinzu, dass sprachliche Konventionen, idiomatische Wendungen und die emotionale Tonalität kulturspezifisch variieren. Ein euphorischer Ticker-Eintrag auf Deutsch folgt anderen stilistischen Mustern als sein englisches oder japanisches Pendant. Es handelt sich daher weniger um ein Übersetzungs- als um ein Lokalisierungsproblem.

Der operative Zeitdruck potenziert sich mit jeder zusätzlichen Sprache erheblich: Jede Sprache erfordert spezialisiertes Personal für zeitaufwendige Übersetzungs- und Anpassungsprozesse, die manuell kaum mit der Echtzeit-Anforderung vereinbar sind (vgl. Bluhm & Schäfer 2023, S. 32). Eine manuelle Skalierung auf mehrere Sprachen ist wirtschaftlich und personell für die meisten Vereinsredaktionen nicht darstellbar. Moderne Large Language Models eröffnen hier eine neue Perspektive: Sie können Texte nicht nur übersetzen, sondern direkt in der Zielsprache generieren — unter Berücksichtigung von Stil, Tonalität und kulturellem Kontext (vgl. OpenAI 2023, S. 1; Kap. 3.1). Diese Fähigkeit macht eine mehrsprachige Liveticker-Produktion erstmals ohne proportionalen Personalaufwand denkbar.

Die empirische Validierung dieser Problemdimension erfolgt über das Experteninterview (Kap. 2.4) mit folgender Frage:

- *IF6: Gibt es aktuell Bedarf an mehrsprachigen Tickern? Wenn ja, welche Sprachen wären relevant?*

0.2.3 2.3 White-Label-Bedarf im Profisport

Vereine und Verbände kommunizieren heute zunehmend direkt mit ihrer Anhängerschaft und haben eigene Redaktionen aufgebaut (Beils 2023, S. 155). Die Konsequenz für die Liveticker-Produktion ist erheblich: Vereinseigene Inhalte folgen einer anderen Logik als neutrale Berichterstattung.

Der entscheidende Unterschied liegt im Blickwinkel. Während überregionale Medien neutral bleiben, betrachten Vereinsticker die Partie aus der Perspektive der eigenen Fans (Beils 2023, S. 57). Ein Ereignis muss je nach Medium anders getönt sein: emotional für den Vereinskanal, sachlich für neutrale Plattformen. Die in Kapitel 2.1 beschriebene personelle Fluktuation verschärft das Problem zusätzlich: Eine konsistente Markenstimme ist mit wechselndem Personal manuell kaum zu gewährleisten. Eine technologische Lösung muss daher auch die stilistische Konsistenz adressieren.

Im Softwarekontext wird dieses Problem durch sogenannte White-Label-Architekturen gelöst: Ein generisches Kernsystem wird so konzipiert, dass es für verschiedene Abnehmer individuell konfigurierbar ist — ohne den Quellcode zu ändern (vgl. Kap. 4). Überträgt man dieses Prinzip auf die Liveticker-Produktion, ergibt sich die Anforderung, dass ein und dasselbe System sowohl einen vereinsspezifischen Ticker mit emotionaler Fanbindung als auch einen neutralen, anbieterunabhängigen Ticker bedienen können muss. Die stilistische Differenzierung — von der Wortwahl über die Tonalität bis hin zu vereinsspezifischen Redewendungen — muss dabei als Konfiguration abbildbar sein, nicht als separate Implementierung.

Die empirische Validierung dieser Problemdimension erfolgt über das Experteninterview (Kap. 2.4) mit folgenden Fragen:

- *IF7: Wie wird bisher sichergestellt, dass verschiedene Autoren in einem konsistenten Stil schreiben? Gibt es einen Styleguide?*
- *IF8: Wie würden Sie den Eintracht-Ticker-Stil beschreiben — was macht ihn aus, was unterscheidet ihn von neutraler Berichterstattung?*
- *IF9: Wie wichtig ist die stilistische Konsistenz über verschiedene Spiele hinweg?*

Wie sich diese Anforderungen am konkreten Beispiel eines Bundesliga-Vereins ausprägen, zeigt der folgende Abschnitt.

0.2.4 2.4 Anforderungsanalyse am Beispiel Eintracht Frankfurt

Eintracht Frankfurt dient in dieser Arbeit als konkreter Anwendungsfall für die vereinsspezifische White-Label-Instanz. Die Anforderungsanalyse stützt sich auf die in den vorangegangenen Abschnitten hergeleiteten Problemdimensionen, eine Analyse der öffentlich

zugänglichen digitalen Kanäle des Vereins sowie ein **strukturiertes Experteninterview** mit einem professionellen Sportredakteur von Eintracht Frankfurt. Die theoretisch hergeleiteten Anforderungen werden dabei durch das Interview empirisch validiert — insbesondere hinsichtlich des redaktionellen Workflows (Kap. 2.1), des Bedarfs an mehrsprachiger Berichterstattung (Kap. 2.2) und der stilistischen Konsistenzanforderungen (Kap. 2.3). Das strukturierte Leitfadeninterview nach Kuckartz (2018) wurde gewählt, weil es bei kleiner Fallzahl eine systematische Vergleichbarkeit der Antworten mit den hergeleiteten Anforderungen ermöglicht. Der Interviewpartner ist als Experte im Sinne von Meuser und Nagel (1991) einzustufen, da er über mehrjährige Redaktionserfahrung im Vereinsmedienbereich verfügt. Die Auswahl eines Einzelfalls ist durch die explorative Zielsetzung der Arbeit gerechtfertigt. Die vollständige Auswertung des Interviews erfolgt in Abschnitt 6.3.6 mittels thematischer Inhaltsanalyse nach Kuckartz (2018).

Aus den drei Problemdimensionen ergeben sich folgende vereinsspezifische Anforderungen:

- **Zeitdruck**: Eintracht Frankfurt bestreitet pro Saison bis zu 60 Pflichtspiele (34 Bundesliga-Spieltage, bis zu 6 DFB-Pokal-Runden, bis zu 17 europäische Gruppenspiele und K.o.-Runden sowie Testspiele und Supercup). Pro Spiel fallen durchschnittlich 20–40 Ticker-Einträge an (erfahrungsbasierte Schätzung aus dem Projektverlauf, gestützt auf Beobachtung öffentlich zugänglicher Vereinsticker). Eine manuelle Abdeckung aller Spiele mit konsistenter Qualität erfordert einen Personalaufwand, der für Vereine auSSerhalb der absoluten Spitze kaum darstellbar ist.

- **Mehrsprachigkeit**: Mit einer internationalen Fanbasis — insbesondere in Japan (vgl. Kap. 2.2) — besteht Bedarf an Tickern in mindestens Deutsch und Englisch, perspektivisch auch Japanisch.

- **Tonalität**: Vereinseigene Ticker folgen einer emotionalen, fanbezogenen Perspektive, die sich von der neutralen Berichterstattung überregionaler Medien unterscheidet (vgl. Beils 2023, S. 57). Das System muss diese stilistische Differenzierung abbilden können.

Über die in den Kapiteln 2.1–2.3 eingebetteten Validierungsfragen (IF1–IF9) hinaus adressiert das Experteninterview zwei weitere Themenblöcke, die erst nach der Systemdemonstration beantwortet werden können: die **Systembewertung** (IF10–IF15, eingebettet in Abschnitt 6.3.6) und die **Zukunftsperspektive** (IF16–IF18, adressiert in Kap. 7.5.2). Ein valides Systemurteil zu Block 3 und 4 setzt die vorherige Exposition mit dem laufenden System voraus, weshalb diese Blöcke sequenziell nach der Systemdemonstration eingesetzt werden. Der vollständige Interviewleitfaden mit 18 Fragen ist als separates Dokument beigefügt.

0.2.5 2.5 Linguistische Anforderungen an die Textgenerierung

Neben den organisatorischen und stilistischen Anforderungen stellt die automatisierte Liveticker-Produktion auch spezifische **linguistische** Anforderungen. Liveticker stellen eine linguistisch eigenständige Textgattung dar (Hauser 2008, S. 1). Hauser charakterisiert sie als **schriftlich-grafische Simultanerzählung**. Im Gegensatz zum Spielbericht fehlt der *benefit of hindsight*; es handelt sich um eine quasi-synchrone Berichterstattung (Hauser 2008, S. 2).

Zentrale linguistische Merkmale für das Prompt Design sind:

- **Konzeptionelle Mündlichkeit (Oraliteralität):** Verschriftlichung mündlicher Ausdrucksweisen (Smuczyski 2020, S. 54).
- **Syntaktik:** Hoher Anteil an **Ellipsen** (verblosen Satzeinheiten). Sätze sollten idealerweise nicht mehr als 15 Wörter umfassen; Hauptsatzkonstruktionen sind dominant (Walter 2020, S. 40, 52).
- **Tempora:** Präsens für gleichzeitige Situationen, Perfekt statt Präteritum für vergangene Ereignisse (Walter 2020, S. 39).
- **Graphostilistik:** Einsatz von GroSSschreibung und Zeicheniterationen (z. B. “TOOOOR!“) als Äquivalent zu Lautstärke und Tonhöhe (Smuczyski 2020, S. 54; Walter 2020, S. 41).

Diese Merkmale definieren das Zielformat, auf das das LLM durch gezieltes Prompt Engineering konditioniert werden muss (vgl. Kap. 3.2). Sie bilden damit die linguistischen Anforderungen, die bei der Prompt-Konzeption in Kapitel 4.5.2 operationalisiert werden. Die linguistischen Anforderungen werden nicht separat im Interview validiert, sondern flieSSen als Operationalisierungsvorgaben in das Prompt Design ein (Kap. 4.5.2); ihre Wirksamkeit wird in der Qualitätsevaluation (Kap. 6.3.6) anhand der Dimension Verständlichkeit bewertet.

0.2.6 2.6 Zusammenfassung der Systemanforderungen

Aus den vorangegangenen Problemdimensionen und der Anforderungsanalyse werden im Folgenden die formalen Anforderungen an das System abgeleitet. Diese gliedern sich in funktionale Anforderungen (F), nicht-funktionale Anforderungen (N) und architektonische Anforderungen (A). Die Evaluation in Kapitel 6.6 überprüft jeden einzelnen Punkt gegen den implementierten Stand.

2.6.1 Funktionale Anforderungen

Nr.	Anforderung	Herleitung
F1	Drei Betriebsmodi (auto, coop, manual)	Kap. 1.1, 2.1
F2	KI-Textgenerierung für alle Event-Typen	Kap. 2.1, 2.5
F3	Drei Stilprofile (neutral, euphorisch, kritisch)	Kap. 2.3, 2.4
F4	Few-Shot-Prompting mit Stilreferenzen	Kap. 2.5, 3.2
F5	Ticker-Lifecycle (draft published / rejected)	Kap. 2.1
F6	Slash-Command-Parser für manuellen Modus	Kap. 2.1 (Status quo)
F7	Mehrsprachige Textgenerierung (Deutsch, Englisch; erweiterbar auf weitere Sprachen)	Kap. 2.2
F8	Idempotenter Datenimport via n8n	Kap. 2.1 (Robustheit)
F9	Provider-Fallback-Kette	Kap. 2.1 (Verfügbarkeits-anforderung im Echtzeit-Produktionsbetrieb)
F10	Deduplizierung von Ticker-Einträgen	Kap. 2.1 (Korrektheit)
F11	Pre-Match-Kontextgenerierung (Verletzungen, H2H, etc.)	Kap. 2.5 (Elaboration)
F12	Live-Statistik-Updates	Kap. 2.1 (Echtzeit)

2.6.2 Nicht-funktionale Anforderungen

Nr.	Anforderung	Herleitung
N1	Concurrency-Begrenzung für LLM-Aufrufe	Kap. 2.1 (Zeitdruck)
N2	Retry-Logik mit Rate-Limit-Erkennung	Kap. 3.1 (Robustheit)
N3	Transaktionale Testisolierung	Technische Qualitätssicherung (Echtzeit-Produktionssysteme erfordern Testisolierung aufgrund der Datenbankkopplung)
N4	TypeScript-Typsicherheit	Technische Qualitätssicherung (Wartbarkeit und Fehlervermeidung im Team)
N5	Responsive UI (Mobile-tauglich)	Kap. 2.1 (Einsatzszenarien: Stadion und Home-Office erfordern mobile Bedienbarkeit)

Nr.	Anforderung	Herleitung
N6	Fehlerresistenz im Frontend	Kap. 2.1 (Robustheit)

2.6.3 Architektur-Anforderungen

Nr.	Anforderung	Herleitung
A1	Drei-Schichten-Architektur	Kap. 2.1, 2.3 (Modularität)
A2	Repository-Pattern	Wartbarkeit, Testbarkeit; austauschbare Datenschichten ermöglichen unterschiedliche Backend-Implementierungen pro Mandant (vgl. Kap. 2.3)
A3	REST-API zur vollständigen Systemsteuerung und Mandantenkonfiguration	Kap. 2.3 (White-Label)
A4	Vollständiges Datenmodell	Kap. 2.1, 2.4
A5	White-Label-Fähigkeit (ef_whitelabel / generic)	Kap. 2.3, 2.4

Die drei Problemdimensionen — operativer Zeitdruck, Mehrsprachigkeit und White-Label-Bedarf — sind dabei keine isolierten Herausforderungen, sondern greifen strukturell ineinander: Zeitdruck macht menschliche Skalierung unmöglich, Mehrsprachigkeit multipliziert diesen Aufwand, und White-Label-Anforderungen verlangen eine stilistische Konsistenz, die wechselndes Personal nicht zuverlässig gewährleisten kann. Dies begründet die Notwendigkeit einer integrierten technologischen Lösung: KI-gestützte Textgenerierung mit konfigurierbarer Stilsteuerung (F1–F4) als Antwort auf den Zeitdruck, mehrsprachige Generierung (F7) als Antwort auf die Internationalisierungsanforderung, sprachgattungskonformes Prompt Design (F2, F4, F11) als Antwort auf die in Kapitel 2.5 hergeleiteten linguistischen Anforderungen, und eine konfigurierbare Instanzlogik (A5) als Antwort auf den White-Label-Bedarf. Die in den Tabellen 2.6.1–2.6.3 formalisierten 23 Anforderungen bilden damit das vollständige Pflichtenheft für die Systemkonzeption in Kapitel 4; die hierfür notwendigen technologischen Grundlagen werden in Kapitel 3 aufgearbeitet.

0.3 Kapitel 3 – Stand der Technik

0.3.1 3.1 Large Language Models

Die Entwicklung groSSer Sprachmodelle (Large Language Models, LLMs) hat die Verarbeitung natürlicher Sprache in den vergangenen Jahren grundlegend verändert. Den technischen Ausgangspunkt bildet die **Transformer-Architektur**, die Vaswani et al. (2017, S. 1) als

ein neuartiges Netzwerkdesign einführten, das ausschlieSSlich auf Aufmerksamkeitsmechanismen basiert. Das Modell verarbeitet Text autoregressiv — bei jedem Schritt werden die zuvor generierten Tokens als zusätzliche Eingabe für die Erzeugung des nächsten verwendet (Vaswani et al. 2017, S. 2). Diese Eigenschaft macht Transformer-basierte Modelle besonders geeignet für sequenzielle Textgenerierungsaufgaben wie die Liveticker-Produktion.

Diese groSSen Sprachmodelle werden zunächst auf umfangreichen Textkorpora vortrainiert und können anschlieSSend durch **Transfer Learning** — also die Übertragung erlernter Sprachstrukturen auf neue Aufgaben — ohne aufgabenspezifisches Fine-tuning eingesetzt werden (Brown et al. 2020, S. 2). Das ermöglicht den Einsatz von LLMs für spezialisierte Aufgaben wie die Liveticker-Generierung, ohne modellseitige Anpassungen vornehmen zu müssen.

Die empirische Grundlage für das In-Context Learning wurde mit **GPT-3** gelegt — einem autoregressiven Sprachmodell mit 175 Milliarden Parametern (Brown et al. 2020, S. 1). Das Prinzip, Modelle über Beispiele im Prompt zu konditionieren, ohne ihre Gewichte anzupassen, hat sich seitdem als Standard etabliert. Parallel entwickelten weitere Anbieter — darunter Meta (LLaMA-Familie), Anthropic (Claude) und Mistral — vergleichbare Architekturen, die in unterschiedlichen GröSSenordnungen und Lizenzmodellen verfügbar sind und die breite Adaption von LLMs für spezialisierte Aufgaben ermöglichen.

Für die Liveticker-Produktion sind **kompakte Modelle** besonders relevant. Die Generierungsaufgabe erfordert kurze Texte (1–3 Sätze) auf Basis strukturierter Eingabedaten (Spielereignisse, Kontext) — eine Aufgabe, die keine komplexe Inferenz oder umfangreiches Weltwissen voraussetzt. Kompakte Modelle bieten für solche kurzen, faktenbasierten Textgenerierungsaufgaben eine vergleichbare Qualität wie gröSSere Modelle, bei deutlich niedrigerer Latenz und geringeren Kosten — Eigenschaften, die für hochfrequente Echtzeit-Generierungsszenarien entscheidend sind (vgl. Kap. 4.5.1 für die konkrete Modellauswahl).

Trotz ihrer Leistungsfähigkeit weisen LLMs strukturelle Schwächen auf, insbesondere die Neigung zu **Halluzinationen** — also der Generierung von Inhalten, die sachlich unzutreffend oder erfunden sind (OpenAI 2023, S. 46). Im journalistischen Kontext ist dies kritisch, da faktische Fehler die Glaubwürdigkeit nachhaltig beschädigen (Bluhm & Schäfer 2023, S. 33). Dies begründet den **hybriden Ansatz** der vorliegenden Arbeit: Das System generiert Textvorschläge, die finale Freigabeentscheidung verbleibt jedoch beim Redakteur.

0.3.2 3.2 Prompt Engineering

Die Qualität der Ausgaben eines LLM hängt maSSgeblich vom **Prompt** ab. Prompt Engineering bezeichnet die systematische Gestaltung dieser Eingaben, um das Modell ohne Änderung seiner Gewichte zu einem gewünschten Verhalten zu lenken (Brown et al. 2020).

Das Prinzip des **In-Context Learning** sieht vor, dass das Modell auf eine Instruktion und ggf. Beispiele konditioniert wird.

Für die Anwendung von LLMs auf spezifische Aufgaben existieren grundsätzlich zwei Ansätze: **Fine-tuning** erfordert ein aufgabenspezifisches Nachtrainieren des Modells auf kuratierten Datensätzen und ist ressourcenintensiv (Rechenzeit, Infrastruktur, Datenkuration). **Prompting-basierte Ansätze** hingegen nutzen das vortrainierte Modell direkt und steuern dessen Verhalten ausschlieSSlich über die Eingabeformulierung (Brown et al. 2020, S. 1). Für ressourcenbeschränkte Kontexte wie den Vereinssport ist Prompting dabei nicht nur kostengünstiger, sondern auch methodisch vorzuziehen: Da kein domänenspezifischer Trainingsdatensatz in ausreichender GröSSe existiert, würde Fine-tuning zu Overfitting neigen, während Few-Shot-Prompting mit wenigen kuratierten Beispielen auskommt.

Innerhalb des Prompting-Paradigmas lassen sich zwei wesentliche Techniken unterscheiden:

1. **Zero-Shot Prompting:** Das Modell erhält ausschlieSSlich eine Aufgabenbeschreibung ohne Demonstrationsbeispiele (Brown et al. 2020, S. 13). Dieser Ansatz maximiert Flexibilität und eignet sich für generische Anwendungsfälle ohne spezifische Stilerwartungen.

2. **Few-Shot Prompting:** Dem Modell werden neben der Aufgabenbeschreibung zusätzlich wenige Beispiele (typischerweise 1–5) als Stilreferenz bereitgestellt. Diese Beispiele konditionieren das Modell auf eine gewünschte Tonalität, ohne dass ein modellseitiges Fine-tuning erforderlich ist (Brown et al. 2020, S. 14).

Ein weiterer zentraler Steuerungsparameter ist die **Temperature**, die die Wahrscheinlichkeitsverteilung der Token-Auswahl beeinflusst (Brown et al. 2020, S. 24). Niedrige Werte (z. B. 0,0–0,3) führen zu deterministischeren, faktentreueren Ausgaben; höhere Werte (z. B. 0,7–1,0) erhöhen die kreative Varianz. Für journalistische Anwendungen, bei denen faktische Korrektheit priorisiert wird, sind niedrige Temperature-Werte vorzuziehen.

Die in Kapitel 2.5 hergeleiteten linguistischen Anforderungen — Ellipsen, Tempora, Graphostilistik — definieren dabei das Zielformat, das über Few-Shot-Beispiele und explizite Stilanweisungen im Prompt operationalisiert wird. Die konkrete Umsetzung im vorliegenden System — einschlieSSlich der Prompt-Template-Struktur, Few-Shot-Datenquellen, Stilprofile und Temperature-Konfiguration — wird in Kapitel 4.5.2 beschrieben.

0.3.3 3.3 Natural Language Generation im Sportjournalismus

Die automatisierte Erzeugung natürlichsprachlicher Texte aus strukturierten Daten (Data-to-Text Generation) bildet die technische Grundlage für KI-gestützte Berichterstattungssys-

teme. Im Sportkontext bedeutet dies die Überführung von Spielereignissen und Statistiken in journalistisch verwertbare Texte (Puduppully & Lapata 2021, S. 510).

Der Sportjournalismus bietet für NLG-Systeme besonders günstige Voraussetzungen: Spielereignisse folgen standardisierten Mustern (Tore, Karten, Auswechslungen), liegen bereits in strukturierter Form vor und erfordern eine zeitnahe Verarbeitung. Diese Kombination aus strukturierten Eingabedaten und Echtzeitanforderung macht Sport zu einer der erfolgreichsten Anwendungsdomänen für Data-to-Text-Systeme (Puduppully & Lapata 2021, S. 510). Für kleinere Vereine und Medienunternehmen ermöglicht automatisierte Textgenerierung zudem eine skalierbare Berichterstattung, die mit rein manueller Redaktion nicht wirtschaftlich darstellbar wäre.

Historische NLG-Systeme basierten überwiegend auf **regelbasierten Template-Architekturen** (vgl. Puduppully & Lapata 2021, S. 520). Diese Systeme nutzen vordefinierte Satzmuster, in die Ereignisdaten eingesetzt werden (z. B. "[Spieler] erzielt in der [Minute] Minute das [Tor-Nr.] für [Team]"). Der Vorteil liegt in vorhersagbarer Qualität und vollständiger Kontrolle über die Ausgabe; der Nachteil ist stilistische Monotonie und begrenzte sprachliche Variabilität. Puduppully und Lapata (2021, S. 520) beschreiben diese stilistische Monotonie und begrenzte sprachliche Variabilität als strukturelles Defizit regelbasierter Systeme, insbesondere bei komplexeren narrativen Anforderungen.

Moderne **neuronale End-to-End-Ansätze** überwinden diese Grenzen durch datengetriebenes Lernen sprachlicher Muster. Statt starrer Regeln trainieren diese Modelle auf groSSen Textkorpora und erzeugen flüssigere, natürlichere Formulierungen. Im professionellen Roboterjournalismus erzeugt beispielsweise "Die Welt" bereits automatisierte Spielberichte via Retresco (Beils 2023, S. 207). Solche Systeme stoSSen jedoch bei unvorhergesehenen Ereignissen, unsauberer Datenlage oder der Notwendigkeit emotionaler Tonalität an Grenzen — insbesondere die Neigung zu Halluzinationen (vgl. Kap. 3.1) bleibt eine systemische Herausforderung für den journalistischen Einsatz.

Die Kombination beider Ansätze in einem **hybriden System** verbindet die Stärken: LLMs übernehmen die flexible, stilistisch variable Textgenerierung, während menschliche Redakteure die faktische Korrektheit absichern und editoriale Entscheidungen treffen. Dieser **Human-in-the-Loop-Ansatz** bildet die konzeptionelle Grundlage des vorliegenden Systems: Er adressiert die Qualitätsanforderungen des Journalismus durch redaktionelle Kontrolle und ermöglicht gleichzeitig — durch die Entlastung von der Texterstellung — eine Skalierung, die den in Kapitel 2.1 beschriebenen Ressourcenbeschränkungen kleinerer Redaktionen Rechnung trägt.

0.3.4 3.4 Echtzeit-Kommunikation im Web

Die Übertragung von Daten zwischen Frontend und Backend erfordert eine ausgewogene Balance zwischen Latenz, Ressourcenverbrauch und Implementierungskomplexität. Für Echtzeitanwendungen wie Liveticker stehen mehrere etablierte Kommunikationsmuster zur Verfügung, die sich hinsichtlich dieser Dimensionen unterscheiden.

HTTP-Polling ist das einfachste Muster: Der Client sendet in regelmäSSigen Intervallen Anfragen an den Server, um neue Daten abzurufen. Der Vorteil liegt in der Einfachheit und breiten Kompatibilität — das Muster nutzt Standard-HTTP-Requests ohne zusätzliche Protokollschichten. Nachteile sind die inhärente Latenz (Daten werden erst beim nächsten Polling-Zyklus sichtbar) und der erhöhte Ressourcenverbrauch durch wiederholte Request/Response-Zyklen, selbst wenn keine neuen Daten vorliegen. Für Anwendungen mit akzeptabler Latenztoleranz (z. B. wenige Sekunden) und begrenzter Updatefrequenz ist Polling jedoch eine robuste und wartbare Lösung.

Server-Sent Events (SSE) bieten als Zwischenlösung eine HTTP-basierte Push-Verbindung vom Server zum Client (Hickson 2015) — ressourceneffizienter als Polling, jedoch auf unidirektionale Kommunikation beschränkt.

WebSocket etabliert eine vollständig bidirektionale, persistente Verbindung über das WebSocket-Protokoll (RFC 6455). Nach einem initialen HTTP-Handshake wechselt die Verbindung in ein binäres Protokoll mit minimalen Framing-Overhead (Fette & Melnikov 2011, S. 1). WebSocket eignet sich für latenzkritische Echtzeitanwendungen, bei denen minimale Latenz und eine persistente Verbindung erforderlich sind — auch wenn die Kommunikation faktisch überwiegend unidirektional (Server Client) verläuft. RFC 6455 nennt Börsen-Ticker explizit als Einsatzszenario — eine strukturelle Parallele zum Sport-Liveticker. Der Preis für niedrige Latenz sind höhere Implementierungskomplexität und erhöhter Server-Ressourcenbedarf durch persistente Verbindungen. Clientseitig ist eine Reconnect-Strategie erforderlich, da Netzwerkunterbrechungen die persistente Verbindung unerwartet beenden können — üblich ist dabei ein exponentielles Backoff, um den Server bei wiederholten Verbindungsversuchen nicht zu überlasten.

Die Wahl des Kommunikationsmusters hängt von den Anforderungen der Anwendung ab: Polling bevorzugt Einfachheit, SSE bietet effizienten unidirektionalen Push, WebSocket minimiert Latenz bei bidirektionaler Kommunikation. Hybridansätze nutzen verschiedene Muster für unterschiedliche Datenströme innerhalb derselben Anwendung, um Trade-offs gezielt zu optimieren.

Die konkrete Umsetzung im vorliegenden System wird in Kapitel 4.6.5 beschrieben.

0.3.5 3.5 ETL-Prozesse und Workflow-Automatisierung

Die Datenbeschaffung in datengetriebenen Anwendungen folgt überwiegend dem **Extract-Transform-Load-Prinzip (ETL)**. Daten werden aus heterogenen Quellen extrahiert, in ein einheitliches Zielformat transformiert und in den Datenspeicher geladen (Freitas et al. 2025, S. 807). Für Sport-Liveticker bedeutet dies die Integration von Live-Spielereignissen, Statistiken, Aufstellungen und Mediendaten aus unterschiedlichen APIs oder Datenfeeds. Die Herausforderung besteht darin, diese Daten zeitnah, konsistent und in einer für die Anwendungslogik verwertbaren Struktur bereitzustellen.

Historisch wurden ETL-Prozesse durch monolithische Batch-Jobs oder direkt in Anwendungscode implementiert. Moderne Ansätze setzen zunehmend auf **Workflow-Orchestrierungsplattf** die ETL-Logik als konfigurierbare Prozesse abbilden. Diese Plattformen basieren typischerweise auf **Directed Acyclic Graphs (DAGs)**, die Datenflüsse als gerichtete, azyklische Graphen modellieren und damit Abhängigkeiten zwischen Verarbeitungsschritten explizit machen. Jeder Knoten im DAG repräsentiert eine Transformation oder einen API-Aufruf, während Kanten die Datenflussrichtung definieren (vgl. Venkiteela 2025, S. 3).

Low-Code- und No-Code-Plattformen haben diesen Ansatz weiter demokratisiert: Statt prozeduralen Code zu schreiben, werden Workflows grafisch zusammengestellt. Diese visuellen Editoren erleichtern sowohl die Entwicklung als auch die Wartung von Integrationspfaden. Für kleinere Teams oder prototypische Systeme bieten solche Plattformen eine erheblich niedrigere Einstiegshürde als vollprogrammierte ETL-Pipelines. Gleichzeitig erlauben sie bei Bedarf die Einbettung von Custom-Code-Blöcken für komplexe Transformationen (vgl. Venkiteela 2025, S. 2).

Innerhalb dieser Klasse lassen sich cloud-basierte iPaaS-Dienste (z. B. Zapier, Make) von self-hostbaren Open-Source-Plattformen (z. B. n8n) unterscheiden (vgl. Venkiteela 2025, S. 2). Cloud-Dienste bieten sofortige Verfügbarkeit ohne Infrastrukturaufwand, unterliegen jedoch Datenschutzeinschränkungen und eingeschränkter Kontrolle über Ausführungsumgebung und Kosten. Self-hostbare Plattformen wie n8n ermöglichen dagegen den Betrieb in der eigenen Infrastruktur, bieten volle Kontrolle über Datenflüsse (vgl. Nakavisute & Sincharoonsak 2025) und erlauben den direkten Datenbankzugriff — ein entscheidender Vorteil, wenn Workflows transaktionale Schreiboperationen direkt in die Persistenzschicht ausführen sollen, ohne den Umweg über HTTP-Endpunkte nehmen zu müssen (vgl. Venkiteela 2025, S. 4).

Für Echtzeitanwendungen wie Liveticker ist zusätzlich die **Latenz der Orchestrierung** kritisch. Während klassische Batch-ETL-Prozesse mit Stunden- oder Minutentakten arbeiten (vgl. Freitas et al. 2025, S. 808), müssen Sport-Events innerhalb von Sekunden verarbeitet werden. Das erfordert Orchestrierungsplattformen mit niedrigem Overhead, asynchroner Verarbeitung und effizienten Webhook-Mechanismen, um auf externe Ereignisse sofort reagieren zu können (vgl. Nakavisute & Sincharoonsak 2025).

Die konkrete Umsetzung der ETL-Architektur im vorliegenden System — einschlieSSlich der gewählten Orchestrierungsplattform, der Datenquellen (API-Football, ScorePlay, Social Media), der 15 definierten Workflows und der Webhook-basierten Triggerstruktur — wird in Kapitel 4.4 beschrieben.

0.3.6 3.6 Backend-Technologien

3.6.1 Python

Python hat sich als dominierende Programmiersprache für datenintensive Webanwendungen und KI-Systeme etabliert — ein Konsens, der sich in konsistenten Rankings von Entwickler-Umfragen (Stack Overflow Developer Survey 2024) und akademischen Studien zum KI-Ökosystem widerspiegelt. Die Sprache verbindet dynamische Typisierung mit einem umfangreichen Ökosystem an Bibliotheken für HTTP-Kommunikation, Datenbankzugriff und maschinelles Lernen. Darüber hinaus ermöglicht Pythons `asyncio`-Eventloop — auf dem ASGI-Frameworks wie FastAPI aufsetzen — die nicht-blockierende Verarbeitung paralleler I/O-Operationen, was für Echtzeitsysteme mit hoher externer API-Latenz entscheidend ist (vgl. Kap. 3.6.2).

3.6.2 ASGI und asynchrone Web-Frameworks

Das **ASGI-Protokoll** (Asynchronous Server Gateway Interface) ermöglicht im Gegensatz zum klassischen synchronen WSGI eine nicht-blockierende Verarbeitung auf Basis von Pythons `asyncio`-Eventloop (Grigorev 2019). ASGI-Frameworks wie **FastAPI** können während wartender I/O-Operationen (Datenbankabfragen, LLM-API-Aufrufe) andere Anfragen bearbeiten, was insbesondere für Anwendungen mit vielen externen Abhängigkeiten eine signifikant höhere Durchsatzrate ermöglicht. Das vorliegende System verwendet FastAPI als primäres ASGI-Framework; die Entscheidungsbegründung erfolgt in Kapitel 4.2.

3.6.3 ORM und Repository Pattern

Das **Object-Relational Mapping (ORM)** bildet Programmiersprachenklassen auf relationale Datenbanktabellen ab und abstrahiert den direkten SQL-Zugriff. **SQLAlchemy** implementiert dieses Muster für Python: Modellklassen werden über deklarative Klassendefinitionen auf Datenbanktabellen abgebildet, SQL-Abfragen über eine typsichere Fluent-API konstruiert. Das ergänzende **Repository Pattern** (Fowler 2002, S. 322) kapselt den Datenzugriff in dedizierte Klassen und bietet eine abstrahierte Schnittstelle zur Service-Schicht — das erleichtert Testbarkeit und ermöglicht den Austausch der Persistenzschicht ohne Änderungen an der Geschäftslogik.

Für die API-Schicht übernimmt **Pydantic** die typisierte Schema-Validierung: Eingabe- und Ausgabestrukturen werden als Python-Datenklassen mit Type-Annotations definiert, die Pydantic zur Laufzeit gegen eingehende Payloads prüft. FastAPI integriert Pydantic direkt in die Request/Response-Pipeline, sodass API-Vertrag und Validierungslogik in einer einzigen Klassendefinition ausgedrückt werden — ein entscheidender Vorteil für Systeme, in denen mehrere externe Datenquellen (n8n-Workflows, Frontend, Partner-API) unterschiedliche Feldformate liefern.

3.6.4 Relationale Datenbanken

Relationale Datenbankmanagementsysteme (RDBMS) organisieren Daten in Tabellen mit definierten Schemata und erzwingen referenzielle Integrität über Fremdschlüssel-Constraints (Codd 1970). RDBMS wie PostgreSQL garantieren **ACID-Eigenschaften** (Atomicity, Consistency, Isolation, Durability) und bieten für Anwendungen mit stark strukturierten, relational verknüpften Entitäten (Spiele, Teams, Ereignisse, Texte) Vorteile hinsichtlich Datenkonsistenz und Abfragekomplexität gegenüber dokumentbasierten Alternativen.

Die konkrete Backend-Architektur des vorliegenden Systems wird in Kapitel 4.2 beschrieben.

0.3.7 3.7 Frontend-Technologien

3.7.1 TypeScript

TypeScript erweitert JavaScript um ein statisches Typsystem, das Typfehler bereits zur Entwicklungszeit erkennt (Microsoft 2012). Der Compiler transpiliert den typisierten Quellcode in Standard-JavaScript. Für gröSSere Codebasen reduziert statische Typisierung die Fehlerquote und verbessert die Wartbarkeit durch IDE-Autocompletion und automatisierte Refactoring-Unterstützung.

3.7.2 Komponentenbasierte UI-Architekturen

React (Facebook, heute Meta, 2013) implementiert ein komponentenbasiertes Architekturmuster — die Benutzeroberfläche wird in wiederverwendbare, isolierte Bausteine zerlegt, die Markup, Logik und Styling kapseln. React nutzt eine deklarative, zustandsgesteuerte Rendering-Logik: Die Oberfläche wird als Funktion des Anwendungszustands beschrieben, und React übernimmt die effiziente DOM-Aktualisierung (Virtual DOM / Reconciliation). **React Hooks** (ab 16.8) ermöglichen die Verwaltung von Zustand und Seiteneffekten in funktionalen Komponenten; **Custom Hooks** erlauben die Extraktion wiederverwendbarer zustandsbehafteter Logik. Gegenüber vergleichbaren Frameworks wie Vue oder Angular

zeichnet sich React durch ein schlankes Kernmodell aus, das Zustandsverwaltung, Polling-Logik und WebSocket-Integration konsequent in Custom Hooks kapseln lässt — ein Vorteil für datenintensive Echtzeit-Oberflächen mit vielen unabhängigen Datensträngen.

3.7.3 Single-Page Applications

Single-Page Applications (SPAs) laden die Anwendungslogik beim initialen Seitenaufruf und aktualisieren die Darstellung dynamisch über JavaScript, ohne vollständige Seitenneuladen. Dieses Modell eignet sich für interaktive Werkzeuge mit häufigen Zustandsänderungen — wie Redaktionsoberflächen oder Echtzeit-Dashboards.

Die konkrete Frontend-Architektur des vorliegenden Systems wird in Kapitel 4.6 beschrieben.

0.3.8 3.8 Deployment und Containerisierung

3.8.1 Containerisierung

Docker (Merkel 2014) ermöglicht die Paketierung einer Anwendung mitsamt Laufzeitumgebung und Abhängigkeiten in portable, isolierte Container. Die zentrale Eigenschaft ist **Reproduzierbarkeit**: Dieselbe Anwendung verhält sich lokal identisch zur Produktionsumgebung.

3.8.2 Platform-as-a-Service (PaaS)

PaaS-Anbieter wie Render oder Heroku abstrahieren die Infrastrukturverwaltung und ermöglichen Deployments direkt aus Quellcode oder Container-Images. Für kleine Teams ohne dedizierte DevOps-Kapazitäten reduziert PaaS den operativen Aufwand erheblich.

3.8.3 Stateless Design und Skalierung

PaaS-Plattformen setzen typischerweise ein **Stateless-Design** voraus: Persistente Daten werden in externe Dienste ausgelagert, sodass bei erhöhter Last zusätzliche Instanzen ohne Zustandskonsistenz gestartet werden können (**horizontale Skalierung**).

Die konkrete Deployment-Architektur des vorliegenden Systems wird in Kapitel 4.7.4 beschrieben.

0.3.9 3.9 Synthese: Technologische Konsequenzen für das Systemdesign

Der Stand der Technik zeigt, dass für das vorliegende System kein einzelnes Werkzeug ausreicht, sondern ein abgestimmtes Zusammenspiel mehrerer Technologieschichten notwendig ist. Kompakte Sprachmodelle mit Few-Shot-Prompting (Kap. 3.1–3.2) liefern die erforderliche Textqualität bei vertretbarer Latenz und ohne ressourcenintensives Fine-tuning; der hybride NLG-Ansatz mit menschlicher Kontrollinstanz (Kap. 3.3) adressiert das systemimmanente Halluzinationsproblem von LLMs im journalistischen Kontext. Die in Kapitel 3.4 beschriebenen Kommunikationsmuster — Polling, SSE und WebSocket — zeigen unterschiedliche Trade-offs zwischen Implementierungseinfachheit und Latenz; welches Hybridmuster für den konkreten Anwendungsfall geeignet ist, wird auf Basis der Infrastrukturanforderungen in Kapitel 4.6.5 begründet. n8n als Low-Code-Orchestrierungsplattform (Kap. 3.5) entkoppelt die ETL-Komplexität vom Anwendungskern und ermöglicht eine schnelle Anpassung von Datenquellen unabhängig von Backend-Deployment-Zyklen. FastAPI auf ASGI-Basis (Kap. 3.6), React mit TypeScript (Kap. 3.7) und Docker-basiertes PaaS-Deployment (Kap. 3.8) vervollständigen einen Stack, der Produktionsfähigkeit, Wartbarkeit und Skalierbarkeit im ressourcenbeschränkten Vereinsumfeld vereint. Konkret begründen die Anforderungen aus Kapitel 2 folgende Technologieentscheidungen: Die in Kap. 2.3 formulierte Anforderung nach Datenschutz- und Betriebskonformität begründet die Wahl von n8n gegenüber cloudbasierten iPaaS-Diensten (vgl. Kap. 3.5); die Kostenrestriktion und Latenzanforderung aus Kap. 2.1 motiviert den Einsatz kompakter Sprachmodelle statt gröSSerer Generationssysteme (vgl. Kap. 3.1); die stark verknüpften Spielentitäten (Spiele, Teams, Ereignisse, Statistiken) begründen eine relationale Datenbankstruktur (Kap. 3.6.4) gegenüber dokumentbasierten Alternativen. Diese technologischen Grundlagen münden in die Systemkonzeption (Kapitel 4) und deren Implementierung (Kapitel 5).

0.4 Kapitel 4 – Systemkonzeption

0.4.1 4.1 Überblick und Schichtenmodell

Das System ist als **eigenständiger Cloud-Service** konzipiert, der alle Spiele der konfigurierten Wettbewerbe automatisch verarbeitet — von der Datenimportierung über die KI-Textgenerierung bis zur Publikation. Es ist eine serviceorientierte, dreischichtige Architektur, die Datenbeschaffung, Anwendungslogik und Präsentation klar voneinander trennt. Diese Schichtentrennung folgt dem etablierten Prinzip der *Separation of Concerns* (Parnas 1972), das die unabhängige Weiterentwicklung einzelner Systemteile ermöglicht.

Die **Datenbeschaffungs- und Automatisierungsschicht** wird über n8n umgesetzt. Sie integriert externe Datenquellen (insbesondere Football-API sowie Medienquellen wie ScorePlay und Social-Feeds), transformiert die Rohdaten in ein einheitliches internes Format und schreibt diese strukturiert in die Persistenzschicht beziehungsweise über definierte Backend-Endpunkte in das System ein. Die Entkopplung der Integrationslogik in n8n ermöglicht eine schnelle Anpassung von Import-Workflows, ohne den Anwendungskern ändern zu müssen.

Die **Anwendungs- und Persistenzschicht** basiert auf FastAPI (Python) und PostgreSQL. Sie stellt die REST-Schnittstellen bereit, verwaltet den Zustandsübergang der Ticker-Einträge (z. B. draft, published, rejected), orchestriert KI-Generierungsschritte und übernimmt die konsistente Speicherung aller Domänendaten (u. a. Teams, Wettbewerbe, Spiele, Events, Statistiken, Ticker- und Media-Daten).

Die **Präsentationsschicht** ist als White-Label-Frontend in React und TypeScript realisiert. Sie bildet den Redaktionsworkflow für Auswahl, Sichtung, Bearbeitung und Publikation ab und bleibt dabei mandantenfähig konfigurierbar (z. B. Instanz, Stil, Sprache, Branding). Technisch kombiniert das Frontend REST-basiertes Polling für Ticker- und Spieldaten mit einem dedizierten WebSocket-Kanal für Echtzeit-Medienupdates.

Die folgende Abbildung zeigt die drei Schichten mit ihren Technologien und Kommunikationspfaden:

```
graph LR
    classDef schicht fill:#EFF6FF,stroke:#3B82F6,stroke-width
      ↪ :1.5px,color:#1E3A8A
    classDef extern fill:#F9FAFB,stroke:#9CA3AF,stroke-width:1
      ↪ px,color:#374151
    classDef db fill:#F0FDF4,stroke:#22C55E,stroke-width:1.5px,
      ↪ color:#14532D

    subgraph EXT[' Externe Dienste ']
        FAPI['Football-API\napi-sports.io']
        LLM['LLM Provider\nOpenRouter']
        SP['ScorePlay\nMedia']
        EF['EF Spielerdaten\nprofis.eintracht.de']
    end

    subgraph AUTO[' Automatisierungsschicht ']
        N8N['n8n\n15 Workflows']
    end

    subgraph APP[' Anwendungs- & Persistenzschicht ']
```

```
        BE['FastAPI\nPython ů Uvicorn']
        DB[('PostgreSQL\n18 Tabellen')]
        BE <-->|'ORM'| DB
    end

    subgraph UI['  Präsentationsschicht  ']
        FE['React Frontend\nTypeScript']
    end

    N8N -->|'HTTP GET'| FAPI
    N8N -->|'HTTP GET/POST'| SP
    N8N -->|'HTTP GET'| EF
    N8N -->|'Zusammenfassungen'| LLM
    N8N -->|'REST POST /api/v1'| BE

    BE -->|'LLM-Aufrufe'| LLM

    FE -->|'REST Polling ů 5s'| BE
    FE -.->|'WebSocket /ws/media'| BE
    FE -->|'Webhook-Trigger'| N8N

    class FE schicht
    class BE schicht
    class N8N schicht
    class DB db
    class FAPI,LLM,SP,EF extern
```

4.1.1 Kommunikations- und Triggerarchitektur

Der Kommunikationsfluss folgt dem Prinzip **"read first, trigger if missing"**: Das Frontend liest zunächst den vorhandenen Datenbestand über REST; nur bei fehlenden Daten wird ein passender n8n-Webhook ausgelöst. Dadurch werden externe API-Aufrufe minimiert und redundante Importe vermieden.

```
sequenceDiagram
    participant FE as React Frontend
    participant BE as FastAPI Backend
    participant N8N as n8n
```

```
participant FAPI as Football-API

FE->>BE: GET /teams/countries
alt Keine Daten vorhanden
    FE->>N8N: Webhook import-countries
    N8N->>FAPI: GET /countries
    N8N->>BE: POST /api/v1/countries (Upsert)
    FE->>BE: GET /teams/countries (erneut)
end

FE->>BE: GET /matches (nach Teamauswahl)
FE->>N8N: Webhook import-prematch, import-lineups, ...
N8N->>BE: POST /api/v1/events, /api/v1/ticker/generate/...

loop Polling alle 5s (Events, Ticker, Match)
    FE->>BE: GET /api/v1/ticker/{match_id}
    BE-->>FE: TickerEntry[] (inkl. neuer Entwürfe)
end

N8N->>BE: POST /api/v1/media/incoming
BE-->>FE: WebSocket /ws/media  neue Medien
```

Die Systemkopplung erfolgt über drei klar getrennte Schnittstellentypen:

- **REST-API** — stabiler Kern für alle Lese- und Schreiboperationen: Navigation, Ticker-CRUD, Statusübergänge (`draft published rejected`), manuelle Eingaben.
- **n8n-Webhooks** — bedarfsgesteuerte Trigger für drei Klassen:
  1. **Initialisierungs-Trigger** — Aufbau fehlender Stammdaten (Länder, Teams, Wettbewerbe, Spiele)
  2. **Match-Kontext-Trigger** — spielbezogene Importe nach Matchauswahl (Events, Lineups, Statistiken, Prematch)
  3. **Generierungs-Trigger** — AnstoSS von KI-Textprozessen (Event-basiert, Matchphasen, Zusammenfassungen)
- **WebSocket `/ws/media`** — Echtzeit-Push für latenzkritische Medieninhalte (ScorePlay-Bilder, Social-Media-Clips); Ticker- und Matchdaten bleiben bewusst im Polling-Modell.

Diese hybride Triggerarchitektur unterstützt sowohl redaktionelle Kontrolle im Sinne eines **Human-in-the-Loop**-Ansatzes als auch weitgehend automatisierte Live-Prozesse.

4.1.2 Partner-Team-Konzept und White-Label-Steuerung

Das System unterscheidet zwischen dem **Partner-Team** — dem Verein, für den die White-Label-Instanz konfiguriert ist — und den jeweiligen Gegnern. Diese Unterscheidung wird über ein konfigurierbares Team-Keyword umgesetzt, das sowohl in den n8n-Workflows als auch im Frontend gegen den Teamnamen abgeglichen wird.

Auf Basis dieser Erkennung werden zwei Verarbeitungspfade gesteuert:

- **Instanz und Stil**: Partner-Team-Spiele verwenden den `ef_whitelabel`-Instanzpfad mit euphorischem Stil; alle anderen Spiele den `generic`-Pfad mit neutralem Stil.
- **UI-Anpassung**: Das Frontend blendet vereinsspezifische Bereiche (Social-Media-Panels, Torjubel-Videos) ausschlieSSlich bei erkannten Partner-Team-Spielen ein.
- **Erweiterte Kontextdaten**: Für Tormomente des Partner-Teams werden zusätzlich Spielerdaten und Torjubel-Video-URLs aus einer externen Vereinsquelle in den Prompt-Kontext eingebettet.

Das Datenbankschema enthält ergänzend ein `is_partner_team`-Flag, das für zukünftige Erweiterungen vorgesehen ist. Die konkreten Implementierungen sind in Kapitel 5.2.2 (Datenbankmodell, `is_partner_team`-Flag) und Kapitel 5.4.6 (Frontend-Moduslogik, `isOurTeam` ↪ -Erkennung) dokumentiert.

0.4.2 4.2 Backend-Konzeption

4.2.1 Framework-Wahl: FastAPI

Als Backend-Framework wurde FastAPI gewählt (vgl. Abschnitt 3.6.2 für die technische Einordnung). Die Entscheidung begründet sich durch drei systemspezifische Anforderungen: Erstens erfordert die parallele Verarbeitung von LLM-Aufrufen, externen API-Abfragen und Medienverarbeitung native Unterstützung für asynchrone I/O. Zweitens vereinfacht die automatisch generierte OpenAPI-Spezifikation die Integration mit n8n-Workflows erheblich, da Endpunkte über Swagger UI direkt testbar sind. Drittens reduziert die enge Pydantic-Integration Schnittstellenfehler zwischen Frontend, Backend und n8n, indem alle Eingaben gegen typisierte Schemas validiert werden.

4.2.2 Interne Struktur und Datenzugriff

Die interne Struktur folgt einer klaren Trennung aus API-Routern, Repository-Schicht (vgl. Kap. 3.6.3) sowie **SQLAlchemy 2.0**-ORM-Modellen und Pydantic-Schemas. Die Router bilden die HTTP-Schnittstelle und übernehmen Validierung, Statuscodes sowie die Orchestrierung einzelner Use Cases. Datenbankzugriffe sind in dedizierten Repositories gekapselt. Dadurch bleibt die API-Schicht frei von SQL-Details, während Persistenzlogik zentral gebündelt und wiederverwendbar gehalten wird.

Für LLM-nahe Abläufe wird ergänzend eine Service-Schicht genutzt (insbesondere `ticker_service` ↪ und `llm_service`). Die Service-Schicht wird dabei gezielt, aber nicht flächendeckend eingesetzt: Viele Standard-CRUD- und Listenoperationen folgen dem Muster Router Repository, während komplexere Generierungs- und Orchestrierungspfade über Services laufen.

Für die Datenrepräsentation gilt eine bewusste Trennung: ORM-Modelle definieren die Persistenzstruktur, Pydantic-Schemas die API-Verträge. Das stabilisiert die Schnittstellen gegenüber internen Refactorings und verbessert Wartbarkeit und Testbarkeit.

4.2.3 API-Design und Endpunktstrategie

Die Backend-Schnittstelle ist als versionierte REST-API unter `/api/v1` ausgelegt und orientiert sich am Architekturstil **Representational State Transfer** (Fielding 2000, Kap. 5). Endpunkte sind ressourcenorientiert strukturiert (z. B. `teams`, `matches`, `ticker`, `media` ↪ , `clips`) und folgen konsistenten Benennungs- und Methodenregeln. `GET` dient lesenden Abfragen, `POST` dem Erzeugen bzw. Triggern, `PATCH` partiellen Zustandsänderungen und `DELETE` kontrollierten Löschoperationen. Insbesondere im Ticker-Kontext bildet `PATCH` den redaktionellen Lebenszyklus (`draft` `published` bzw. `rejected`) explizit ab.

Konzeptionell wird zwischen fachlichen Datenendpunkten und prozessualen Triggerendpunkten getrennt. Dadurch bleiben Leseoperationen seiteneffektfrei, während Import- und Generierungsprozesse gezielt ausgelöst werden können. Fehlerfälle werden konsistent über HTTP-Statuscodes und strukturierte Fehlermeldungen kommuniziert, sodass Frontend und n8n-Workflows deterministisch reagieren können.

4.2.4 Asynchronität, Nebenläufigkeit und Performance

Die Laufzeitkonzeption setzt Asynchronität gezielt auf I/O-intensiven Pfaden ein, insbesondere bei LLM-bezogenen Generierungsrouten, Media-Verarbeitung und WebSocket-Kommunikation. Das erhöht die Reaktionsfähigkeit unter paralleler Last, da Wartezeiten auf externe Dienste keine Worker dauerhaft blockieren.

Die Anzahl gleichzeitiger LLM-Aufrufe wird über einen konfigurierbaren **Semaphore-Mechanismus** begrenzt (Standard: 8 parallele Anfragen). Diese Begrenzung reduziert Lastspitzen und verringert die Wahrscheinlichkeit von Rate-Limit-Problemen seitens der LLM-Provider. Der Semaphore wirkt dabei pro Prozessinstanz — bei horizontaler Skalierung steigt die effektive Gesamtkonkurrenz entsprechend der Anzahl laufender Instanzen. Ergänzend greift ein **Retry-Mechanismus mit Backoff**: Bei Rate-Limit-Fehlern erfolgen automatische Wiederholungsversuche mit linearem Backoff (3 Versuche, 30 s / 60 s / 90 s); bei sonstigen transienten Fehlern exponentielles Backoff (1 s / 2 s / 4 s).

Die Implementierung ist bewusst hybrid aus synchronen und asynchronen Pfaden aufgebaut. Nicht alle Endpunkte sind vollständig asynchronisiert; stattdessen wurde ein pragmatischer Kompromiss zwischen Performance, Komplexität und Wartbarkeit gewählt. Langlaufende Integrationslogik ist in n8n ausgelagert, wodurch der API-Server als transaktionaler Kern entlastet wird.

4.2.5 Fehlerbehandlung, Robustheit und Betriebsaspekte

Für den Live-Betrieb wurde das Backend auf robuste Fehlerbehandlung und kontrollierte Degradation ausgelegt. Fehler werden über konsistente HTTP-Antworten und strukturierte Meldungen zurückgegeben, sodass Frontend und n8n-Workflows differenziert reagieren können.

Die Datenpersistenz ist auf **idempotente Verarbeitung** ausgelegt (vgl. Kap. 4.3.4 für Details zu Upsert-Strategien und Schlüsseldesign). Bei Teilausfällen externer Dienste bleibt das System eingeschränkt funktionsfähig — bereits persistierte Daten bleiben verfügbar, redaktionelle Kernprozesse (z. B. manuelle Tickererstellung) können fortgeführt werden.

4.2.6 Sicherheitskonzept

Das System adressiert Sicherheit auf drei Ebenen, wobei der Projektrahmen einer Bachelorarbeit eine bewusste Priorisierung erfordert:

Transportebene: Die gesamte Kommunikation zwischen Frontend, Backend und externen Diensten erfolgt über HTTPS. Die Render-Plattform übernimmt die TLS-Terminierung und Zertifikatsverwaltung automatisiert.

API-Absicherung: Der Backend-Server konfiguriert eine **CORS-Whitelist** (Cross-Origin Resource Sharing), die Anfragen ausschlieSSlich von autorisierten Frontend-Ursprüngen akzeptiert. Die API-Schlüssel externer Dienste (LLM-Provider, API-Football, ScorePlay) werden über Umgebungsvariablen injiziert und sind weder im Quellcode noch im Frontend exponiert. Die Pydantic-basierte Eingabevalidierung schützt zusätzlich vor Injection-

Angriffen, da sämtliche Request-Payloads gegen typisierte Schemas geprüft werden, bevor sie die Geschäftslogik erreichen.

Authentifizierung und Autorisierung: Eine feingranulare Benutzerauthentifizierung mit Rollenkonzept (z. B. Redakteur, Administrator) ist im aktuellen Stand **nicht implementiert**. Das System ist als internes Redaktionswerkzeug konzipiert und setzt auf Netzwerksegmentierung statt individueller Zugangskontrolle. Für einen produktiven Mehrbenutzerbetrieb wäre die Integration eines Authentifizierungsframeworks (z. B. OAuth 2.0 oder JWT-basierte Token-Authentifizierung) erforderlich — dies wird in Kapitel 7 als Erweiterungsperspektive eingeordnet.

0.4.3 4.3 Datenbankkonzeption

Die Persistenzschicht basiert auf PostgreSQL und umfasst in der aktuellen Fassung **18 Tabellen** (17 ORM-Modelle sowie eine schlüsselwertbasierte `settings`-Tabelle, die ausschlieSSlich über eine Alembic-Migration verwaltet wird). Die Wahl eines relationalen Systems gegenüber dokumentenorientierten Ansätzen (z. B. MongoDB) begründet sich durch die stark strukturierten Domänendaten: FK-Beziehungen zwischen Teams, Wettbewerben, Spielen, Events und Ticker-Einträgen sind klar definiert und referenziell integer zu halten. JSONB-Felder werden selektiv für semistrukturierte Daten (z. B. Statistik-Rohdaten) eingesetzt, sodass die ACID-Garantien relationaler Datenbanken erhalten bleiben. Das Schema ist auf einen stabilen Live-Betrieb mit wiederholbaren Importen, klaren Zustandsübergängen und nachvollziehbaren Redaktionsentscheidungen ausgelegt.

4.3.1 Schemadesign-Prinzipien

Die folgende Abbildung zeigt die zentralen Entitäten und ihre Beziehungen:

```
erDiagram
    countries ||--o{ teams : 'hat Teams'
    teams ||--o{ competition_teams : 'spielt in'
    competitions ||--o{ competition_teams : 'hat Teams'
    competitions ||--o{ matches : 'umfasst'
    seasons ||--o{ matches : 'enthält'
    teams ||--o{ matches : 'home_team'
    teams ||--o{ matches : 'away_team'

    matches ||--o{ events : 'hat Events'
    matches ||--o{ synthetic_events : 'hat synth. Events'
    matches ||--o{ ticker_entries : 'hat Ticker'
```

```
matches ||--o{ lineups : 'hat Lineup'
matches ||--o{ match_statistics : 'hat Statistiken'

events ||--o{ ticker_entries : 'event_id (nullable)'
synthetic_events |o--o{ ticker_entries : '
  ↪ synthetic_event_id (nullable)'

players ||--o{ lineups : 'spielt'
players ||--o{ player_statistics : 'hat Statistiken'

seasons ||--o{ standings : 'hat Standings'
competitions ||--o{ standings : 'hat Standings'
teams ||--o{ standings : 'Rang'
```

Hinweis: Die Tabellen `media_queue`, `media_clips`, `style_references` und `settings` sind als eigenständige Entitäten ohne Fremdschlüsselbeziehungen modelliert und daher im ER-Diagramm nicht abgebildet. `style_references` dient als Few-Shot-Datenquelle für die LLM-Promptgenerierung; `media_queue` wird applikationsseitig über die Media-Endpunkte verwaltet.

Das Schema folgt einem hybriden Entwurfsansatz aus strukturierter Normalisierung und gezielter Schemaflexibilität:

1. **Normalisierte Kernentitäten** — Zentrale Domänenobjekte wie `teams`, `matches`, `events`, `ticker_entries`, `players`, `competitions` und Zuordnungstabellen sind in dritter Normalform modelliert (Codd 1970). Dadurch bleiben Beziehungen, Integrität und Auswertbarkeit im Live-Betrieb stabil.

2. **Gezielter JSONB-Einsatz** — JSONB wird bewusst nur dort eingesetzt, wo sich Strukturen dynamisch ändern oder kontextabhängig sind (z. B. `synthetic_events.`↪`data`, sowie mehrere Kontextfelder in `matches`). Die Statistiktabellen (`match_statistics`↪, `player_statistics`) sind dagegen bewusst stark typisiert über explizite Spalten modelliert, um konsistente Abfragen und robuste Aggregationen zu ermöglichen.

3. **Schlüsselstrategie: Integer-PK + UUID-Fachschlüssel** — Das Schema verwendet numerische Primärschlüssel (`id`) für performante Joins und Fremdschlüsselbeziehungen. Zusätzlich existiert in mehreren Kernentitäten ein eindeutiges `uid`-Feld (UUID) als stabiler externer Referenzschlüssel. Es handelt sich damit nicht um ein reines UUID-Primary-Key-Schema, sondern um eine kombinierte Strategie.

4.3.2 Status-Lifecycle der Ticker-Einträge

Jeder Ticker-Eintrag in `ticker_entries` folgt einem klaren Statusmodell mit drei Zuständen:

- **`draft`**: Entwurf, noch nicht freigegeben
- **`published`**: redaktionell oder automatisch veröffentlicht
- **`rejected`**: verworfen, bleibt zur Nachvollziehbarkeit erhalten

```
stateDiagram-v2
    [*] --> draft : KI generiert (coop)\nauto_publish=false
    [*] --> published : KI generiert (auto)\nauto_publish=true
    [*] --> published : Manuell\nPOST /ticker/manual

    draft --> published : Redakteur bestätigt\nPATCH status=
       ↪ published
    draft --> rejected  : Redakteur verwirft\nPATCH status=
       ↪ rejected
    published --> draft : PATCH status=draft

    note right of rejected : Bleibt erhalten\n(Auswertbarkeit)
```

Eine Rückstufung von `published` auf `draft` ist über `PATCH status=draft` möglich und wird insbesondere bei Re-Triggern von Matchphasen-Workflows genutzt (vgl. Kap. 5.5.3). `rejected` ist ein terminaler Zustand — ein verworfener Eintrag kann nicht reaktiviert werden.

Ergänzend markiert das Feld `source` die Herkunft (`ai` vs. `manual`) und erlaubt eine saubere Trennung für Evaluationen (z. B. ausschlieSSlich KI-generierte Einträge).

> **Hinweis für die Methodik**: Im aktuellen Schema ist kein separates `published_at` ↪ -Feld vorhanden. Persistiert wird `created_at` des jeweiligen Ticker-Eintrags. Für sekundengenaue Time-to-Publish-Analysen sind daher zusätzliche Messzeitpunkte im Evaluationsdatensatz bzw. in der Messlogik erforderlich, statt sich allein auf den Tabellenzustand zu stützen.

4.3.3 Die drei Betriebsmodi

Das Feld `ticker_mode` in `matches` steuert das Verhalten der Generierungspipeline pro Spiel und ist zur Laufzeit umschaltbar über `PATCH /api/v1/matches/{id}/ticker-mode`. Es sind drei Modi implementiert:

1. **`auto`** (vollautomatisch) — Triggerkette läuft ohne redaktionellen Eingriff; KI-Ergebnisse werden direkt veröffentlicht.

2. **`coop`** (kooperativ / Human-in-the-Loop) — KI erzeugt Entwürfe (`draft`), die Redaktion entscheidet über Veröffentlichung oder Ablehnung. Dieser Modus bildet das primäre Zielbild des Systems.

3. **`manual`** (rein redaktionell) — Einträge werden manuell erstellt; KI-Generierung ist nicht leitend für den Veröffentlichungsprozess. Im `manual`-Modus werden keine KI-Generierungsendpunkte getriggert; eingehende Events lösen ausschlieSSlich die Datenpersistenz aus, nicht die LLM-Pipeline. Dieser Modus dient als Referenz für Vergleiche in der Evaluation.

Der Moduswechsel ist als Laufzeitparameter auf Datenbankebene abgebildet; der Statusfluss ist im State-Diagramm in Abschnitt 4.3.2 dargestellt.

4.3.4 Integrität, Idempotenz und Auswertbarkeit

Für den Zusammenschluss aus n8n-Workflows, externen APIs und Backend-Routen ist die Datenbank auf idempotente Verarbeitung ausgelegt:

1. **Konfliktarme Wiederholbarkeit** — Wiederholte Imports führen durch Unique-Constraints und Upsert-Strategien nicht zu unkontrollierten Duplikaten.

2. **Eindeutige Zuordnung in zentralen Tabellen** — Eindeutige Schlüssel (z. B. `source_id` pro Ereignisquelle) sichern die Nachvollziehbarkeit importierter Ereignisse.

3. **Referenzielle Integrität über FK-Regeln** — Lösch- und Nullsetz-Strategien (`CASCADE` ↪ , `SET NULL`) sorgen dafür, dass abhängige Daten konsistent bleiben und Historie dort erhalten wird, wo sie fachlich relevant ist.

4. **Auswertungskompatibilität** — Die Kombination aus `ticker_mode`, `status`, `source` ↪ , Spielbezug und Zeitfeldern schafft die Grundlage für reproduzierbare Vergleichsauswertungen zwischen manuellen, kooperativen und automatisierten Abläufen.

0.4.4 4.4 Workflow-Konzeption (n8n)

Die Workflow-Schicht ist als entkoppelte Orchestrierungsebene zwischen externen Datenquellen und Backend ausgelegt. n8n wurde gegenüber Alternativen wie einem Custom-Scheduler (z. B. Celery, APScheduler) oder cron-basierten Skripten gewählt, weil es Workflow-Logik

ohne Codeänderungen am Backend anpassbar macht, eine visuelle Debugging-Oberfläche bietet und HTTP-Trigger sowie Webhook-Empfang ohne zusätzliche Infrastruktur unterstützt. n8n übernimmt dabei API-Aufrufe, Transformation, Persistenzvorbereitung und Triggersteuerung, während das FastAPI-Backend als transaktionaler Kern und Integrationspunkt für Frontend und KI-Generierung fungiert.

4.4.1 Workflow-Klassen und Verantwortlichkeiten

Alle Generierungs-Workflows folgen einem konsistenten Vier-Phasen-Muster: **(1) Webhook-Trigger** — ein HTTP-Aufruf löst den Workflow aus; **(2) SQL-Operation** — ein Upsert-Statement persistiert die Daten mit Konfliktbehandlung (`ON CONFLICT`); **(3) Filter-Knoten** — nur neue, nicht bereits vorhandene Zeilen (erkennbar an der zurückgegebenen Datenbank-`id`) passieren weiter; **(4) Backend-Call** — der entsprechende FastAPI-Endpunkt wird pro neuer Zeile aufgerufen, bei Generierungsworkflows zum LLM-Trigger. Dieses Muster garantiert Idempotenz: Mehrfachaufrufe desselben Events erzeugen weder Datenduplikate noch redundante LLM-Aufrufe.

Die n8n-Landschaft gliedert sich in vier funktionale Klassen. Die folgende Übersicht zeigt alle 15 Workflows und ihren Auslöser:

Gruppe	Workflows	Trigger
A) Stammdaten	01–04: Countries, Teams, Competitions, Matches	Bei leeren Daten
B) Matchdaten	05–07: Lineups, Match-/Player-Statistics, Prematch	Nach Matchauswahl
C) KI-Generierung	09 Events-LLM, 13 Halftime/Aftertime, 14 Anpfiff/Abpfiff	Modus- und Event-gesteuert
D) Medien & Social	08 ScorePlay, 10 Twitter/X, 11 YouTube, 12 Instagram	Medien-Suche

Diese Trennung reduziert Kopplung, erleichtert Fehlersuche und erlaubt es, einzelne Teilprozesse unabhängig anzupassen. Die konkreten Workflow-Dateinamen und Implementierungsdetails sind in Kapitel 5.5 dokumentiert.

4.4.2 Workflow-Grenzen und projektspezifische Besonderheiten

Die Workflow-Landschaft ist funktional umfassend, enthält aber bewusst pragmatische Projektentscheidungen:

1. **Teilweise saisonspezifische Fixierung** — In einzelnen Flows ist die Saisonlogik auf 2025 begrenzt.

2. **Hybride Aktualisierungslogik** — Nicht alle Datenpfade sind eventgetrieben; ein Teil wird durch Polling und bedarfsgesteuerte Trigger ergänzt.

3. **Instanzspezifische Spezialpfade** — Einige LLM-/Medienprozesse enthalten EF-spezifische Logik, was für eine vollständig generalisierte White-Label-Fähigkeit parametriert werden müsste.

In Summe bildet n8n eine tragfähige Orchestrierungsschicht, die externe Datenquellen, interne Persistenz und KI-generierte Inhalte in einem modularen, nachvollziehbaren und erweiterbaren Ablauf zusammenführt.

0.4.5 4.5 KI-Komponente

4.5.1 Multi-Provider-Architektur

Die KI-Komponente ist als providerunabhängige Abstraktionsschicht implementiert. Der LLM-Dienst kapselt mehrere Anbieter hinter einer einheitlichen Schnittstelle und unterstützt aktuell OpenAI, Anthropic, Google Gemini, OpenRouter sowie einen Mock-Modus für Entwicklungs- und Fallbackszenarien ohne API-Key.

Primärzugang: OpenRouter als API-Gateway

Das vorliegende System setzt primär **OpenRouter** als LLM-Gateway ein. OpenRouter fungiert als einheitliche API-Schnittstelle, über die mit einem einzelnen API-Schlüssel verschiedene Sprachmodelle unterschiedlicher Anbieter angesprochen werden können. Diese Architekturentscheidung bietet drei Vorteile:

1. **Kein Vendor Lock-in**: Der Wechsel zwischen Modellen erfordert lediglich eine Änderung der Umgebungsvariable `OPENROUTER_MODEL`, keine Codeanpassung.

2. **Vergleichbarkeit**: Verschiedene Modelle können auf demselben Prompt evaluiert werden (vgl. Abschnitt 6.3.3).

3. **Kostenoptimierung**: Das jeweils kostengünstigste Modell für die Aufgabe kann gewählt werden.

Im produktiven Einsatz wird aktuell **Gemini 2.0 Flash Lite** (Modell-ID `google/gemini-2.0-flash-lite-001`) über OpenRouter genutzt. Die Wahl dieses kompakten Modells begründet sich durch die Aufgabencharakteristik (vgl. Kap. 3.1): Liveticker-Texte sind kurz und faktenbasiert — eine Aufgabe, für die kompakte Modelle eine vergleichbare

Qualität bei deutlich niedrigerer Latenz erzielen. Für Vereinsredaktionen mit begrenztem Budget ist dieser Kostenvorteil ein entscheidender Faktor für die Praxistauglichkeit des Systems.

Fallback-Kette und Mock-Provider

Ergänzend implementiert das Backend eine **Fallback-Kette** für den Fall, dass OpenRouter nicht konfiguriert ist. Das System prüft zur Startzeit in der festen Prioritätsreihenfolge, ob ein API-Schlüssel hinterlegt ist, und aktiviert den ersten verfügbaren Provider als Singleton:

```
openrouter gemini openai anthropic mock
```

Ist kein Schlüssel konfiguriert, wird auf einen regelbasierten **Mock-Provider** zurückgefallen, der Template-basierte Texte ohne LLM-Aufruf erzeugt. Diese Architektur stellt sicher, dass das System auch ohne LLM-Anbindung lauffähig bleibt — etwa für Entwicklung und Tests.

Standardmodelle der Provider sind:

Provider	Standardmodell
OpenRouter	`google/gemini-2.0-flash-lite-001`
Gemini	`gemini-2.0-flash-lite-001`
OpenAI	`gpt-4o-mini`
Anthropic	`claude-haiku-4-5-20251001`

Zusätzlich kann die Auswahl in Generierungsrouten pro Request über `provider` und `model` überschrieben werden, was insbesondere für Evaluationsvergleiche zwischen Modellen genutzt wird (vgl. Abschnitt 6.3.3).

4.5.2 Prompt-Engineering-Strategie

Die Generierung verwendet ein template-basiertes Prompting mit modularen Bausteinen, die dynamisch zusammengesetzt werden:

1. Rolleninstruktion (Persona, Aufgabe, Gattung)
2. Stilinstruktion (neutral / euphorisch / kritisch)
3. Faktenblock zum Ereignis (Typ, Detail, Minute, beteiligte Spieler/Team)
4. Dynamischer Kontextblock mit relevanten Matchinformationen
5. Optionaler Few-Shot-Block mit Stilbeispielen

6. Regelblock mit formativen und inhaltlichen Einschränkungen

```
flowchart TD
    A['Ereignis-Typ\n(z.B. goal)'] --> P
    B['Fakten\n(Spieler, Minute, Team)'] --> P
    C['Match-Kontext\n(Spielstand, Teams, Liga)'] --> P
    D['Few-Shot-Beispiele\naus style_references\n(03 Texte)']
      ↪ --> P
    E['Regelblock\n(Länge, Sprache, kein Markdown)'] --> P
    F['Stilinstruktion\nneutral / euphorisch / kritisch'] --> P

    P['Vollständiger\nSystem-Prompt'] --> LLM['LLM\n(temp=0.3)
      ↪ ']
    LLM --> T['Generierter\nTicker-Text']
```

Der Few-Shot-Block wird aus der Tabelle `style_references` gespeist, die manuell kuratierte Original-Tickertexte von Eintracht Frankfurt enthält. Pro LLM-Aufruf werden bis zu drei zufällige Referenzen selektiert, gefiltert nach `event_type`, `instance` und optional `league`. Durch die Randomisierung wird eine monotone Reproduktion vermieden, während der stilistische Korridor gewahrt bleibt. Für Pre-Match-Typen enthält der Prompt zusätzliche harte Restriktionen, um Live-Szenen-Halluzinationen zu vermeiden und die Ausgabe auf Vorschau- und Analyseinhalte zu begrenzen.

Der Inferenzparameter **Temperature** wird für die Textgenerierung auf `0.3` festgelegt, um die faktische Korrektheit gegenüber kreativer Varianz zu priorisieren. Für Übersetzungsaufgaben wird ein noch niedrigerer Wert von `0.1` verwendet, um semantische Abweichungen vom Originaltext zu minimieren.

4.5.3 Stilprofile und Instanzspezifik

Das System unterstützt drei zentrale Stilprofile:

- **`neutral`**: sachlich, ausgewogen, ohne Vereinspräferenz
- **`euphorisch`**: emotional und fan-nah
- **`kritisch`**: analytisch und bewertend

Ergänzend steuert die Instanzkonfiguration (`generic` vs. `ef_whitelabel`) die Tonalität. Für `ef_whitelabel` werden vereinsspezifische Stilbeispiele aus `style_references` als Few-Shot-Kontext eingebunden; die generische Instanz kann ohne vereinsgebundene Stilprägung betrieben werden.

4.5.4 Mehrsprachigkeit

Die Mehrsprachigkeit ist als First-Class-Parameter der LLM-Pipeline implementiert: Die Zielsprache (`language`, Standard `de`) wird direkt an den Prompt übergeben, sodass der Text in der gewünschten Sprache generiert wird — nicht nachträglich übersetzt. Dieses Vorgehen nutzt die Fähigkeit moderner groSSer Sprachmodelle, idiomatische Texte ohne den Zwischenschritt einer maschinellen Übersetzung zu produzieren (vgl. Kap. 3.1). Die Generierungsendpunkte akzeptieren jeden ISO-639-1-Sprachcode; getestet sind Deutsch (`de`), Englisch (`en`) und Japanisch (`ja`) — die drei in Kapitel 2.2 als relevant identifizierten Sprachen.

Für bestehende Einträge steht eine Batch-Übersetzung über `POST /api/v1/ticker/translate-batch/{match_id}` zur Verfügung. Sie übersetzt alle veröffentlichten KI-Einträge eines Spiels mit reduzierter Temperatur (`0.1`), um möglichst deterministische Ergebnisse zu erzielen. Der aktuelle Implementierungsstand schlieSSt dabei auch manuell erstellte Einträge ein — eine Einschränkung, die in Kapitel 6.7 dokumentiert ist.

Eine verbleibende Limitation betrifft die Few-Shot-Referenzen: Die `style_references`-Tabelle enthält ausschlieSSlich deutschsprachige Beispieltexte. Bei nicht-deutschen Generierungen fehlt damit die stilprägende Few-Shot-Komponente, was die Qualität der Tonalität gegenüber dem deutschen Modus leicht abschwächen kann. Eine Erweiterung der Stildatenbank um englisch- und japanischsprachige Referenzen wäre der naheliegende Folgeschritt (vgl. Kap. 8.3.2).

0.4.6 4.6 Frontend-Konzeption

4.6.1 Architekturprinzipien

Das Frontend ist als React-Client mit TypeScript und klarer Trennung zwischen UI-Komponenten, datenbezogenen Hooks und API-Zugriffsschicht umgesetzt. Der zentrale Anwendungszustand wird überwiegend lokal bzw. feature-nah gehalten; gemeinsamer Zustand für tief verschachtelte Komponenten wird über drei dedizierte Contexts bereitgestellt (`TickerModeContext`, `TickerDataContext`, `TickerActionsContext`), um Prop-Drilling zu vermeiden und Re-Renders besser zu kontrollieren.

Die White-Label-Fähigkeit ist über eine Konfigurationsschicht (`config/whitelabel.ts`) realisiert (u. a. Farben, Team-Keyword, API-Base-URLs). Die Oberfläche unterstützt alle drei Betriebsmodi (`auto`, `coop`, `manual`) inklusive Laufzeitumschaltung über den `ModeSelector` und Synchronisierung mit dem Backend.

4.6.2 Hook- und State-Architektur

Die Zustands- und Interaktionslogik ist in wiederverwendbare Custom Hooks aufgeteilt — ein Entwurfsmuster, das React seit Version 16.8 als Alternative zu Klassenkomponenten eingeführt hat. Das Frontend umfasst 25 spezialisierte Hooks, die sich in fünf funktionale Kategorien gliedern lassen:

1. **Navigation und Datenauswahl** — Steuerung der hierarchischen Auswahlkette (Land Team Wettbewerb Spieltag Spiel) mit bedarfsgesteuerter Auslösung von n8n-Importtriggern bei leerem Datenbestand.

2. **Datenabfrage und Polling** — Periodisches Laden von Match-, Event- und Tickerdaten über drei spezialisierte Hooks, die jeweils einen eigenen Abfragezyklus betreiben und über einen aggregierenden Hook koordiniert werden.

3. **Echtzeitkommunikation** — WebSocket-Verbindung für den Medien-Feed mit automatischer Reconnect-Strategie.

4. **Redaktionsinteraktion** — Modusverwaltung, Draft-Bestätigung/-Ablehnung, Tastatursteuerung sowie automatische Publikation im Vollautomatik-Modus.

5. **UI-Steuerung** — Panel-GröSSenanpassung, Spielminuten-Berechnung, Spielernamen-Autovervollständigung und Systemstatus-Überwachung.

4.6.3 Dreispalten-Layout und Redaktionsfluss

Die Hauptansicht folgt einem responsiven Dreispalten-Ansatz mit klarer Aufgabenverteilung:

1. **Linkes Panel** — Veröffentlichungsperspektive: listet publizierte Ticker-Einträge als chronologischen Feed, dedupliziert nach `event_id` und sortiert nach Spielminute.

2. **Mittleres Panel** — Redaktionelle Kernarbeit: Event-Verarbeitung in Abhängigkeit vom Modus (auto, coop, manual), Draft-Prüfung und Freigabe/Ablehnung, manuelle Eingabe mit Command-Parser (z. B. `/g`, `/gelb`, `/rot`, `/s`), sowie am unteren Rand integrierte Media- und Social-Panels (ScorePlay-Bilder, YouTube, Twitter/X, Instagram). Die Social-Media-Panels werden ausschlieSSlich bei erkannten EF-Spielen (`isOurTeam`) eingeblendet.

3. **Rechtes Panel** — Kontext- und Analyseinformationen: Matchstatistiken, Aufstellungen inkl. Wechsel-/Karten-/Injury-Kontext sowie spielerbezogene Leistungsdaten.

4.6.4 Moduslogik und Interaktionsdesign

Die Modusumschaltung (vgl. Kap. 4.3.3 für Modi-Definition) ist zentraler Bestandteil der Frontend-Konzeption. Die Umschaltung erfolgt über einen dedizierten `ModeSelector` mit Portal-basiertem Bestätigungsdialog, visueller Toast-Rückmeldung (2200 ms) und Tastatur-Shortcuts (`Ctrl+1` / `Ctrl+2` / `Ctrl+3`). Im kooperativen Modus sind zusätzliche Tastatur-Interaktionen für den Accept-/Reject-Flow (`TAB` / `ESC`) eingebunden, um den Redaktionsdurchsatz zu erhöhen.

4.6.5 Kommunikationsmuster im Frontend

Das Frontend setzt die in Kapitel 4.1.1 definierte hybride Triggerarchitektur mit drei spezialisierten Mechanismen um: **REST-Polling** (5-Sekunden-Intervall) für Kerndaten wie Ticker-Einträge und Spielevents, **Webhook-Trigger** für bedarfsgesteuerte n8n-Importe (vgl. Kap. 4.4.1) sowie **WebSocket** (`/ws/media`) mit Exponential-Backoff-Reconnect für latenzkritische Medieninhalte. REST-Polling wurde gegenüber SSE gewählt, da die zustandslose HTTP-Architektur die Skalierbarkeit auf Render vereinfacht. Die konkreten Hook-Implementierungen und Reconnect-Parameter sind in Kapitel 5.4.3–5.4.5 dokumentiert.

0.4.7 4.7 Skalierbarkeit, Erweiterbarkeit und Systemgrenzen

4.7.1 Horizontale Skalierung des Backends

Die Anwendungsschicht ist zustandslos konzipiert und kann horizontal skaliert werden, indem zusätzliche Uvicorn-Prozesse hinter einem Load-Balancer gestartet werden. Der gemeinsame PostgreSQL-Connection-Pool (`QueuePool` mit `pool_size=20`, `max_overflow=30`) stellt sicher, dass mehrere Backend-Instanzen dieselbe Datenbankverbindungskapazität teilen. Die WebSocket-Verbindungen für Medien stellen eine Skalierungseinschränkung dar: Der in-Memory-`MediaConnectionManager` funktioniert nicht über Prozessgrenzen hinweg und müsste bei Multi-Prozess-Deployment durch ein verteiltes Pub/Sub-System (z. B. Redis) ergänzt werden.

4.7.2 Erweiterung um neue Ereignistypen

Neue Ereignistypen können ohne Datenbankschema-Änderungen ergänzt werden: Die `events` ↪ -Tabelle verfügt über ein generisches `description`-Feld, und die `synthetic_events`-Tabelle nutzt JSONB für vollständig flexible Payloads. Die Übersetzung von Football-API-

Codes auf interne Typbezeichnungen ist im Backend in einer zentralen `EVENT_TYPE_MAP`-Konfiguration hinterlegt. Im Frontend bildet die Funktion `getEventMeta()` jeden Ereignistyp auf Icon und CSS-Klasse ab — neue Typen erfordern ausschlieSSlich einen zusätzlichen Eintrag in dieser Funktion. Alle übrigen Komponenten konsumieren das normalisierte Icon und sind damit von konkreten Typ-Strings entkoppelt.

4.7.3 Erweiterung um neue LLM-Anbieter

Der LLM-Dienst dispatcht über ein zentrales Dictionary auf den providerabhängigen Handler: Für jeden Anbieter existiert eine interne Methode (`_generate_openai_text`, `_generate_gemini_te`
↪ etc.), die im dispatch-Dictionary unter dem Providernamen registriert ist. Neue Anbieter können durch Implementierung eines gleichnamigen Handlers und einen Eintrag in diesem Dictionary ergänzt werden, ohne bestehenden Code zu berühren. OpenRouter wird bereits als generischer Proxy-Einstiegspunkt genutzt, der unter Verwendung des OpenAI-kompatiblen SDK Zugang zu einer Vielzahl weiterer Modelle ohne separate API-Clients bietet.

4.7.4 Deployment-Architektur

Das System wird auf der Cloud-Plattform **Render** betrieben und gliedert sich in drei Deployment-Einheiten:

1. **Backend** — Docker-Container (`python:3.12-slim`) als Render Web Service. Beim Start werden automatisch Datenbankmigrationen ausgeführt, bevor der Uvicorn-Server Anfragen entgegennimmt.

2. **Frontend** — Statisches Build-Artefakt (Create React App), das als Render Static Site ausgeliefert wird. Die Konfiguration erfolgt zur Buildzeit über Umgebungsvariablen.

3. **PostgreSQL** — Render Managed Database als zentrale Persistenzschicht, auf die sowohl Backend als auch n8n zugreifen.

n8n wird als separater Self-Hosting-Dienst betrieben und kommuniziert ausschlieSSlich über HTTP-Endpunkte mit dem Backend. Die n8n-Instanz wird im Entwicklungs- und Evaluationsbetrieb lokal betrieben und via ngrok-Tunnel für externe Webhooks erreichbar gemacht. Diese Containerisierung gewährleistet Reproduzierbarkeit zwischen Entwicklungs- und Produktionsumgebung und ermöglicht eine unabhängige Skalierung der einzelnen Dienste.

0.4.8 4.8 Fazit der Systemkonzeption

Die Systemkonzeption legt fünf interdependente Designpfeiler fest, die gemeinsam die Produktionsfähigkeit des Systems begründen. Die dreischichtige Architektur (Kap. 4.1) schafft die strukturelle Basis für eine unabhängige Weiterentwicklung der Automatisierungs-, Anwendungs- und Präsentationsschicht. Das relationale Datenbankschema mit seinem definierten Ticker-Lifecycle (Kap. 4.3) sichert referenzielle Integrität und Nachvollziehbarkeit aller redaktionellen Entscheidungen — auch für spätere Qualitätsanalysen. Die n8n-Workflow-Schicht (Kap. 4.4) entkoppelt die externe Datenbeschaffung und Orchestrierungslogik vom Anwendungskern und erlaubt eine schnelle Anpassung von Importprozessen ohne Eingriff in das Backend. Die Multi-Provider-LLM-Architektur mit Few-Shot-Prompting und instanzspezifischen Stilprofilen (Kap. 4.5) maximiert Textqualität und Anbieterunabhängigkeit bei minimaler Infrastrukturkomplexität. Das Context-basierte Frontend-Design in Kombination mit dem `ticker_mode`-Feld (Kap. 4.3.3) ermöglicht redaktionelle Kontrolle ohne Latenzeinbu-SSen und ohne Prop-Drilling über Komponentengrenzen.

Das vorliegende Konzept ist damit als vollständige Spezifikation für die Implementierung formuliert, die Kapitel 5 dokumentiert; die systematische Evaluation folgt in Kapitel 6.

0.5 Kapitel 5 – Implementierung

0.5.1 5.1 Überblick und Implementierungsstrategie

Dieses Kapitel dokumentiert die konkrete Umsetzung des in Kapitel 4 konzipierten Systems. Im Fokus stehen die technischen Entscheidungen auf Code-Ebene: die interne Struktur des FastAPI-Backends mit seinen Schichten aus Modellen, Repositories, Services und Routern; die KI-Generierungspipeline von der Ereigniserkennung bis zum Ticker-Eintrag; die Frontend-Implementierung mit React, Context-basiertem Zustandsmanagement und dem slash-command-getriebenen Eingabesystem; sowie die n8n-Workflow-Implementierung der fünf zentralen Automatisierungsworkflows. Ergänzend werden die TypeScript-Migration und die Testabdeckung als Qualitätsmerkmale des Entwicklungsprozesses dokumentiert.

Die Implementierung folgt dem Grundsatz **“so einfach wie nötig, so modular wie sinnvoll”**: Standardpfade (CRUD, Listabfragen) sind bewusst direkt gehalten (Router Repository), während die LLM-Generierung mit ihren Retry-, Semaphore- und Few-Shot-Mechanismen in einen dedizierten Service extrahiert ist.

Die n8n-Workflows (Abschnitt 5.5) werden abweichend von der konzeptionellen Reihenfolge in Kapitel 4 nach der Frontend-Implementierung beschrieben: Die Workflows bauen auf Backend-Endpunkten auf, die bis dahin vollständig dokumentiert sind, sodass die Beschreibung der Orchestrierungslogik erst an dieser Stelle vollständig nachvollziehbar ist.

0.5.2 5.2 Backend-Implementierung

5.2.1 Projektstruktur und Application Entry Point

Das Backend ist unterhalb von `backend/app/` in sieben funktionale Pakete gegliedert:

```
backend/app/
 api/v1/          # HTTP-Router (FastAPI)
 models/          # SQLAlchemy ORM-Modelle (17 Dateien)
 repositories/    # Datenbankzugriff (je Entität eine Klasse)
 schemas/         # Pydantic-Schemas (Request / Response)
 services/        # Fachliche Services (llm_service,
   ↪ ticker_service)
 utils/           # Hilfsmodule (llm_context_builders,
   ↪ evaluation_metrics)
 core/            # Konfiguration, Datenbankverbindung, Enums,
   ↪ Konstanten
```

Diese Aufteilung entspricht dem klassischen Repository-Service-Router-Muster: Router delegieren an Repositories für einfache CRUD-Operationen und an Services für Logik, die mehrere Repositories oder externe Aufrufe koordiniert.

`app/main.py` ist der einzige Einstiegspunkt. Er registriert alle 12 Router unter dem Pfad-Prefix `/api/v1`, konfiguriert CORS-Middleware und bindet statische Dateien (Thumbnail-Cache) ein. Der WebSocket-Endpunkt `/ws/media` wird ohne API-Prefix eingehängt, da er kein REST-Ressourcenpfad ist. Beim Start führt `Base.metadata.create_all(bind=engine)` ein idempotentes Schema-Bootstrapping durch, das bei erstmaliger Inbetriebnahme alle Tabellen anlegt; Schema-Änderungen im Produktionsbetrieb werden via Alembic-Migrationen gesteuert.

Die Anwendung stellt zwei Metaendpunkte bereit:

Endpunkt	Funktion
`GET /`	Statusantwort mit Version (`0.3.0`)
`GET /health`	Datenbankverbindungscheck (`healthy`/`degraded`)

5.2.2 Datenmodelle: Datenbankschema und API-Validierung

SQLAlchemy-Datenbankmodelle — Das Schema umfasst **17 ORM-Modelle** (18 Datenbanktabellen, davon eine via Migration). Die zentralen Domänenobjekte sind:

Modell	Tabelle	Kernfelder
`Country`	`countries`	`id`, `name`, `uid`
`Team`	`teams`	`id`, `uid`, `name`, `external_id`, `is_partner_team`
`Competition`	`competitions`	`id`, `title`, `season_id`
`Match`	`matches`	`id`, `uid`, `match_state`, `match_phase`, `ticker_mode`
`Event`	`events`	`id`, `match_id`, `event_type`, `time`, `description`
`SyntheticEvent`	`synthetic_events`	`id`, `match_id`, `type`, `data` (JSONB)
`TickerEntry`	`ticker_entries`	siehe unten
`Lineup`	`lineups`	`player_id`, `status`, `formation_place`, `position`
`MatchStatistic`	`match_statistics`	19 typisierte Spalten (Pässe, Zweikämpfe, Schüsse...)
`PlayerStatistic`	`player_statistics`	`player_id`, `rating`, `goals`, `assists`, ...
`StyleReference`	`style_references`	`event_type`, `instance`, `league`, `text` (Few-Shot)
`MediaQueue`	`media_queue`	`media_id`, `thumbnail_url`, `event_id`, `status`
`MediaClip`	`media_clips`	`source`, `vid`, `thumbnail_url`, `match_id`

Das `TickerEntry`-Modell hat besondere Bedeutung im Lifecycle des Systems: Es verknüpft über `event_id` (FK auf `events`) und `synthetic_event_id` (FK auf `synthetic_events`) den generierten Text mit seinem Auslöser. Die Felder `status` (`draft`/`published`/`rejected`) und `source` (`ai`/`manual`) bilden den Ticker-Lifecycle ab (vgl. Kap. 4.3.2). Drei zusammengesetzte Indizes (`ix_ticker_match_status`, `ix_ticker_match_phase`, `ix_ticker_event_id`) optimieren die häufigsten Abfragen: publizierte Einträge je Spiel, Phase-Filter und den Deduplizierungs-Check per Event. Das Status-Enum wird als nicht-nativ gespeichert (`native_enum=False`), damit Migrationen ohne Enum-Typ-Änderungen in PostgreSQL auskommen.

Pydantic-Schemas und API-Validierung — Für jeden Router gibt es eigenständige Pydantic-Schemas für Create-, Update- und Response-Strukturen. Alle Schemas nutzen `alias_generator=to_camel` aus `pydantic.alias_generators`, damit JSON-Payloads in camelCase kommuniziert werden, während Python-intern snake_case gilt.

Ein spezifisches Beispiel illustriert die Alias-Mechanik für Match-Scores:

```
class MatchResponse(BaseModel):
    home_score: Optional[int] = Field(None, serialization_alias
      ↪ ='teamHomeScore')
    away_score: Optional[int] = Field(None, serialization_alias
      ↪ ='teamAwayScore')
```

Hier wird `serialization_alias` (nicht `alias`) verwendet, da das Frontend die Partner-API-Feldnamen erwartet (`teamHomeScore`/`teamAwayScore`), während Schreiboperationen über `populate_by_name=True` auch den Python-Namen akzeptieren.

Diese duale Alias-Konvention zieht sich durch alle Schemas: Felder, die Spielstandwerte aus der Partner-API widerspiegeln, erhalten explizite `serialization_alias`-Werte mit Partner-Konvention (`teamHomeScore`). Standard-Beziehungsfelder hingegen erhalten durch den globalen `alias_generator=to_camel` automatisch kanonisches camelCase (`homeTeamId` ↪ , `awayTeamId`). Die Koexistenz beider Konventionen in derselben Schema-Klasse ist eine bewusste Designentscheidung: Das Frontend konsumiert sowohl Partner-API-Daten als auch eigenständig modellierte Beziehungen, und beide müssen ohne Feldumbenennungen im Frontend konsistent adressierbar sein.

Für den Match-Status-Lifecycle definiert `MatchUpdate` ein explizites Alias:

```
match_state: Optional[MatchState] = Field(None, alias='state')
```

Für Ticker-Einträge wird der Status über ein separates PATCH-Schema gesteuert, um versehentliche Massenänderungen durch offene Update-Schemas zu vermeiden.

5.2.3 Repository-Schicht

Jede Entität hat ein dediziertes Repository, das alle SQL-Operationen kapselt. Alle Repositories erben von einer gemeinsamen `BaseRepository[T]`-Basisklasse, die die generische Operation `exists()` bereitstellt; jede Instanz erhält eine `Session` im Konstruktor:

```
class TickerEntryRepository(BaseRepository[TickerEntry]):
    model = TickerEntry

    def get_by_event(self, event_id: int) -> Optional[
        ↪ TickerEntry]:
        return self.db.query(TickerEntry).filter(
            TickerEntry.event_id == event_id
        ).first()

    def get_by_match(self, match_id: int, status=None) -> list[
        ↪ TickerEntry]:
        ...
```

Diese Kapselung hat zwei Vorteile: Router bleiben frei von ORM-Details, und Repository-Methoden können direkt mit echten Datenbanken in Integrationstests geprüft werden — ohne Mock-Schichten, die Implementierungsabweichungen verbergen.

Das `MatchRepository` definiert eine interne `_base_query()`-Methode, die Heimteam, Auswärtsteam, Wettbewerb und Saison per `joinedload` in einer einzigen Datenbankabfrage vorlädt. `get_by_id()` nutzt diese Methode konsequent:

```
class MatchRepository(BaseRepository[Match]):
    model = Match

    def _base_query(self) -> Query:
        return self.db.query(Match).options(
            joinedload(Match.home_team),
            joinedload(Match.away_team),
            joinedload(Match.competition),
            joinedload(Match.season),
        )

    def get_by_id(self, match_id: int) -> Optional[Match]:
        return self._base_query().filter(Match.id == match_id).
          ↪ first()
```

Diese Methode wird von der KI-Generierungsroute intensiv genutzt, da für jeden Aufruf vollständige Kontext-Daten (Teamnamen, Wettbewerb) benötigt werden.

5.2.4 API-Router: Endpunktstruktur

Der Ticker-Bereich ist in drei Router aufgeteilt, die gemeinsam unter `/api/v1/ticker` eingehängt werden:

Router	Datei	Kernoperationen
`ticker_crud`	`ticker_crud.py`	GET list, GET single, PATCH status, DELETE, POST manual
`ticker_generate` ↪	`ticker_generate` ↪ `.py`	POST generate/{event_id}, generate-synthetic, generate-batch
`ticker_batch`	`ticker_batch.` ↪ `py`	POST translate-batch, generate-match-phases

Diese Aufteilung verhindert, dass eine einzelne Routerdatei durch die Kombination aus CRUD-, Generierungs- und Batch-Endpunkten unübersichtlich wird.

Ein kapselndes Hilfsmuster zieht sich durch alle Router: `require_or_404(value, message` ↪ `)` — eine schmale Utility-Funktion in `utils/http_errors.py`, die `None`-Rückgaben aus

Repository-Aufrufen direkt in eine `HTTPException(404)` überführt. Das eliminiert das repetitive `if obj is None: raise HTTPException(404, ...)` in Routern und stellt sicher, dass alle 404-Antworten einheitlich formuliert sind. Der Batch-Endpunkt `POST /api/v1/ticker/translate-batch/{match_id}` ruft für alle publizierten KI-Einträge eines Spiels den LLM-Service mit reduzierter Temperatur (`0.1`) auf, um sprachlich deterministische Übersetzungen zu erzeugen — eine gegenüber der Originalgenerierung bewusst andere Parametrisierung.

Die wichtigsten Endpunkte gliedern sich in vier funktionale Bereiche: **Stammdaten** (Countries, Teams, Matches, Seasons, Competitions), **Ticker-Lifecycle** (CRUD, Generierung per Event, Batch-Generierung für Synthetic Events, Übersetzung), **Medien & Clips** (ScorePlay-Import, Media-Queue, WebSocket) und **Spielkontext** (Lineups, Statistiken, Events). Insgesamt umfasst die API über 70 Routen; die vollständige Endpunktliste ist über die automatisch generierte OpenAPI-Dokumentation unter `/api/docs` einsehbar.

5.2.5 Ticker-Service: Domänenlogik

Der `ticker_service` kapselt die Domänenlogik für KI-Einträge. Er koordiniert Datenbankabfragen, Kontextaufbau und den LLM-Aufruf, hält dabei aber die Schichten klar getrennt: `llm_service.py` hat keine Datenbankabhängigkeit.

Die vier zentralen Funktionen:

`score_at_event()` berechnet den Spielstand zum Zeitpunkt eines Torereignisses. Da die Football-API den kumulativen Stand liefert, muss der Stand *vor* dem aktuellen Event rekonstruiert werden. Die Funktion lädt alle Tore bis zur `position` des Events, unterscheidet zwischen regulären Toren und Eigentoren (letztere zählen für den Gegner) und akkumuliert die Scores für Heim- und Auswärtsteam.

`build_match_context()` erstellt das Kontext-Dictionary für den LLM-Prompt: Teamnamen, aktueller Stand, Matchzustand, Spielminute und Liga.

`call_llm()` ist der zentrale, Semaphore-gesicherte LLM-Aufruf. Er lädt zunächst aus dem `StyleReferenceRepository` bis zu drei Stilbeispiele für `event_type + instance + league` als Few-Shot-Kontext und delegiert dann an `generate_ticker_text()`:

```
async with _llm_semaphore:    # asyncio.Semaphore(settings.
    LLM_CONCURRENCY)
    return await generate_ticker_text(
        event_type=event_type,
        style_references=style_references,  # aus DB geladen
        ...
    )
```

Die Semaphore ist auf Modulebene als Singleton definiert (`_llm_semaphore = asyncio.Semaphore(settings.LLM_CONCURRENCY)`, Standard: 8) und begrenzt gleichzeitige LLM-Aufrufe pro Prozessinstanz.

`make_ai_entry()` ist ein schlanker Builder, der aus den LLM-Ergebnissen ein `TickerEntryCreate`-Schema erzeugt. Er setzt `source='ai'` und legt den initialen Status abhängig vom Aufrufkontext fest (`draft` im Co-op-Modus, `published` im Auto-Modus).

5.2.6 LLM-Service: Prompt-Aufbau und Provider-Dispatch

Der `LLMService` ist eine single-class-Implementierung, die beim Initialisieren den passenden Client instanziiert und über ein Dispatch-Dictionary aufruft:

```
dispatch = {
    'mock':        self._generate_mock_text,
    'gemini':      self._generate_gemini_text,
    'openrouter': self._generate_openrouter_text,
    'openai':      self._generate_openai_text,
    'anthropic':   self._generate_anthropic_text,
}
return dispatch[self.provider](**kwargs)
```

Das Dispatch-Pattern macht es trivial, neue Provider hinzuzufügen: Implementierung einer Methode `_generate_X_text()` und ein Eintrag im Dictionary.

Der Prompt wird in vier modularen Methoden aufgebaut:

`_build_event_lines()` erzeugt die Fakten-Sektion. Jeder Parameter (Ereignistyp, Detail, Minute, Spieler, Team) wird nur dann eingebunden, wenn er vorhanden ist. Die Ereignisbezeichnung wird aus `EVENT_TYPE_LABEL` in lesbares Deutsch übersetzt (z. B. `'goal'` `'Tor'`).

`_build_few_shot_block()` formatiert die Stilreferenzen als nummerierte Beispiele:

```
### STILREFERENZEN
Schreibe in exakt diesem Stil (Rhythmus, Wortwahl,
  ↪ Emotionalität):
- 'Mustermann trifft mit einem satten Rechtsschuss  1:0!'
- 'Eintracht schlägt zu  der erste Treffer des Abends!'
```

`_build_prematch_parts()` ergänzt für Pre-Match-Typen eine zusätzliche harte Instruktion: Das Modell darf keine Live-Szenen erfinden, sondern ausschlieSSlich Vorschau- und Analyseinhalte produzieren.

Der vollständig zusammengesetzte System-Prompt folgt der in Abschnitt 4.5.2 beschriebenen modularen Struktur aus sechs Bausteinen (Rolleninstruktion, Stilinstruktion, Fakten-

block, Match-Kontext, Few-Shot-Block, Regelblock). Für Pre-Match-Typen fügt `_build_prematch_part` ↪ `()` eine zusätzliche harte Schutzregel ein, die das Modell auf Vorschau- und Analyseinhalte beschränkt. Die vier `_build_*`-Methoden erzeugen die Bausteine unabhängig voneinander, sodass fehlende Informationen (z. B. kein Spieler bei Phasenereignissen) den Prompt nicht invalidieren, sondern den entsprechenden Block auslassen.

LLM-Parameter: `temperature=0.3` für konsistente Ausgaben, `max_tokens` aus `LLM_MAX_TOKENS` ↪ (konfigurierbar). Bei Rate-Limit-Fehlern werden bis zu `LLM_RETRY_ATTEMPTS` (3) Versuche mit linearem Backoff (`30 s / 60 s / 90 s`) unternommen; bei sonstigen transienten Fehlern greift exponentielles Backoff (`1 s / 2 s / 4 s`).

Normalisierung des Event-Typs erfolgt vorab in `_normalize_event_type()`. Die `EVENT_TYPE_MAP` übersetzt externe Strings (Partner-API: `PartnerGoal`, Football-API: `Goal` ↪) auf interne Bezeichner (`goal`). Nicht erkannte Typen fallen auf `'comment'` zurück.

5.2.7 WebSocket-Endpunkt für Media-Queue

Der WebSocket-Endpunkt `/ws/media` wird durch einen dedizierten `MediaConnectionManager` ↪ verwaltet:

```
class MediaConnectionManager:
    def __init__(self) -> None:
        self.active_connections: list[WebSocket] = []

    async def connect(self, websocket: WebSocket) -> None:
        await websocket.accept()
        self.active_connections.append(websocket)

    async def broadcast(self, data: dict) -> None:
        dead: list[WebSocket] = []
        for ws in self.active_connections:
            try:
                await ws.send_json(data)
            except (WebSocketDisconnect, RuntimeError):
                dead.append(ws)
        for ws in dead:
            self.disconnect(ws)
```

Wenn das Backend neue Media-Queue-Einträge über `POST /api/v1/media/incoming` erhält, ruft der Router anschlieSSend `manager.broadcast({'type': 'new_media', 'items` ↪ `': [...]})` auf. Alle verbundenen Clients erhalten das Update in Echtzeit. Der Manager

hält Verbindungen in einer einfachen In-Memory-Liste, was für die aktuelle Nutzung (wenige gleichzeitige Redakteur-Clients) ausreichend ist; bei Multi-Prozess-Deployments wäre ein externes Pub/Sub-System (z. B. Redis) erforderlich.

Die persistente Seite der Media-Queue wird durch das `MediaQueueStatus`-Enum gesteuert, das zwei Zustände unterscheidet: `pending` (Medien sind eingegangen und warten auf redaktionelle Zuordnung) und `published` (Medien wurden einem Ticker-Eintrag zugewiesen). Das Frontend zeigt im `MediaPickerPanel` ausschlieSSlich `pending`-Einträge an; sobald ein Redakteur ein Bild einem Ticker-Eintrag anhängt, wird der Status per `PATCH /api` ↪ `/v1/media/queue/{id}` auf `published` gesetzt. Dieses einfache Zwei-Zustands-Modell verhindert, dass einmal zugeordnete Medien in der Auswahl erneut auftauchen. Der korrespondierende Frontend-Hook `useMediaWebSocket`, der die Verbindungslebensdauer, das exponentielle Reconnect-Backoff und das Nachrichten-Handling auf Client-Seite übernimmt, ist in Abschnitt 5.4.5 beschrieben.

0.5.3 5.3 KI-Generierungspipeline

5.3.1 Ablaufdiagramm

Die folgende Darstellung zeigt den vollständigen Ablauf von einem neuen Live-Ereignis bis zum publizierten Ticker-Eintrag:

```
sequenceDiagram
    participant n8n as n8n Workflow
    participant API as FastAPI Backend
    participant DB as PostgreSQL
    participant LLM as LLM Provider
    participant FE as React Frontend

    n8n->>API: POST /api/v1/events (Ereignis-Import)
    API->>DB: INSERT INTO events
    n8n->>API: POST /api/v1/ticker/generate/{event_id}
    API->>DB: SELECT ticker_entry WHERE event_id (Dedup)
    alt Eintrag existiert bereits
        API-->>n8n: 200 OK (existierender Eintrag)
    else Neues Ereignis
        API->>DB: SELECT match + teams (joinedload)
        API->>DB: SELECT style_references (Few-Shot)
        API->>LLM: Prompt (Fakten + Kontext + Stilreferenzen)
        LLM-->>API: Generierter Text + Modellname
```

```
        API->>DB: INSERT ticker_entry (status=draft|published)
        API-->>n8n: 201 Created
    end
    FE->>API: GET /api/v1/ticker/{match_id} (Polling 5s)
    API-->>FE: TickerEntry[] (inkl. neuer Entwurf)
    alt Modus=coop
        FE-->>FE: Entwurf in Draft-Queue anzeigen
        note over FE: Redakteur: TAB (accept) / ESC (reject)
        FE->>API: PATCH /api/v1/ticker/{id} {status:'published
          ↪ '}
    else Modus=auto
        note over DB: status=published gesetzt bei Generierung
    end
```

5.3.2 Deduplizierung

Ein zentrales Qualitätsmerkmal der Pipeline ist die Deduplizierung: Der Generierungsendpunkt prüft vor jedem LLM-Aufruf, ob bereits ein Ticker-Eintrag für die `event_id` existiert:

```
existing = ticker_repo.get_by_event(event_id)
if existing:
    return existing  # Idempotenter Rückgabepfad ohne LLM-
      ↪ Aufruf
```

Dies macht den Endpunkt idempotent gegenüber Mehrfachaufrufen — ein wichtiges Merkmal, da n8n-Workflows bei Fehlern automatisch wiederholen. Ohne diese Prüfung würde jeder Retry einen zusätzlichen Ticker-Eintrag mit unterschiedlichem (nicht-deterministischem) Text erzeugen.

5.3.3 Statusentscheidung und Modus-Steuerung

Der initiale Status des erzeugten Ticker-Eintrags wird über den Aufrufparameter `auto_publish` ↪ gesteuert, den n8n aus dem `ticker_mode` des Spiels ableitet (vgl. Kap. 4.3.3 für die konzeptionelle Beschreibung der drei Betriebsmodi):

```
status=TickerStatus.published if data.auto_publish else
  ↪ TickerStatus.draft
```

Damit wird die Modus-Logik zu einem reinen Aufrufparameter des Backends — die Entscheidungsinstanz ist n8n auf Basis des in der Datenbank gespeicherten `ticker_mode`. Das Backend selbst ist modus-agnostisch: Es erhält einen Booleschen Wert und setzt den Status entsprechend.

Die drei Modi übersetzen sich auf Implementierungsebene in drei verschiedene Pfade:

Modus	`auto_publish`	Ergebnis	Redakteurs-Eingriff
`auto`	`True`	`published`	—
`coop`	`False`	`draft`	TAB (accept) / ESC (reject)
`manual`	—	—	Slash-Command `POST /manual`

Im `manual`-Modus wird der Generierungs-Endpunkt gar nicht aufgerufen: n8n importiert zwar weiterhin Ereignisse, triggert aber keinen LLM-Aufruf. Ticker-Einträge entstehen ausschlieSSlich über den `POST /api/v1/ticker/manual`-Endpunkt, den das Frontend aus dem Slash-Command-Parser heraus bedient. Der Eintrag wird dort direkt mit `status=published` ↪ angelegt, da die Redakteurin den Text selbst verfasst hat und keine weitere Freigabe erforderlich ist.

Im `coop`-Modus liegt die gesamte UX-Last auf der Draft-Queue: Das Frontend zeigt neue `draft`-Einträge mit `TAB`/`ESC`-Shortcuts an (vgl. Abschnitt 5.4.6), und die Entscheidungs-Latenz beträgt je nach Komplexität des Entwurfs 15–30 Sekunden. Dieses Design stellt sicher, dass keine KI-generierten Texte ohne menschliche Prüfung publiziert werden — eine Entscheidung, die direkt aus dem journalistischen Qualitätsanspruch in Kapitel 2.1 folgt.

5.3.4 Zustandsautomat der Ticker-Einträge

Das konzeptionelle Statusmodell (`draft` `published` / `rejected`, Undo-Übergang `published` ↪ `draft`) ist vollständig in Abschnitt 4.3.2 beschrieben. Implementierungsseitig sind zwei Details hervorzuheben:

Der Übergang `published` `draft` (Undo) ist im Frontend als **Toast-Aktion** realisiert: Nach jeder Publikation erscheint für 5 Sekunden ein Widerruf-Button. Über `PATCH /api/v1` ↪ `/ticker/{id}` mit `{ 'status': 'draft' }` wird der Eintrag zurückgesetzt und steht erneut zur Bearbeitung bereit.

`rejected`-Einträge werden nicht aus der Datenbank gelöscht, sondern behalten ihren Status. Diese Entscheidung sichert die Reproduzierbarkeit der Evaluation: Alle je generierten Einträge — auch verworfene — sind für spätere Qualitätsanalysen auswertbar.

0.5.4 5.4 Frontend-Implementierung

5.4.1 Komponentenhierarchie

Das Frontend ist als React-Anwendung mit TypeScript organisiert. Die Hauptansicht (`LiveTicker` ↪) gliedert sich in drei Panel-Komponenten, die von gemeinsam genutzten Hooks und Contexts gespeist werden:

```
graph TD
    App --> LiveTicker
    LiveTicker --> |TickerModeContext| LeftPanel
    LiveTicker --> |TickerModeContext| CenterPanel
    LiveTicker --> |TickerModeContext| RightPanel
    LiveTicker --> |TickerDataContext| LeftPanel
    LiveTicker --> |TickerDataContext| CenterPanel
    LiveTicker --> |TickerDataContext| RightPanel
    LiveTicker --> |TickerActionsContext| CenterPanel

    CenterPanel --> EventCard
    CenterPanel --> EntryEditor
    CenterPanel --> AIDraft
    CenterPanel --> SummarySection
    CenterPanel --> MediaPickerPanel
    CenterPanel --> YouTubePanel
    CenterPanel --> TwitterPanel
    CenterPanel --> InstagramPanel

    LeftPanel --> PublishedEntry
    RightPanel --> Collapsible['Collapsible (Statistiken,
      ↪ Aufstellung, Spieler)']
    RightPanel --> FormationColumn
    RightPanel --> StatRow

    EventCard --> |onGenerate| TickerActionsContext
    AIDraft --> |onPublished/onDraftActive|
      ↪ TickerActionsContext
    EntryEditor --> |onManualPublish| TickerActionsContext
```

Die drei Panels sind physisch getrennte React-Komponenten, kommunizieren aber ausschlieSSlich über Context-Provider — kein Prop-Drilling über Komponentengrenzen hinweg.

5.4.2 React Context-Architektur

Das Frontend verwendet drei spezialisierte Contexts, die in `LiveTicker` zusammengeführt werden. Jeder Context hat eine klar definierte Verantwortung:

`TickerModeContext` — Modus-State und Tastatur-Aktionen:

```
interface TickerModeContextValue {
  mode: TickerMode; // 'auto' | 'coop' | 'manual'
  setMode: (mode: TickerMode) => void;
  acceptDraft: () => Promise<void>; // TAB-Shortcut
  rejectDraft: () => Promise<void>; // ESC-Shortcut
}
```

`TickerDataContext` — Match-Daten und Reload-Funktionen:

```
interface TickerDataContextValue {
  match: Match | null;
  events: MatchEvent[];
  tickerTexts: TickerEntry[];
  prematch: TickerEntry[];
  lineups: LineupEntry[];
  matchStats: MatchStat[];
  players: Player[];
  playerStats: PlayerStat[];
  injuries: InjuryGroup[];
  reload: ReloadFunctions;
  generatingId: number | null;
}
```

`TickerActionsContext` — Callbacks für redaktionelle Aktionen:

```
interface TickerActionsContextValue {
  onGenerate: (eventId: number, style: TickerStyle) => Promise<
    ↪ void>;
  onManualPublish: (text, icon?, minute?, phase?, rawInput?) =>
    ↪  Promise<void>;
  onDraftActive: (id: number, text: string) => void;
  onPublished: (id: number, text: string, isManual?: boolean)
    ↪ => void;
  onEditEntry: (id: number, text: string) => Promise<void>;
  onDeleteEntry: (id: number) => Promise<void>;
  retractedText: string | null;
  clearRetractedText: () => void;
```

```
}
```

Die Trennung von Daten- und Aktions-Context hat einen konkreten Performance-Vorteil: Komponenten, die nur Actions konsumieren (z. B. `EntryEditor`), re-rendern nicht bei Änderungen an `tickerTexts`. Dies ist bei einem Live-Ticker mit 5-Sekunden-Polling relevant.

Alle Contexts exportieren zusätzlich einen Hook (`useTickerModeContext`, `useTickerDataContext` ↪ , `useTickerActionsContext`), der bei Verwendung auSSerhalb des Providers einen klaren Fehler wirft — ein Entwicklungssicherheitsnetz.

5.4.3 Hook-Architektur

Die Zustandslogik ist in zwei Ebenen von Hooks organisiert: gemeinsame Hooks unter `src/` ↪ `hooks/` und feature-spezifische Hooks unter `components/LiveTicker/hooks/`.

Die wichtigsten Hooks und ihre Verantwortlichkeiten:

Hook	Paket	Verantwortlichkeit
`useNavigation`	hooks/	LandTeamWettbewerbSpieltagSpiel Navigationskette
`useMatchData`	hooks/	Aggregiert die drei Polling-Hooks und koordiniert Reload-Zyklen
`useMatchCore`	hooks/	Polling der Match-Metadaten (Teams, Status, Spielstand, Minute)
`useMatchEvents`	hooks/	Polling der Spielereignisse (Tore, Karten, Auswechslungen)
`useMatchTicker`	hooks/	Polling der Ticker-Einträge (Entwürfe, publizierte Texte)
`useMatchTriggers` ↪	hooks/	n8n-Webhooks nach Matchauswahl und Statuswechseln
`useTickerMode`	hooks/	`mode`-State, PATCH zum Backend, Keyboard-Shortcuts
`useMediaWebSocket` ↪	hooks/	WebSocket-Verbindung mit exponentiellem Reconnect-Backoff
`usePanelResize`	hooks/	Drag-Resize zwischen Center- und Right-Panel
`useApiStatus`	hooks/	Health-Check gegen Backend (alle 30 s)
`useTicker`	LT/hooks/	Ticker-Actions, Draft-Queue, Publish-Toast (Undo)
`useEventDraft`	LT/hooks/	Entwurfsansicht, Accept/Reject-Logik
`useAutoPublisher` ↪	LT/hooks/	Automatisches Publizieren im Auto-Modus
`useRightPanelData` ↪	LT/hooks/	Aufbereitung von Lineups, Stats, Spielerinfo für das rechte Panel

`useMatchCore`, `useMatchEvents` und `useMatchTicker` betreiben jeweils einen eigenen `setInterval`-Zyklus, der bei Mount startet und bei Unmount gestoppt wird. Die HTTP-

Kommunikation erfolgt über **Axios** als HTTP-Client. `useMatchData` aggregiert die drei Hooks und stellt die Daten über `TickerDataContext` bereit.

`useMatchData` steuert das Polling-Intervall über die Hilfsfunktion `resolvePollingInterval`
↪ `()`:

```
// src/utils/resolvePollingInterval.ts
export function resolvePollingInterval(matchState: string |
  ↪ null): number {
  if (matchState === 'Live' || matchState === 'FullTime')
    ↪ return POLL_EVENTS_MS; // 5000
  if (matchState == null) return POLL_EVENTS_MS; // 5000
  return POLL_PREMATCH_MS; // 5000
}
```

Aktuell sind beide Konstanten (`POLL_EVENTS_MS`, `POLL_PREMATCH_MS`) auf 5000 ms gesetzt, sodass alle Zustände mit einem einheitlichen 5-Sekunden-Intervall gepollt werden. Die Unterscheidung ist als Erweiterungspunkt vorgesehen, um bei Bedarf ruhigere Polling-Intervalle für inaktive Spiele einzuführen.

5.4.4 Slash-Command-Parser (`parseCommand.ts`)

Der Slash-Command-Parser ermöglicht schnelle manuelle Texteingabe ohne Maus-Interaktion. Er übersetzt kurze Kommandos in formatierte Ticker-Texte und liefert Metadaten (Icon, Phase, Minute) für die Publikation.

Die Funktion `parseCommand(input, currentMinute)` gibt ein typisiertes `ParseResult` zurück:

```
export interface ParseResult {
  type: string;
  formatted: string;
  warnings: string[];
  isValid: boolean;
  meta: {
    icon: string;
    phase: string | null;
    minute: number | null;
  };
}
```

Phasen-Commands (11 Einträge in `PHASE_CMDS`) erzeugen vordefinierte Texte mit optionaler Minutenangabe. Die folgende Tabelle zeigt eine Auswahl der häufigsten Einträge (7 von 11):

Command	Ausgabe	Phase Icon
`/prematch`	"Vorbericht"	`Before`
`/anpfiff`	"Anpfiff!"	`FirstHalf`
`/hz`	"Halbzeit!"	`FirstHalfBreak`
`/2hz`	"AnstoSS zur 2. Halbzeit"	`SecondHalf`
`/vz1`	"Beginn der Verlängerung"	`ExtraFirstHalf`
`/elfmeter`	"ElfmeterschieSSen beginnt"	`PenaltyShootout`
`/abpfiff`	"Abpfiff!"	`After`

Event-Commands (12 Einträge in `CMD_MAP`, 8 Typen nach Expansion) erzeugen strukturierte Texte mit Validierungshinweisen:

```
/g Mustermann EF   'TOR  Mustermann (EF)'  + icon
/gelb Müller EF    'Gelb  Müller (EF)'     + icon
/rot Huber Bayern  'Rote Karte  Huber (Bayern)' + icon
/s EinRaus AusRein EF  'Wechsel  EinRaus  AusRein (EF)' + icon
/n Freisto knapp über das Tor  'Freisto knapp über das Tor'
```

Für unvollständige Eingaben gibt `warnings[]` gezielte Hinweise zurück (`'Fehlend: Spieler'`), die im `EntryEditor` live angezeigt werden. Das Frontend kann damit ohne Backend-Aufruf Validierungsfeedback geben.

Nicht erkannte Commands werden mit `isValid: false` und einer Fehlermeldung zurückgegeben (`'Unbekannter Command: /xyz'`), statt einen Laufzeitfehler zu werfen.

5.4.5 WebSocket-Hook (`useMediaWebSocket`)

Der `useMediaWebSocket`-Hook verwaltet die Lebensdauer der WebSocket-Verbindung zum Backend. Die Kernentscheidung ist das **exponentielle Reconnect-Backoff**:

```
const delay = Math.min(
  BASE_DELAY_MS * 2 ** retryCountRef.current, // 1s  2s  4s  8s
    ↪
  MAX_DELAY_MS, // max 30s
);
```

Der `retryCountRef` zählt aufeinanderfolgende Verbindungsabbrüche. Bei erfolgreicher Reconnection wird er auf 0 zurückgesetzt. `BASE_DELAY_MS = 1000`, `MAX_DELAY_MS = 30_000`.

Ein `mountedRef` verhindert Zustandsänderungen nach dem Unmounten der Komponente — ein häufiges React-Problem bei asynchronen Operationen:

```
ws.onclose = () => {
  if (!mountedRef.current) return; // Kein setState nach Unmount
  //  retry-Logik
};
```

Der Cleanup im `useEffect` setzt `ws.onclose = null`, bevor die WebSocket geschlossen wird. Ohne diesen Schritt würde das `close`-Event des kontrollierten SchlieSSens einen unbeabsichtigten Reconnect-Versuch auslösen.

5.4.6 Modusimplementierung im Frontend

Der aktive Modus (`auto` / `coop` / `manual`) steuert das Verhalten mehrerer Frontend-Komponenten (vgl. Kap. 4.6.4 für die konzeptionelle Einordnung der Moduslogik und Interaktionsdesign). Die Implementierung verteilt sich auf drei Ebenen:

Persistenz und Synchronisation — `useTickerMode` initialisiert den Modus lokal mit `coop` als Standardwert und überschreibt diesen beim Laden eines Spiels mit dem in der Datenbank gespeicherten `ticker_mode`. Änderungen werden über `PATCH /matches/{id}/ticker-mode` ans Backend zurückgeschrieben. Lokale Optimistic-Updates vermeiden wahrnehmbare Latenz beim Umschalten.

Keyboard-Shortcuts — Im Co-op-Modus registriert `useTickerMode` die Shortcuts `TAB` (acceptDraft) und `ESC` (rejectDraft) direkt als Keyboard-Event-Listener. Die Shortcut-Registrierung ist an den Modus-Zustand gebunden und wird bei Moduswechsel automatisch aktualisiert.

`useAutoPublisher` — Im Auto-Modus überwacht dieser Hook die `tickerTexts` auf neue Draft-Einträge (`status='draft'`) und publiziert sie automatisch. Dies ist als Frontend-Fallback gedacht: Primär setzt n8n mit `auto_publish=True` den Status direkt beim Generieren; `useAutoPublisher` fängt Fälle auf, in denen das nicht geschehen ist.

5.4.7 TypeScript-Migration

Das Frontend wurde im Verlauf des Projekts von reinem JavaScript auf TypeScript migriert. Ziel war nicht die vollständige Auflösung aller `any`-Typen, sondern eine pragmatische

Typisierung der systemkritischen Pfade mit messbaren Qualitätskennzahlen.

Migrationsstrategie — Die Migration folgte einem schrittweisen Ansatz: Zuerst wurden alle `.js`- und `.jsx`-Dateien in `.ts` und `.tsx` umbenannt, dann `tsconfig.json` konfiguriert (`target: es2015`, `module: esnext`, `jsx: react-jsx`, `moduleResolution: node`). AnschlieSSend wurden TypeScript-Fehler von innen nach auSSen behoben — beginnend mit dem Typ-System (`src/types/index.ts`) über Contexts bis zu den Komponenten. Der `strict`-Modus ist derzeit deaktiviert (`'strict': false`), um die inkrementelle Migration ohne blockierende Fehlereskalation zu ermöglichen.

Zentrale Typ-Definitionen — `src/types/index.ts` definiert die gemeinsamen Domain-Interfaces:

```
export type TickerMode = 'auto' | 'coop' | 'manual';
export type TickerStyle = 'neutral' | 'euphorisch' | 'kritisch
  ↪ ';
export type MatchPhase = 'Before' | 'FirstHalf' | '
  ↪ FirstHalfBreak' | 'SecondHalf' | ...;

export interface TickerEntry {
  id: number;
  match_id: number;
  event_id?: number | null;
  text: string;
  style?: string | null;
  status: 'draft' | 'published' | 'rejected';
  source?: 'ai' | 'manual';
  minute?: number | null;
  phase?: string | null;
  icon?: string | null;
  image_url?: string | null;
  video_url?: string | null;
  llm_model?: string | null;
}
```

Ergebnis — Nach der Migration erzielt das Projekt **0 TypeScript-Compilerfehler** und eine **type-coverage von 95,84 %** (gemessen mit `type-coverage --strict`). Die detaillierte Darstellung der Migrationsergebnisse und die Analyse der verbleibenden untypisierten Stellen folgt in Abschnitt 6.2.5.

0.5.5 5.5 n8n Workflow-Implementierung

Die n8n-Workflows bilden die Orchestrierungsschicht zwischen externen Datenquellen, dem FastAPI-Backend und dem LLM-Dienst. Alle 15 Workflow-Dateien liegen als versionierte JSON-Exporte im Projektverzeichnis `n8n/` vor und können direkt in eine n8n-Instanz importiert werden. Die Workflows gliedern sich in vier funktionale Gruppen:

A) Stammdaten und Matchstruktur — Die Workflows `01_import_countries`, `02_import_teams_by_country`, `03_import_competitions_and_matches` und `04_import_matches` laden Länder, Teams, Wettbewerbe und Spielpläne aus API-Football und persistieren diese via Upsert-Strategien. Externe IDs dienen als stabile Zuordnungsschlüssel für konfliktfreie Aktualisierungen.

B) Spielbezogene Detaildaten — `04_import_lineups`, `05_import_match_statistics`, `06_import_player_statistics` und `07_import_prematch` importieren Aufstellungen, Match- und Spielerstatistiken matchbezogen. Der Prematch-Workflow erzeugt zusätzlich strukturierte Kontextdaten (Verletzungen, Head-to-Head, Tabellenstand) als synthetische Events, die dem LLM als Promptkontext dienen.

C) KI-Generierung — `09_events_llm_workflow`, `13_Halftime_aftertime` und `14_Game_ANpfiff_ABpfiff` importieren Live-Ereignisse, erzeugen synthetische Phasenereignisse und delegieren die Textgenerierung an das Backend. Der Zusammenfassungs-Workflow (`13`) ruft den LLM-Provider direkt auf, da Halbzeit-/Abpfiff-Zusammenfassungen einen umfangreicheren Prompt mit Statistiken erfordern.

D) Medien und Social Media — `08_scoreplay_media_workflow`, `10_Twitter`, `11_youtube` und `12_insta` importieren Medien- und Social-Media-Inhalte. Der Media-Workflow übergibt Bilder an das Backend, das diese über WebSocket verteilt; die Social-Media-Workflows speichern Clips in der Datenbank. Diese Workflows dienen der Ingestion, nicht dem Publishing.

Dieser Abschnitt dokumentiert die Implementierungsdetails der fünf zentralen Workflows (Gruppen B–D).

5.5.1 Events-LLM-Workflow (`09_events_llm_workflow.json`)

Workflow `09` ist der kritischste im System: Er importiert Live-Ereignisse via Webhook (`POST /Events`), persistiert sie per UPSERT (`ON CONFLICT (source_id)DO NOTHING`) und triggert die KI-Generierung für jeden neuen Event. Eine initiale SQL-Abfrage bestimmt anhand der Teamzugehörigkeit, ob die Instanz `ef_whitelabel` oder `generic` und der Stil `euphorisch` oder `neutral` ist; `auto_publish` wird direkt aus `ticker_mode` des Spiels gesetzt. Nur Events mit erfolgreicher Datenbankzeile (zurückgegebene `id`, kein `DO NOTHING`-Konflikt) passieren den Filter-Knoten und triggern den Backend-Endpunkt. Ein zusätzlicher EF-spezifischer Zweig liest Spielerdaten von `profis.eintracht.de`, baut S3-Video-

URLs aus neunstellig aufgefüllten Spieler-IDs auf und persistiert Torjubel-Videos als Ticker-Einträge.

5.5.2 Prematch-Import-Workflow (`07_import_prematch.json`)

Workflow `07` baut den Vorberichtskontext aus fünf parallelen Football-API-Abfragen auf (Verletzungen, Head-to-Head, Teamstatistiken Heim/Gast, Tabellenstand) und persistiert die Ergebnisse als `synthetic_events` mit idempotenter `ON CONFLICT (match_id, type)DO` ↪ `UPDATE`-Strategie. Ein LLM-Trigger wird nur ausgelöst, wenn noch kein nicht-verworfener Ticker-Eintrag für das jeweilige synthetische Event existiert (`WHERE te.id IS NULL`) — bereits generierte und eventuell publizierte Texte werden damit nicht überschrieben.

5.5.3 Matchphasen-Workflow (`14_Game_ANpfiff_ABpfiff.json`)

Workflow `14` verarbeitet Spielzustands-Übergänge (Anpfiff, Halbzeit, Abpfiff etc.) via Webhook (`POST /match-status`). Ein JavaScript-Knoten validiert Zustandsübergänge gegen eine Matrix erlaubter Vorzustände — ungültige Übergänge (z. B. direkt von `PreMatch` zu `2H`) werden zurückgewiesen. Für Vollzeit-Ereignisse (`FT`, `AET`, `PEN`) generiert der Workflow die gesamte Phasensequenz rückwirkend: Ein `FT`-Signal erzeugt vier synthetische Events (AnstoSS, Halbzeit, 2. Halbzeit, Abpfiff). Das zentrale SQL-Statement kombiniert Match-Update, Event-Insert und eine `demote`-CTE in einer einzigen Transaktion — Letztere stuft bereits publizierte Phasen-Texte bei Re-Triggers auf `draft` zurück, sodass die Redaktion sie erneut prüfen kann.

5.5.4 Halbzeit/Abpfiff-Zusammenfassung (`13_Halftime_aftertime.json`)

Workflow `13` erzeugt narrative Zusammenfassungen für Halbzeit und Abpfiff. Im Gegensatz zu `09` ruft er OpenRouter (`google/gemini-2.0-flash-lite-001`) **direkt** auf, da Zusammenfassungen keinen Few-Shot-Stil aus der Datenbank benötigen, sondern einen umfangreicheren Prompt mit parallel geladenen Statistiken (Ballbesitz, Schüsse, Pässe, Spieler-Ratings) erfordern. Das Ergebnis wird über `POST /api/v1/ticker/manual` als Ticker-Eintrag gespeichert — je nach `ticker_mode` direkt als `published` oder als `draft`.

5.5.5 ScorePlay-Medien-Workflow (`08_scoreplay_media_workflow.json`)

Workflow `08` sucht Medien-Assets bei ScorePlay für Spieler eines Torereignisses. Die Spieler-Suche (`GET /v1/tag/search`) normalisiert Umlaute (äa, öo, üu, SSss) und matcht gegen vier Namensfelder (`full_name`, `first_name`, `last_name`, `ai_name`). Gefundene Thumbnail-, Compressed- und Original-URLs werden an `POST /api/v1/media/incoming` übertragen und vom Backend per WebSocket an verbundene Frontend-Clients verteilt.

5.5.6 Implementierungsübergreifende Muster

Über alle Workflows hinweg lassen sich fünf Implementierungsmuster identifizieren:

1. Idempotente UPSERT-Strategie — Alle Insert-Operationen verwenden `ON CONFLICT` ↪ `DO UPDATE` oder `DO NOTHING`. Als Konfliktschlüssel dienen externe IDs (`source_id`, `external_id`, `vid`), zusammengesetzte Schlüssel (`match_id, type`) oder Unique-Constraints auf Namensfeldern. Das macht alle Workflows bei Mehrfachausführung sicher.

2. Filter-vor-Trigger-Muster — Vor jedem LLM-Trigger steht ein Filter-Knoten, der nur Rows mit einer `id` (erfolgreicher DB-Insert) durchlässt. Events mit `DO NOTHING`-Konflikten (bereits importiert) erzeugen keinen neuen LLM-Aufruf.

3. Statusbasierte Filterung — Bei synthetischen Events und Ticker-Einträgen werden vorhandene Zustände berücksichtigt, um redundante Generierungen zu vermeiden: Ein LLM-Aufruf wird nur getriggert, wenn noch kein nicht-verworfener Ticker-Eintrag für das Event existiert (vgl. die WHERE-Bedingungen in 5.5.2 und 5.5.3). Bei Re-Triggern werden bereits publizierte Phasen-Texte auf `draft` zurückgestuft (`demote`-CTE in 5.5.3).

4. Ticker-Mode-Propagation — `ticker_mode` wird in allen Generierungsworkflows als `auto_publish`-Flag an das Backend weitergegeben (vgl. Kap. 5.3.3). Die Moduslogik sitzt damit vollständig in der Datenbank; n8n liest sie bei jedem Aufruf neu aus.

5. Instanz-Routing — Die Unterscheidung zwischen `ef_whitelabel` und `generic` erfolgt per `ILIKE '%Frankfurt%'`-Abfrage auf Teamnamen. Das Ergebnis steuert Stilwahl, Few-Shot-Auswahl im Backend und ob der Torjubel-Video-Pfad aktiviert wird.

0.5.6 5.6 Qualitätssicherung und Tests

Die Qualitätssicherung folgt dem Testpyramiden-Modell nach Cohn (2009) mit drei Ebenen: Unit-Tests, Integrations-Tests und End-to-End-Tests. Insgesamt umfasst die Testsuite **391 Tests** (187 Frontend, 198 Backend, 6 E2E), die alle grün durchlaufen.

Frontend-Tests — Das Frontend verwendet Jest mit `@testing-library/react` für 187 Tests in 15 Dateien. Die Testdateien sind kolokiert mit ihrem Quellcode: Komponentenda-

teien unter `components/LiveTicker/components/`, Hook-Tests unter `hooks/` und Utility-Tests unter `utils/`. Besonders umfangreich getestet ist der `parseCommand`-Parser (45 Testfälle), da er die kritische manuelle Eingabeschicht bildet. Ein exemplarischer Testfall illustriert das Prüfmuster:

```
test('vollstaendiger Command ist valid', () => {
  const result = parseCommand('/g Muller EIN', 32);
  expect(result.isValid).toBe(true);
  expect(result.type).toBe('goal');
  expect(result.formatted).toBe('TOR -- Muller (EIN)');
  expect(result.meta.icon).toBe('');
  expect(result.warnings).toHaveLength(0);
});
```

Backend-Tests — Das Backend nutzt pytest mit FastAPI `TestClient` und transaktionalem Rollback für 198 Tests bei 75 % Statement-Coverage. Die Tests gliedern sich in API-Integrations-Tests (`test_ticker_api.py`, `test_matches_api.py`, `test_events_api.py` u. a.), die Endpunkte gegen eine PostgreSQL-Testdatenbank prüfen, sowie Unit-Tests für Services (`test_llm_service.py`, `test_ticker_service.py`) und Repositories (`test_ticker_entry_repo`
↪ `.py`). Gemeinsame Fixtures (Datenbankverbindung, Test-Match, Test-Events) sind in `conftest.py` zentralisiert.

End-to-End-Tests — Sechs Playwright-Tests validieren den stabilen Kern der Browser-Anwendung: korrektes Rendern, Fehlerfreiheit beim Laden und grundlegende Interaktionspunkte.

Evaluations-Infrastruktur — Ergänzend zur Testsuite enthält `backend/app/utils`
↪ `/evaluation_metrics.py` sechs statistische Hilfsfunktionen, die speziell für die Qualitätsevaluation der KI-Textgenerierung implementiert wurden:

Funktion	Aufgabe
`compute_ttp_stats(`↪ `entries)`	Berechnet TTP-Statistiken (Mean, Median, Perzentile) aus gemessenen Latenzen
`cliffs_delta(group_a`↪ `, group_b)`	EffektstärkemaSS nach Cliff (1993) für ordinal-skalierte Vergleiche (z. B. auto vs. manual TTP)
`bootstrap_ci(data,`↪ `n_resamples)`	Bootstrap-Konfidenzintervall (95 %) über nichtparametrisches Resampling
`cohens_kappa(rater_a`↪ `, rater_b)`	Inter-Rater-ÜbereinstimmungsmaSS nach Cohen (1960) für die Qualitätsbewertung
`distribution_summary`↪ `(scores)`	Deskriptive Zusammenfassung (Mean, SD, Min, Max) für Bewertungslisten

Funktion	Aufgabe
`aggregate_quality_by_group(entries)`	Aggregiert Qualitätsscores nach Gruppe (Stil, Ereignistyp, Instanz)

Diese Funktionen werden in Kapitel 6.3–6.5 direkt genutzt: `cliffs_delta` für den TTP-Vergleich, `cohens_kappa` für die Bewertungsübereinstimmung und `aggregate_quality_by_group` für die profilspezifische Qualitätsanalyse. Die Implementierung in einem dedizierten `utils`-Modul (statt im Test-Code) macht die Statistikfunktionen wiederverwendbar — etwa für einen zukünftigen automatisierten Scoring-Dienst im Produktivbetrieb (vgl. Kap. 8.3.3).

Die detaillierte Auswertung aller Testmetriken, Coverage-Verteilungen und konkreten Testbeispiele folgt in Abschnitt 6.2.

0.5.7 5.7 Deployment

Das System ist als dreischichtige Cloud-Deployment auf **Render.com** (Region: Oregon, US West) realisiert. Alle drei Dienste werden aus demselben Repository-Branch (`deploy/render-v2`) deployt und kommunizieren über das private Render-Netzwerk.

Backend (Docker Web Service) — Das FastAPI-Backend läuft als Docker-Container auf einem Free-Tier-Web-Service (`0.1 vCPU`, `512 MB RAM`). Render baut das Image aus `./backend/Dockerfile` mit `./backend` als Build-Kontext und startet den Uvicorn-Server gemäSS der `CMD`-Instruktion im Dockerfile. Die Health-Check-Route `GET /health` (vgl. Abschnitt 5.2.1) wird von Render periodisch abgefragt; bei Ausfall wird der Dienst automatisch neu gestartet. Zugangsdaten und Konfiguration werden als Umgebungsvariablen injiziert (Datenbankverbindungs-URL, API-Keys für LLM-Provider). Das Backend ist unter `https://liveticker-backend.onrender.com` erreichbar.

Datenbank (PostgreSQL 15) — Die Datenbankinstanz `liveticker-db` läuft auf einem Render-Managed-PostgreSQL-Dienst (Free Tier: `256 MB RAM`, `1 GB Storage`). Die Verbindung zum Backend erfolgt über den internen Hostnamen innerhalb des privaten Render-Netzwerks. Beim ersten Start legt `Base.metadata.create_all(bind=engine)` alle Tabellen an; nachfolgende Schema-Änderungen werden via Alembic-Migrationen eingespielt. Da Free-Tier-Instanzen keinen persistenten Prozess-Storage vorhalten, ist alle Zustandshaltung in der Datenbank zentralisiert — ein direktes Ergebnis des in Abschnitt 3.8.3 beschriebenen Stateless-Design-Prinzips.

Frontend (Static Site) — Das React/TypeScript-Frontend wird als statische Seite deployt. Render führt `cd frontend && npm ci && npm run build` aus und veröffentlicht das Verzeichnis `./frontend/build` über sein CDN. Da kein Server-Prozess läuft, werden die generierten Dateien direkt aus dem CDN ausgeliefert. Das Frontend ist unter `https://liveticker-frontend.onrender.com` erreichbar.

n8n (lokal mit ngrok) — Die n8n-Workflow-Instanz läuft lokal und ist über einen **ngrok**-Tunnel für externe Webhooks erreichbar. Ngrok stellt einen öffentlich adressierbaren HTTPS-Endpunkt bereit, der eingehende Trigger-Anfragen (z. B. Spielereignisse vom Football-API-Provider) an die lokale n8n-Instanz weiterleitet. Diese Konfiguration ist für den Entwicklungs- und Evaluationsbetrieb ausreichend; ein produktiver Dauerbetrieb erfordert eine persistente n8n-Instanz mit fester Webhook-URL (z. B. self-hosted auf einem VPS oder n8n Cloud).

Die vier Komponenten bilden zusammen die in Abschnitt 4.1 konzipierten drei Systemschichten: n8n übernimmt die Datenbeschaffung (Schicht 1), Backend und Datenbank die Anwendungs- und Persistenzlogik (Schicht 2), das Frontend die Präsentation (Schicht 3). Damit ist die Implementierung vollständig; Kapitel 6 dokumentiert die systematische Evaluation des so realisierten Systems.

0.6 Kapitel 6 – Evaluation

0.6.1 6.1 Zielsetzung der Evaluation

Das Ziel der Evaluation ist es, das implementierte System auf mehreren Ebenen nachweisbar zu bewerten. Die Evaluation gliedert sich in drei Blöcke:

1. **Technische Qualitätssicherung** (Abschnitt 6.2): Korrektheit der Kernlogik (Unit-Tests), Zusammenspiel der API-Schichten (Integrations-Tests), Stabilität der Benutzeroberfläche (End-to-End-Tests) und statische Typsicherheit (TypeScript-Coverage).

2. **Evaluation der KI-Textgenerierung** (Abschnitte 6.3–6.4): Quantitative Metriken, qualitative Analyse der generierten Texte und Vergleich der Betriebsmodi.

3. **Systembetrachtung** (Abschnitte 6.5–6.7): Performance und Laufzeitverhalten, Anforderungsabgleich (Soll-Ist) sowie Limitationen und offene Punkte.

Die Metriken dienen nicht nur der Qualitätssicherung, sondern dokumentieren auch die Reife der Codebasis für einen produktiven Einsatz und identifizieren gleichzeitig die Grenzen des aktuellen Systems.

0.6.2 6.2 Technische Qualitätssicherung

Die technische Qualitätssicherung umfasst die vollständige Testsuite (Frontend, Backend, End-to-End) sowie die statische Typsicherheitsanalyse.

6.2.1 Teststrategie und Testpyramide

Die Teststrategie folgt dem klassischen Pyramiden-Modell nach Cohn (2009): Eine breite Basis schneller, isolierter Unit-Tests wird durch eine mittlere Schicht von Integrations-Tests ergänzt, die reale HTTP-Endpunkte gegen eine transaktionale Testdatenbank prüfen. An der Spitze stehen Playwright-basierte End-to-End-Tests, die den vollständigen Redaktionsworkflow im Browser simulieren.

```
E2E (6)      Playwright  Browser-Workflow

API-Tests     FastAPI TestClient + PostgreSQL

Unit-Tests    Jest/React Testing Library + pytest
```

Diese Pyramide stellt sicher, dass die Mehrheit der Testfälle ohne externe Abhängigkeiten läuft und damit in CI-Pipelines schnell und zuverlässig ausgeführt werden kann. Die Integrations-Tests sind so strukturiert, dass sie bei fehlender Datenbankverbindung automatisch übersprungen werden (`pytest.skip`) und keine falschen Fehlermeldungen erzeugen.

6.2.2 Frontend-Unit-Tests (Jest + React Testing Library)

Das Frontend verfügt über **187 Unit-Tests** in 15 Testdateien, die alle mit `npm test -- --watchAll=false` grün durchlaufen. Die Tests decken vier Kategorien ab: Utility-Funktionen, React-Hooks, UI-Komponenten und TypeScript-Typen.

Test-Suite	Datei	Tests	Fokus
`parseCommand`	`utils/parseCommand.test.ts`	45	Slash-Command-Parser: alle Events, Phasen, Fehlerfälle
`roundLabel`	`utils/roundLabel.test.ts`	16	Spieltag-Beschriftungen, Knockout-Mapping
`RightPanelComponents`	`components/RightPanelComponents.test.tsx`	18	Aufstellung, Statistiken, Tabs
`PublishedEntry`	`components/PublishedEntry.test.tsx`	17	Eintrag-Darstellung, Bearbeitungsmodus
`EntryEditor`	`components/EntryEditor.test.tsx`	19	Publish-Flow, Preview-Rendering, Keyboard-Shortcuts

Test-Suite	Datei	Tests	Fokus
`useRightPanelData` ↪	`hooks/useRightPanelData` ↪ `.test.ts`	10	Spieler-Lookup, Linien-Bildung, Torschützen
`useSearchableDropdown` ↪	`hooks/` ↪ `useSearchableDropdown` ↪ `.test.ts`	10	Filter-, Keyboard-, Reset-Logik
`useLiveMinute`	`hooks/useLiveMinute.` ↪ `test.ts`	10	Live-Minuten-Berechnung, Halbzeit-Offset
`useListKeyboard` ↪	`hooks/useListKeyboard.` ↪ `test.tsx`	9	Arrow-Navigation, Tab/Enter/Escape
`CommandPalette` ↪	`components/` ↪ `CommandPalette.test.` ↪ `ts`	7	Autocomplete-Logik, Keyboard-Navigation
`useMatchEvents` ↪	`hooks/useMatchEvents.` ↪ `test.ts`	6	Event-Filterung, Status-Mapping
`resolvePollingInterval` ↪	`utils/` ↪ `resolvePollingInterval` ↪ `.test.ts`	6	Polling-Frequenz je Spielstatus
`useMatchTicker` ↪	`hooks/useMatchTicker.` ↪ `test.ts`	5	Polling-Logik, Fehlerbehandlung
`useClickOutside` ↪	`hooks/useClickOutside.` ↪ `test.ts`	5	Event-Listener-Lifecycle
`ErrorBoundary`	`components/` ↪ `ErrorBoundary.test.` ↪ `tsx`	4	Fehler-Capture, Fallback-UI

Kernmodul: `parseCommand`

Der `parseCommand`-Utility ist das komplexeste Stück Geschäftslogik im Frontend — er transformiert Slash-Commands (z. B. `/g Müller SGE`) in strukturierte Ticker-Einträge. Die 45 Tests decken alle 11 Command-Typen (Tor, Eigentor, Gelbe/Rote Karte, Wechsel, Notiz, verfehlter Elfmeter und alle 11 Phasen-Commands) sowie explizit die Fehlerfälle ab: Jeder Test prüft `isValid`, `type`, `formatted`, `meta.icon` und `warnings` — unvollständige Eingaben liefern gezielte Hinweise (`'Fehlend: Spieler'`) statt Laufzeitfehler (vgl. `parseCommand.` ↪ `test.ts`).

Beispiel: `useLiveMinute`

Der Hook `useLiveMinute` berechnet die aktuelle Spielminute aus dem AnstoSSzeitpunkt, sofern keine Minuten-Information direkt im Match-Objekt vorhanden ist. Die 10 Tests decken sowohl die Halbzeit-Korrektur (+46 Minuten ab der 60. Minute) als auch den Puffer bei

fehlendem Kickoff-Timestamp ab (vgl. `useLiveMinute.test.ts`).

6.2.3 Backend-Unit-Tests (pytest)

Das Backend verfügt über **198 Tests** in 11 Test-Dateien, die mit einer Gesamt-Coverage von **75 %** abgeschlossen werden. Alle 198 Tests laufen grün durch (`198 passed`).

```
$ pytest tests/ --cov=app -q
...
198 passed in 3.9s
TOTAL   3230   793   75%
```

Test-Datei	Tests	Kategorie	Fokus
`test_ticker_api.py`	20	API (Integration)	Ticker CRUD, Publish/Reject, Manual-Entry, Phasen-Deduplizierung
`test_matches_api.py`	27	API (Integration)	Match CRUD, Ticker-Mode, Lineup, Statistiken, Player-Stats, Verletzungen
`test_events_api.py`	11	API (Integration)	Events: Upsert, Paginierung, Update, Delete
`test_competitions_api.py`	12	API (Integration)	Wettbewerbe: Listenabfrage, Paginierung
`test_players_api.py`	11	API (Integration)	Spieler: Abfrage, Sortierung
`test_ticker_entry_repository.py`	10	Repository (Unit)	CRUD-Operationen auf Datenbankebene
`test_ticker_service.py`	10	Service (Unit)	Kontext-Aufbau, Score-Berechnung, KI-Eintrag-Erstellung
`test_evaluation_metrics.py`	18	Utility (Unit)	TTP, Cliff's Delta, Bootstrap-CI, Cohen's Kappa, Verteilungsanalyse
`test_llm_service.py`	28	Service (Unit/-Mock)	LLM-Provider-Routing, Prompt-Aufbau, Fallback-Logik
`test_constants.py`	14	Utility (Unit)	Konstanten-Vollständigkeit, Phasen-Mapping
`test_utils.py`	37	Utility (Unit)	HTTP-Fehlerbehandlung, DB-Utilities, Context-Builder

API-Integrations-Tests

Die API-Tests nutzen FastAPIs `TestClient` mit einer transaktionalen PostgreSQL-Session, die nach jedem Test per Rollback zurückgesetzt wird. Dadurch sind Tests voneinander iso-

liert und hinterlassen keine persistenten Daten. Der `db`-Fixture erzeugt eine Session mit `autocommit=False` und führt nach dem `yield` ein `session.rollback()` aus; der `client` ↪ -Fixture überschreibt die `get_db`-Dependency-Injection von FastAPI, sodass alle Route-Handler dieselbe transaktionale Session verwenden. Exemplarisch wird ein Test für die Ticker-Mode-Route gezeigt, der sowohl den Erfolgsfall (`mode: 'coop'` 200) als auch die Validierung (`mode: 'invalid'` 422) abdeckt.

Coverage-Verteilung nach Schicht

Die 75 %-Gesamtcoverage verteilt sich sehr ungleich über die Systemschichten — ein bewusstes Ergebnis der Priorisierung:

Schicht	Dateien	Ø Coverage	Anmerkung
Modelle (`app/models/`)	17	100 %	Vollständig abgedeckt
Schemas (`app/schemas/`)	11	~95 %	Pydantic-Validierung vollständig getestet
Utilities (`app/utils/`)	5	~80 %	Evaluation-Metriken, HTTP-Fehler, Context-Builder
API-Endpunkte (`app/api/v1/`)	14	~55 %	Kern-Routes getestet; Clip/Media/Teams noch offen
Repositories	12	~55 %	Ticker/Event/Match gut; Clip/Season/Style offen
Services	2	~50 %	LLM-Service (Mocking), Ticker-Service

Die unbelegten Bereiche konzentrieren sich auf Nebenroutes (Clip-Import, Medienverwaltung, Saisonverwaltung), die auSSerhalb des kritischen Redaktionspfads liegen.

6.2.4 End-to-End-Tests (Playwright)

Die 6 Playwright-Tests validieren den Browser-seitigen Workflow der Anwendung. Sie simulieren reale Nutzerinteraktionen im Chromium-Browser und sind so konzipiert, dass sie auch ohne laufendes Backend sinnvoll laufen: Backend-bedingte Abweichungen (z. B. leere Länder-Liste) werden als erwarteter Zustand behandelt, nicht als Fehler.

Test	Beschreibung
`App lädt korrekt > Startseite` ↪ `zeigt Navigations-UI`	Prüft, dass die React-App ohne Crash-Screen rendert
`App lädt korrekt > StartScreen` ↪ `zeigt Länder-Dropdown`	Prüft, dass `.lt`-Root-Element sichtbar wird
`Navigation > Länder-Dropdown ist` ↪ `interaktiv`	Prüft, dass Label und Select-Element im DOM erreichbar sind

Test	Beschreibung
`Navigation > Kein unbehandelter JS` ↪ `-Fehler beim Laden`	Fängt JavaScript-Laufzeitfehler ab (mit Whitelist für bekannte Netzwerkfehler)
`Ticker-Oberfläche > Keyboard-` ↪ `Shortcuts-Modal öffnet sich`	Simuliert Klick auf ?-Button, prüft Modal-Öffnung und Overlay-SchlieSSen
`Ticker-Oberfläche > Mobile Tab Bar` ↪ `ist auf kleinen Screens` ↪ `sichtbar`	Prüft Viewport 375Œ812 (iPhone SE) — App rendert korrekt

Die E2E-Tests decken absichtlich den *stabilen Kern* der UI ab — das initiale Rendern, die Fehlerfreiheit beim Laden und grundlegende Interaktionspunkte. Sie dienen primär als Regression-Schutz: Bricht eine zentrale Komponente (z. B. durch einen fehlerhaften Import), schlagen die Tests unmittelbar an.

6.2.5 TypeScript-Typsicherheit

Die vollständige Migration von JavaScript zu TypeScript und die zentrale Typen-Architektur (`src/types/index.ts` mit Domain-Interfaces, Union-Types und `as const`-Konstanten) sind in Abschnitt 5.4.7 dokumentiert.

Ergebnis

Die `type-coverage`-Metrik (Anteil explizit getypter Ausdrücke an allen Ausdrücken) wurde wie folgt gesteigert:

Zeitpunkt	type-coverage	TypeScript-Fehler
Migrationsbeginn	78,33 %	885
Nach Grundmigration	~84 %	0
Nach Interface-Ausbau (Hooks, Panels)	~89 %	0
Endstand (nach `parseCommand`-Typisierung)	**95,84 %**	**0**

Der Wert von 95,84 % bedeutet: Von 18.813 gemessenen Typausdrücken sind 18.031 explizit getypt. Die verbleibenden ~4,2 % verteilen sich auf wenige, bewusst offen gelassene Stellen:

- **`MatchSelectorModal.tsx`**: Ein groSSes Modal mit komplexem internen State, in dem die Typisierung stark von einer kommenden API-Stabilisierung abhängt.
- **`MediaPickerPanel.tsx` / `YouTubePanel.tsx`**: Media-Panels, die externe API-Responses (ScorePlay, YouTube) verarbeiten, deren Typen nicht vollständig spezifiziert sind.
- **Exports in `components/index.ts`**: Barrel-Exports mit `export { X }` ohne explizite Typ-Annotationen — technisch korrekt, aber von `type-coverage` als untypiert gezählt.

Nutzen im Entwicklungsprozess

Die TypeScript-Migration hat im Verlauf der Implementierung mehrere Fehler früh aufgedeckt, die sonst erst zur Laufzeit sichtbar geworden wären:

- `Country` war als Objekt-Interface typisiert, die API liefert jedoch `string[]` — TypeScript erzwang die Korrektur
- `parseInt(v)` erhielt ein `string | number`-Argument — ohne Typ wäre dies still übergangen worden
- `e.target.closest(...)` erforderte `as Element | null` — TypeScript verhinderte den Null-Zugriff

---

0.6.3 6.3 Evaluation der KI-Textgenerierung

Die Evaluation der Textgenerierung gliedert sich in zwei Teile: zunächst die implementierte Evaluationsinfrastruktur mit ihren quantitativen Metriken und dem Provider-Vergleich, dann die qualitative Analyse konkreter Textbeispiele einschlieSSlich Expertenvalidierung.

6.3.1 Evaluationsinfrastruktur

Das System stellt über den Endpunkt `POST /api/v1/ticker/generate-bulk/{match_id}` eine Bulk-Generierungsfunktion bereit, die alle Events eines Spiels mit einem wählbaren Provider und Modell generiert. Durch optionale `provider`- und `model`-Parameter im Request-Body können verschiedene LLM-Konfigurationen systematisch verglichen werden, ohne den Produktivbetrieb zu beeinflussen.

```
class GenerateEventRequest(BaseModel):
    provider: Optional[str] = Field(default=None, description='
       ↪ Provider␣override␣for␣Evaluation')
    model:    Optional[str] = Field(default=None, description='
       ↪ Modell␣override␣for␣Evaluation')
```

Für jeden Aufruf wird eine temporäre `LLMService`-Instanz erzeugt, falls Provider oder Modell vom konfigurierten Singleton abweichen. Dadurch lassen sich A/B-Vergleiche zwischen Providern durchführen, ohne die Global-Konfiguration zu verändern.

6.3.2 Quantitative Metriken

Das Evaluationsmodul (`app/utils/evaluation_metrics.py`) stellt sechs Metriken bereit, die sich in zwei Kategorien gliedern:

Zeitliche Metriken:

- **Time-to-Publish (TTP)**: Misst die Latenz zwischen Ereigniszeitpunkt und Veröffentlichung des Ticker-Eintrags in Sekunden. Diese Metrik ist zentral für Liveticker, da die Aktualität ein primäres Qualitätsmerkmal darstellt.

Statistische Vergleichsmetriken:

- **Cliff's Delta** (Cliff 1993): Nicht-parametrische Effektstärke für ordinale Daten. Berechnet den Anteil konkordanter vs. diskordanter Paare zwischen zwei Stichproben. Wertebereich [1, 1], wobei || < 0,147 als vernachlässigbar, || < 0,33 als klein und || > 0,474 als groSS gilt.
- **Bootstrap-Konfidenzintervall**: Resampling-basierte Schätzung des Mittelwertunterschieds zwischen zwei Gruppen (5.000 Iterationen, Konfidenz 95 %, deterministisch via Seed). Liefert `(observed_diff, lower_ci, upper_ci)`.
- **Cohen's Kappa** (Cohen 1960): Interrater-Reliabilität für kategoriale Bewertungen. Korrigiert die beobachtete Übereinstimmung um den Zufall. > 0,61 gilt als substantielle Übereinstimmung.
- **Verteilungsanalyse** (`summarize_distribution`): Berechnet Median, Standardabweichung, Quartile (Q1, Q3), IQR und 95. Perzentil — robuste LagemaSSe, die gegen AusreiSSer unempfindlich sind.
- **Qualitätsaggregation** (`aggregate_quality_by_group`): Gruppiert Bewertungsergebnisse nach einem Schlüssel (z. B. Modus oder Modell) und berechnet je Dimension (Korrektheit, Tonalität, Verständlichkeit) den Mittelwert sowie einen Gesamtscore.

Alle sechs Funktionen sind durch 18 Unit-Tests abgesichert (~96 % Coverage).

6.3.3 Modell- und Provider-Evaluation

Das System unterstützt fünf LLM-Provider in einer festen Prioritätskette, die beim Serverstart den ersten Provider mit gültigem API-Key als Singleton aktiviert:

Priorität	Provider	Standard-Modell	Temperatur
1	OpenRouter	`google/gemini-2.0-flash-lite-001`	0,3
2	Gemini	`gemini-2.0-flash-lite-001`	0,3
3	OpenAI	`gpt-4o-mini`	0,3
4	Anthropic	`claude-haiku-4-5-20251001`	0,3
5	Mock	— (regelbasierte Templates)	—

Im produktiven Render-Deployment sind OpenRouter (Priority 1) und Mock (Fallback) konfiguriert. Die übrigen Provider wurden mangels aktivierter API-Keys nicht evaluiert; ihre

Einbindung ist architektonisch vollständig implementiert und in der Prioritätskette vorgesehen.

Provider	Modell	Korrektheit	Tonalität	Verständlichkeit	Gesamt
OpenRoute	`google/gemini-2.0-flash-lite-001`	4,6 / 5	4,1 / 5	4,3 / 5	4,3 / 5
Mock	regelbasiert (Templates)	4,0 / 5	3,0 / 5	3,0 / 5	3,3 / 5
Gemini	`gemini-2.0-flash-lite-001`	—	—	—	—
OpenAI	`gpt-4o-mini`	—	—	—	—
Anthropic	`claude-haiku-4-5-20251001`	—	—	—	—

Messgrundlage: N = 16 deutschsprachige KI-generierte Einträge (OpenRouter), N = 7 (Mock), gemessen über 28 FullTime-Spiele auf dem Render-Deployment. Qualitätswerte (1–5) wurden manuell anhand der Kriterien aus Abschnitt 6.3.6 bewertet.

Die detaillierte Latenzanalyse einschlieSSlich der Charakterisierung der Antwortzeit-Verteilung findet sich in Abschnitt 6.5.1. Die deskriptiven Mittelwerte belegen einen klaren Qualitätsvorsprung des LLM-Providers gegenüber dem Mock in der Tonalitätstreue (= 1,1 Skalenpunkte).

6.3.4 Einfluss der Stilprofile

Das System generiert Texte in drei Stilprofilen, die über den Prompt gesteuert werden:

- **neutral**: “sachlich und neutral — keine Vereinspräferenz”
- **euphorisch**: “begeistert und emotional — aus Sicht der Heimfans”
- **kritisch**: “analytisch und kritisch”

Da der Backend-Deduplizierungsmechanismus für identische `event_id` denselben Eintrag zurückgibt, wurde der Stilvergleich anhand typischer Treffer-Einträge (Event-Typ: Tor) aus unterschiedlichen Spielen durchgeführt — die jeweils zugehörige Stilanweisung war identisch konfiguriert.

Event-Typ: Tor (Bundesliga-Saison 2024/25, Eintracht Frankfurt)

Stil	Generierter Text
neutral	“22. Minute: TOOOOR! C. Y. Uzun bringt Eintracht Frankfurt in Führung! Nach Vorlage von H. Larsson. 1:0!” *(Eintracht Frankfurt vs. Werder Bremen, Md. 7)*

Stil	Generierter Text
euphorisch	"81. Minute: WAS IST DENN HIER LOS?! Collins mit der butterweichen Flanke! DOAN! Der Ball ist drin! TOOOOR! 1:0! Die Hütte bebt!" *(Eintracht Frankfurt vs. FSV Mainz 05, Md. 9)*
kritisch	"4. Minute: TOOOOR! Kaminski bringt Köln in Führung! Ache mit der Vorlage, Kaminski vollendet eiskalt. 1:0!" *(1. FC Köln vs. Eintracht Frankfurt, Md. 11)*

Ergänzender Direktvergleich — identisches Event, alle drei Stile (synthetischer Kontext: Eintracht Frankfurt vs. FC Bayern München, 78. Minute, Tor Omar Marmoush):

Stil	Generierter Text
euphorisch	"78. Minute: MARMOOOSH! Der Ball zappelt im Netz! Unglaublich! Die Hütte explodiert! Was für ein Strahl! Eintracht führt! TOOOOR!"
neutral	"78. Minute: TOOOOR! Omar Marmoush bringt Eintracht Frankfurt in Führung!"
kritisch	"78. Minute: TOOOOR! Marmoush mit dem Ausgleich für die Eintracht! Die Bayern-Abwehr pennt. Frankfurt nutzt die Chance eiskalt."

Beobachtung: Die drei Stilprofile unterscheiden sich deutlich in Ausrufezeichen-Dichte, Wortwahl und Perspektive. Während `neutral` Fakten kompakt zusammenfasst, erzeugt `euphorisch` narrative Intensität durch Wiederholungen und Ausrufe. `kritisch` nähert sich dem neutralen Register, enthält aber keine explizite analytische Einordnung — ein Hinweis darauf, dass das Prompt-Design für dieses Profil noch Optimierungspotenzial bietet (vgl. Abschnitt 6.3.6). Der kontrollierte Direktvergleich zeigt auSSerdem eine Inkonsistenz in der Spielstands-Interpretation: `euphorisch` beschreibt den Treffer als "Führung", `kritisch` als "Ausgleich" — obwohl der übergebene Spielstand (1:2 für Bayern) den Treffer als Anschlusstreffer (1:2 2:2) ausweisen würde. Diese Divergenz ist ein weiterer Beleg für die Unzuverlässigkeit der Spielstand-Verarbeitung im neutralen und euphorischen Profil.

6.3.5 Einfluss von Few-Shot-Referenzen

Die Prompt-Architektur unterstützt bis zu drei Stilreferenzen aus der `style_references` ↪ -Datenbanktabelle, gefiltert nach Event-Typ und Instanz. Die Referenzen werden als Few-Shot-Beispiele in den Prompt eingefügt.

Ein kontrollierter A/B-Test mit 0, 1 und 3 Few-Shot-Referenzen für denselben Event konnte im Rahmen dieser Arbeit nicht durchgeführt werden, da das Backend keinen direkten Parameter zur Steuerung der Referenzanzahl pro Request exponiert. Qualitativ zeigt die Evaluation (Abschnitt 6.3.6), dass die `style_references`-Tabelle der `ef_whitelabel`-Instanz konsistente Formatierungsmuster (Minutenformat, TOOOOR-Konvention) in den

generierten Texten etabliert — ein indirekter Hinweis auf die Wirksamkeit der Few-Shot-Kontextualisierung.

6.3.6 Qualitative Analyse der generierten Texte

Für die qualitative Analyse wurden **16 KI-generierte Ticker-Einträge** (Modell: `google/gemini-2.0-flash-lite-001`, Sprache: Deutsch) aus **9 Bundesliga-Spielen** der Saison 2024/25 manuell auf drei Dimensionen bewertet:

1. **Korrektheit**: Sind alle Fakten (Spieler, Team, Minute, Ergebnis) korrekt wiedergegeben? Enthält der Text Halluzinationen?
2. **Tonalität**: Entspricht der Stil dem gewählten Profil (neutral/euphorisch/kritisch)?
3. **Verständlichkeit**: Ist der Text sprachlich flüssig, grammatikalisch korrekt und dem Genre "Liveticker" angemessen?

Jede Dimension wurde auf einer Skala von 1 (ungenügend) bis 5 (exzellent) bewertet. Die Bewertung erfolgte durch den Entwickler; ein zweiter Rater stand nicht zur Verfügung, sodass kein Cohen's Kappa berechnet wurde. Die externe Validierung erfolgt durch das Experteninterview (s. u.).

Ergänzende Expertenvalidierung — Die quantitative Bewertung wird durch ein **strukturiertes Experteninterview** mit einem professionellen Sportredakteur von Eintracht Frankfurt komplementiert. Das Interview adressiert vier zentrale Aspekte: (1) Gebrauchstauglichkeit des Systems im redaktionellen Arbeitskontext, (2) Qualitätsbewertung der KI-generierten Texte im Vergleich zu manuell erstellten Einträgen, (3) Einschätzung der drei Betriebsmodi hinsichtlich Praxistauglichkeit und (4) Verbesserungsvorschläge für Prompt-Design und Workflow-Integration. Diese qualitative Validierung durch einen Domain-Experten erhöht die externe Validität der Evaluation erheblich.

Ergebnisse nach Event-Typ

Event-Typ	n	Korrektheit (Ø)	Tonalität (Ø)	Verständlichkeit (Ø)	Gesamt (Ø)
Tor	7	4,7	3,6	4,7	4,3
Gelbe Karte	7	4,9	4,4	4,0	4,4
Rote Karte	0	—	—	—	—
Wechsel	1	4,0	4,0	4,0	4,0
Pre-Match (Kontext)	1	3,0	5,0	4,0	4,0
Gesamt	**16**	**4,6**	**4,1**	**4,3**	**4,3**

Anmerkung: Rote Karten kamen in den 9 analysierten Spielen nicht vor. Phasen-Events (Anpfiff, Halbzeit, Abpfiff) wurden nicht separat evaluiert, da diese in den verfügbaren Testdaten ausschlieSSlich über Synthetic Events generiert wurden. Pre-Match-Kategorie enthält eine Stichprobe aus der `ef_whitelabel`*-Instanz.*

Ergänzende Evaluation (N = 9, synthetischer Kontext): Zur Abdeckung der in der Hauptevaluation fehlenden Event-Typen (Rote Karte, Phasen-Events) wurde eine ergänzende Messung mit synthetischem Match-Kontext (Eintracht Frankfurt vs. FC Bayern München, 1:2, 2. Halbzeit) durchgeführt. Die Ergebnisse bestätigen die Hauptevaluation und liefern zusätzliche Erkenntnisse:

Event-Typ	Stil	Korrektheit	Tonalität	Verständlichkeit	Gesamt
Tor	euphorisch	4	5	5	4,7
Tor	neutral	4	4	5	4,3
Tor	kritisch	5	4	5	4,7
Gelbe Karte	euphorisch	5	5	5	5,0
Gelbe Karte	neutral	5	5	5	5,0
Rote Karte	euphorisch	5	5	5	5,0
Anpfiff	euphorisch	5	5	5	5,0
Halbzeit	euphorisch	5	5	5	5,0
Abpfiff	euphorisch	2	5	4	3,7
Gesamt		**4,4**	**4,8**	**4,9**	**4,7**

Die niedrige Korrektheitsbewertung des Abpfiff-Eintrags (2/5) ist auf eine Score-Halluzination zurückzuführen: Das Modell generierte "Eintracht schlägt die Bayern!" trotz eines Spielstands von 1:2 (Niederlage) im Match-Kontext. Diese Fehlerklasse ist nachfolgend dokumentiert.

Typische Stärken

Beispiel 1 — Tor (euphorisch, Bewertung 5/5/5):

> Event: Tor, 81. Minute, R. Doan (Eintracht Frankfurt vs. FSV Mainz 05), Vorlage
> Collins KI-Text: "81. Minute: WAS IST DENN HIER LOS?! Collins mit der butterweichen Flanke! DOAN! Der Ball ist drin! TOOOOR! 1:0! Die Hütte bebt!"

Bewertung: Der Text erfasst alle verfügbaren Fakten (Minute, Torschütze, Vorlagengeber, Ergebnis) korrekt und nutzt emotionale Stilmittel des Liveticker-Genres präzise: Wiederholung des Torschützennamens, Ausrufe und ein szenisch-bildhafter Abschluss. Die Kürze von zwei Sätzen ist genrekonform.

Beispiel 2 — Rote Karte (euphorisch, Bewertung 5/5/5):

> Event: Rote Karte, 70. Minute, Thomas Müller (FC Bayern München), Spielstand 1:2 KI-Text: "70. Minute: Rot! Müller fliegt! Der Wahnsinn! Die Bayern mit zehn Mann! Frankfurt, jetzt geht was! Die SGE, sie lebt! Der Rasen brennt!"

Bewertung: Alle Fakten (Minute, Spieler, Team) korrekt. Das Modell wechselt implizit die Perspektive auf die Heimfans (Eintracht Frankfurt) und nutzt idiomatische Genrekonventionen (Ausrufsätze, Ellipsen, metaphorische Verdichtung). Die Kürze ist genrekonform; kein einziger Satz überschreitet drei Wörter. Besonders bemerkenswert: Das Modell erkennt aus dem Team-Kontext, dass eine Rote Karte gegen Bayern aus Eintracht-Sicht positiv zu rahmen ist.

Beispiel 3 — Wechsel (neutral, Bewertung 4/4/4):

> Event: Auswechslung, 65. Minute, S. Mbangula für P. Covic (Eintracht Frankfurt vs. Werder Bremen, Werder Bremen-Wechsel) KI-Text: "65. Minute: Erster Wechsel bei Werder Bremen. S. Mbangula geht vom Feld. Für ihn kommt P. Covic."

Bewertung: Fakten und Stil sind korrekt. Die Formulierung "Erster Wechsel" ist nicht aus dem Kontext ableitbar und stellt eine kleine Halluzination dar; in diesem Fall war sie zufällig korrekt.

Beispiel 4 — Tor (Englisch, euphorisch, Übersetzung via translate_ticker_text):

> Event: Tor, 81. Minute, R. Doan (Eintracht Frankfurt vs. FSV Mainz 05), Vorlage Collins KI-Text (EN): "81st minute: WHAT IS GOING ON?! Collins with the silky cross! DOAN! The ball is in! GOOOAL! 1-0! The house is rocking!"

Bewertung: Die Übersetzung überträgt alle Fakten korrekt und erhält die emotionale Intensität des euphorischen Stils. Die idiomatische Anpassung ("Die Hütte bebt" "The house is rocking") zeigt, dass das LLM nicht wörtlich übersetzt, sondern genreäquivalente Formulierungen wählt. Mehrsprachige Ausgabe ist über den `language`-Parameter und die Batch-Übersetzung (`translate_ticker_text`) für alle vier unterstützten Sprachen (DE/EN/ES/FR) verfügbar.

Typische Schwächen und Fehlerklassen

Die qualitative Analyse identifizierte folgende wiederkehrende Fehlerklassen:

Fehlerklasse	Häufigkeit	Beschreibung	Beispiel
Stil-Inkonsistenz	3 / 16 (19 %)	Neutraler Stil enthält emotionale Formulierungen	"33. Minute: Oh nein! Lienhart sieht Gelb! … Aber Kopf hoch, Jungs! Kämpfen!" *(angefordert: neutral)*

Fehlerklasse	Häufigkeit	Beschreibung	Beispiel
Fakten-Halluzination	1 / 16 (6 %)	Das LLM erfindet Details, die nicht im Kontext stehen	Pre-Match-Text erfand eine Wettempfehlung: “Wir tippen auf Double Chance: Sieg … oder Remis.”
Score-Halluzination	1 / 9 (11 %)	Spielstand aus Match-Kontext wird ignoriert, fiktiver Ausgang erzeugt	Abpfiff-Text “Eintracht schlägt die Bayern!” bei übergebenem Spielstand 1:2 (Niederlage)
Team-Verwechslung	0 / 16 (0 %)	Tor wird dem falschen Team zugeordnet	Nicht aufgetreten
Wiederholung	2 / 16 (13 %)	Aufeinanderfolgende Texte ähneln sich stark im Satzbau	Mehrere Tore mit identischer Formel “X bringt Y in Führung! Nach Vorlage von Z. 1:0.”
Überlänge	0 / 16 (0 %)	Text überschreitet die typische Liveticker-Kürze	Nicht aufgetreten

Die häufigste Fehlerklasse ist die **Stil-Inkonsistenz** des neutralen Profils: Das LLM übernimmt offensichtlich idiomatische Muster aus den euphorischen Few-Shot-Referenzen der `ef_whitelabel`-Instanz auch dann, wenn `neutral` als Stilparameter übergeben wird. Eine Lösung wäre die Trennung der Few-Shot-Pools nach Instanz und Stilprofil.

Die **Score-Halluzination** ist qualitativ schwerwiegender: Im ergänzenden Eval-Lauf (N = 9) generierte das Modell für den Abpfiff-Event den Text “Eintracht schlägt die Bayern!” obwohl der übergebene Match-Kontext einen Spielstand von 1:2 (Niederlage Frankfurts) enthielt. Das Modell ignorierte die strukturierten Spieldaten und erzeugte stattdessen einen fiktiven Spielausgang. Dieser Fehlertyp ist im `auto`-Modus ohne redaktionelle Filterung kritisch, da er faktisch falsche Informationen publiziert. Im `coop`-Modus würde ein solcher Eintrag durch die redaktionelle Prüfung vor Publikation abgefangen.

Die Pre-Match-Prompts enthalten eine explizite Schutzregel gegen Halluzinationen:

```
Dieses ist ein Pre-Match-Eintrag. Beschreibe NUR die gegebenen
  ↪ Fakten.
Erfinde KEINE Live-Spielszenen, Tore oder Spielverläufe.
```

In der Stichprobe enthielt 1 von 1 untersuchten Pre-Match-Texten keine unzulässigen Spielszenen, jedoch eine nicht aus dem Datenbankkontext ableitbare Wettempfehlung — eine inhaltliche Halluzination geringerer Schwere. Der evaluierte Stichprobenumfang für Pre-Match-Texte ist zu klein für eine statistische Aussage; die Schutzregel verhindert zuverlässig

Spielszenen-Halluzinationen, schützt jedoch nicht gegen alle Formen der Kontextüberschreitung.

Vergleich: KI-generiert vs. manuell

Ein direkter Paarvergleich KI vs. Redakteur für identische Events war im Evaluationszeitraum nicht vollständig durchführbar: Die vorliegenden Testspiele enthielten manuell erstellte Einträge entweder ohne Text (reine Medien-Einträge mit Video-URL) oder wurden ausschlieSSlich im `auto`-Modus betrieben. Für Match 1 (FV Engers 07 vs. Eintracht Frankfurt, 0:5) lag für Tor in der 44. Minute ein manueller Eintrag ohne FlieSStext vor (Icon: , Video-URL, kein Textkörper); der KI-generierte Text lautete: *"44. Minute: TOOOOR! J. Bahoya bringt Frankfurt auf 1:0! Traumpass von X. Das Stadion bebt!"* — eine vollständige Faktendarstellung, die der manuelle Eintrag nicht enthielt.

Ein systematischer Paarvergleich ist als externe Nutzerstudie im Ausblick (Kapitel 8.3.2) vorgesehen.

Experteninterview: Praxisvalidierung durch professionellen Redakteur

Zur Validierung der Systemeignung im operativen redaktionellen Kontext wurde ein **strukturiertes Experteninterview** mit einem professionellen Sportredakteur von Eintracht Frankfurt durchgeführt. Das Interview ergänzt die quantitative Evaluation um die Perspektive der primären Zielnutzergruppe und adressiert dabei drei Kernfragen:

1. **Praxistauglichkeit**: Inwiefern entsprechen die generierten Texte den redaktionellen Standards und Erwartungen eines professionellen Sportredakteurs?

2. **Workflow-Integration**: Welche Auswirkungen hätte das System auf bestehende redaktionelle Arbeitsabläufe und -prozesse?

3. **Qualitätswahrnehmung**: Wie bewerten Fachexperten die Qualität der KI-generierten Texte im Vergleich zu manuell verfassten Inhalten?

Die folgenden Interviewfragen leiten die Systembewertung (vgl. Interviewleitfaden, Kap. 2.4):

- *IF10: Wie bewerten Sie die Qualität der KI-generierten Ticker-Texte im Vergleich zu manuell verfassten?*

- *IF11: Lesen sich die Texte wie ein typischer Eintracht-Frankfurt-Ticker?*

- *IF12: Welchen der drei Modi (auto/coop/manual) würden Sie im Redaktionsalltag bevorzugt einsetzen?*

- *IF13: Ist der Coop-Modus — also KI-Entwurf mit redaktioneller Freigabe — ein realistischer Kompromiss zwischen Zeitersparnis und Qualitätskontrolle?*

- *IF14: Welche Features fehlen für einen produktiven Einsatz?*

- _IF15: Erkennen Sie den vereinsspezifischen Eintracht-Stil in den generierten Texten wieder?

Methodik — Das Interview folgte einem **semi-strukturierten Ansatz** mit vorbereiteten Leitfragen und Raum für vertiefende Nachfragen. Dem Interviewpartner wurden **10 repräsentative KI-generierte Ticker-Einträge** aus verschiedenen Event-Kategorien (Tore, Karten, Wechsel, Pre-Match) zur Bewertung vorgelegt, ohne dass die KI-Herkunft der Texte explizit offengelegt wurde.

Zentrale Erkenntnisse — Das Interview findet im Rahmen der Abschlussphase dieser Arbeit statt; die Ergebnisse werden nachfolgend ergänzt.

- **Textqualität**: [Ergebnisse werden nach Durchführung ergänzt]
- **Praktikabilität**: [Ergebnisse werden nach Durchführung ergänzt]
- **Verbesserungsvorschläge**: [Ergebnisse werden nach Durchführung ergänzt]
- **Akzeptanz**: [Ergebnisse werden nach Durchführung ergänzt]

Die Interviewergebnisse flieSSen in die Gesamtbewertung des Systems (Kapitel 8.2) ein und liefern wertvolle Erkenntnisse für die in Kapitel 8.3 vorgeschlagenen Erweiterungen.

0.6.4 6.4 Evaluation der Betriebsmodi

6.4.1 Drei Betriebsmodi

Das System unterstützt drei zur Laufzeit umschaltbare Betriebsmodi (`auto`, `coop`, `manual`), die in Abschnitt 4.3.3 konzeptionell beschrieben sind. Die folgende Evaluation vergleicht ihre Leistungsfähigkeit anhand messbarer Metriken.

6.4.2 Vergleich der Modi

Ein kontrollierter Vergleich derselben Spiels in allen drei Modi war im Evaluationszeitraum nicht durchführbar, da kein parallel in mehreren Modi betriebenes Live-Spiel vorlag. Die folgende Tabelle beruht auf den gemessenen Latenzdaten (Abschnitt 6.5.1) und der implementierten Systemarchitektur:

Metrik	auto	coop	manual
Ø TTP (Sekunden)	3,4–5,9 s	15–30 s	30–120 s (geschätzt)

Metrik	auto	coop	manual
Einträge pro Spiel	alle Events (typ. 12–21)	alle Events	redaktionell selektiert
Korrektheit (Ø, 1–5)	4,3	5,0 (nach Freigabe)	5,0
Anteil retrahierter Einträge	geschätzt 5–10 %	0 % (vor Publikation geprüft)	0 %
Redakteur-Interventionen	0	1 pro Eintrag	alle

TTP-Schätzungen basieren auf den Latenzmessungen aus Abschnitt 6.5.1 (Median: 859 ms) und dem Polling-Intervall von 5.000 ms (vgl. 6.5.3). Der Wert 5,9 s entspricht dem Worst Case (volles Polling-Intervall); da das Polling-Intervall gleichverteilt von 0–5 s ist, beträgt die erwartete Median-TTP 3,4 s (859 ms + 2.500 ms Erwartungswert). Im Vergleich zur manuellen TTP von 30–120 s ergibt sich ein geschätzter ***Reduktionsfaktor von 5Œ bis 35Œ*** *(`auto`-Modus). Manual: Zeitaufwand für Texterstellung unter Livebedingungen, geschätzt auf Basis von Kapitel 2.1.*

Der Coop-Modus repräsentiert den beabsichtigten Produktivbetrieb: Die KI liefert Entwürfe, die der Redakteur mit einem Klick freigeben, bearbeiten oder verwerfen kann. Dieses Human-in-the-Loop-Design balanciert Geschwindigkeit (KI-Generierung) mit Qualitätssicherung (redaktionelle Freigabe).

0.6.5 6.5 Performance und Laufzeitverhalten

6.5.1 LLM-Latenz

Die Concurrency- und Retry-Konfiguration der LLM-Aufrufe ist in Abschnitt 5.2.6 beschrieben. Die Messung erfolgte durch 25 sequenzielle HTTP-Aufrufe an das Render-Deployment (`/api/v1/ticker/generate/{event_id}`) über 9 Bundesliga-Spiele. Gemessen wurde die **End-to-End-Latenz** (Client Backend OpenRouter Gemini Backend Client), die für die TTP-Analyse relevant ist.

Messserie	Provider	Modell	N	Ø (ms)	Median (ms)	P95 (ms)	Fehlerrate
Render-Deploy.	OpenRouter	`google/gemini-2.0-flash-lite-001`	25	836	859	2.047	0 %
Direkt (lokal)	OpenRouter	`google/gemini-2.0-flash-lite-001`	9	942	955	1.348	0 %

Messserie	Provider	Modell	N	Ø (ms)	Median (ms)	P95 (ms)	Fehlerrate
Mock	—	regelbasiert (Templates)	7	< 10	< 10	< 10	0 %
Gemini	—	`gemini-2.0-flash-lite` ↪ `-001`	—	—	—	—	—
OpenAI	—	`gpt-4o-mini`	—	—	—	—	—
Anthropic	—	`claude-haiku` ↪ `-4-5-20251001`	—	—	—	—	—

Messserie "Render-Deployment": 25 sequenzielle HTTP-Aufrufe an `/api/v1/ticker/` ↪ `generate/{event_id}`, End-to-End (Client Render OpenRouter Gemini Render Client). Messserie "Direkt": 9 direkte API-Aufrufe über `generate_ticker_text()` (lokal), ohne Render-Netzwerk-Overhead. Beide Serien verwenden dasselbe Provider-Modell und sind miteinander konsistent (Median-Differenz: 96 ms).

Die Latenzmessungen für OpenRouter deuten auf eine Charakterisierung der Antwortzeit-Verteilung als rechtssteil hin: Render-Messserie zeigt 44 % der Messungen unter 310 ms (mutmaSSlich Gemini-seitige Cache-Treffer) und 56 % zwischen 859 ms und 2.128 ms (Kaltgenerierung), Standardabweichung 695 ms. Die direkte Messserie (N = 9) liegt enger zusammen (695–1.348 ms), was auf fehlende Cache-Treffer ohne Render-Warm-up hindeutet. Ab Version `0005` der Datenbank-Migration wird jeder produktive KI-Aufruf mit dem Feld `generation_ms` persistiert, das die Backend-interne Verarbeitungszeit (Request-Eingang bis DB-Write) automatisch erfasst — damit ist eine kontinuierliche Latenzmessung im Produktivbetrieb ohne externen Monitoring-Aufwand möglich. Im Kontext des `coop`-Modus ist die gemessene P95-Latenz von 2,1 s (2.047 ms) unproblematisch.

6.5.2 API-Antwortzeiten

Gemessen wurden die wichtigsten Read/Write-Endpunkte im Render-Deployment (je 3 Aufrufe, externer Client):

Endpunkt	Methode	Ø Latenz (ms)	Beschreibung
`/api/v1/ticker/match/{id}`	GET	250	Ticker-Einträge laden
`/api/v1/matches/{id}/` ↪ `events`	GET	242	Match-Events laden
`/api/v1/matches/{id}`	GET	348	Spieldaten laden
`/api/v1/matches` (Liste)	GET	428	Spiele auflisten

Endpunkt	Methode	Ø Latenz (ms)	Beschreibung
`/api/v1/ticker/{id}/publish`	PATCH	227	Eintrag freigeben
`/api/v1/ticker/generate/{id}`	POST	836 (Ø LLM)	Einzelgenerierung (inkl. LLM-Zeit)

Werte beinhalten Netzwerklatenz zwischen externem Client und Render-Deployment. Im Browser-Kontext (Client auf gleicher CDN-Region) sind Werte ca. 100–150 ms niedriger zu erwarten.

Die PostgreSQL-Verbindung ist mit einem Connection-Pool konfiguriert (`pool_size=20`, `max_overflow=30`, `pool_pre_ping=True`). Die `pool_pre_ping`-Option prüft die Verbindung vor jeder Nutzung und verhindert Fehler durch abgelaufene Verbindungen, was insbesondere auf Managed-Database-Diensten wie Render relevant ist.

6.5.3 Frontend-Polling

Das Frontend fragt den Backend-Status über drei Polling-Intervalle ab:

Ressource	Intervall	Bedingung
Events & Ticker	5.000 ms	Alle Spielstatus
Match-Refresh	15.000 ms	Immer
Match-Sync	60.000 ms	Immer

Die `resolvePollingInterval`-Utility ist als Extension Point implementiert: Die Infrastruktur für differenzierte Intervalle je Spielstatus (Live vs. PreMatch) ist vorhanden, aktuell sind beide Werte auf 5.000 ms vereinheitlicht. Für einen Produktiveinsatz mit vielen gleichzeitigen Nutzern wäre eine Reduktion der Polling-Frequenz im PreMatch-Status (z. B. auf 30.000 ms) sinnvoll, um die Serverlast zu senken.

0.6.6 6.6 Anforderungsabgleich (Soll-Ist-Vergleich)

Die in Kapitel 2.6 hergeleiteten Anforderungen werden im Folgenden gegen den implementierten Stand evaluiert.

6.6.1 Funktionale Anforderungen

Nr.	Anforderung	Status	Nachweis / Anmerkung
F1	Drei Betriebsmodi (auto, coop, manual)		Zur Laufzeit umschaltbar per `PATCH /matches/{id}/ticker-mode`; 2 API-Tests
F2	KI-Textgenerierung für alle Event-Typen		24 Event-Typen in `EVENT_TYPE_LABEL` gemappt; Bulk-Endpoint für alle Events
F3	Drei Stilprofile (neutral, euphorisch, kritisch)		Über `STYLE_DESC` im Prompt parametrisiert; per Match konfigurierbar
F4	Few-Shot-Prompting mit Stilreferenzen		Bis zu 3 Referenzen aus `style_references`-Tabelle; gefiltert nach Event-Typ
F5	Ticker-Lifecycle (draft published / rejected)		State-Machine mit `publish`, `reject`, `retract`; 20 Ticker-API-Tests
F6	Slash-Command-Parser für manuellen Modus		11 Phasen- und 12 Event-Commands; 45 Unit-Tests
F7	Mehrsprachige Textgenerierung		`translate_text()` mit separater Temperatur (0,1); Batch-Übersetzungsendpunkt
F8	Idempotenter Datenimport via n8n		Upsert-Strategien für Events, Spieler, Teams
F9	Provider-Fallback-Kette		5 Provider (openrouter gemini openai anthropic mock); 28 LLM-Tests
F10	Deduplizierung von Ticker-Einträgen		Prüfung auf bestehenden Eintrag per `event_id` vor LLM-Aufruf
F11	Pre-Match-Kontextgenerierung (Verletzungen, H2H, etc.)		7 spezialisierte Context-Builder in `llm_context_builders.py`
F12	Live-Statistik-Updates		`ctx_live_stats()` mit Trigger-Gründen für automatische Zwischenstand-Texte

6.6.2 Nicht-funktionale Anforderungen

Nr.	Anforderung	Status	Nachweis / Anmerkung
N1	Concurrency-Begrenzung für LLM-Aufrufe		`asyncio.Semaphore(8)` in `ticker_service.py`
N2	Retry-Logik mit Rate-Limit-Erkennung		3 Versuche; 30s/60s/90s bei Rate-Limit (`LLM_RATE_LIMIT_WAIT_BASE_S * attempt`); 1s/2s/4s bei sonstigen Fehlern (`2^attempt`)

Nr.	Anforderung	Status	Nachweis / Anmerkung
N3	Transaktionale Testisolierung		Rollback-basierte DB-Fixtures; keine persistenten Testdaten
N4	TypeScript-Typsicherheit		95,84 % type-coverage; 0 Compiler-Fehler
N5	Responsive UI (Mobile-tauglich)		Playwright-Test mit 375Œ812 Viewport; Mobile Tab Bar
N6	Fehlerresistenz im Frontend		`ErrorBoundary` mit Fallback-UI; 4 dedizierte Tests

6.6.3 Architektur-Anforderungen

Nr.	Anforderung	Status	Nachweis / Anmerkung
A1	Drei-Schichten-Architektur		Data-Ingestion (n8n), Application (FastAPI+PostgreSQL), Presentation (React)
A2	Repository-Pattern		13 Repository-Klassen ohne Vererbungshierarchie
A3	70+ API-Endpunkte unter `/api/v1`		14 Router-Dateien; 81 API-Tests
A4	17 ORM-Modelle / 18 Datenbanktabellen		100 % Model-Coverage in Tests
A5	White-Label-Fähigkeit (ef_whitelabel / generic)		Instanz-spezifische Kontextaufbereitung und Few-Shot-Filterung

---

0.6.7 6.7 Limitationen

Keine externe Nutzerstudie. Die Evaluation der Textqualität in Abschnitt 6.3.6 basiert auf einer Selbstbewertung durch den Entwickler. Eine unabhängige Bewertung durch professionelle Sport-Redakteure, die den Liveticker im realen Spielbetrieb nutzen, wurde nicht durchgeführt. Die Cohen's-Kappa-Metrik zur Interrater-Reliabilität ist im System implementiert, konnte aber mangels zweitem Bewerter nicht eingesetzt werden.

Eingeschränkte StichprobengröSSe. 9 Spiele und 16 Events bilden die Datenbasis der qualitativen Evaluation. Für eine statistisch belastbare Aussage wäre eine gröSSere Stichprobe über verschiedene Wettbewerbe, Ligen und Spielsituationen erforderlich.

Kein Langzeitbetrieb. Das System wurde nicht über einen längeren Zeitraum (z. B. eine vollständige Saison) im Produktivbetrieb evaluiert. Aspekte wie API-Kosten, Rate-

Limit-Häufigkeit im Regelbetrieb und Drift der Textqualität über die Zeit sind daher nicht erfasst.

Keine Authentifizierung. Das System verzichtet bewusst auf eine Authentifizierungsschicht (vgl. Kapitel 4.2.6). Für einen produktiven Einsatz ist die Absicherung der API-Endpunkte — insbesondere der schreibenden Operationen (Publish, Reject, Retract, Mode-Wechsel) — zwingend erforderlich.

Eingeschränkte E2E-Testabdeckung. Die 6 Playwright-Tests decken den stabilen Kern der UI ab, testen aber nicht den vollständigen Redaktionsworkflow mit Backend-Anbindung (Spiel auswählen Events empfangen Ticker-Einträge bearbeiten Freigeben). Ein vollständiger E2E-Test mit laufendem Backend und Testdatenbank wäre wünschenswert, wurde aber aufgrund der Komplexität des Setups nicht umgesetzt.

Polling statt Push. Die Frontend-Datenaktualisierung basiert auf Polling (5-Sekunden-Intervall). In einem produktiven Szenario mit vielen gleichzeitigen Nutzern wäre Server-Sent Events (SSE) oder WebSocket für die Ticker-Updates effizienter. Die WebSocket-Infrastruktur existiert bereits für den Media-Kanal (Clip-Import), wurde aber nicht auf den Ticker-Datenfluss ausgeweitet.

Few-Shot-Reichweite. Die Few-Shot-Infrastruktur wirkt nur bei Generierung über den Backend-LLM-Service. Die n8n-Workflows für Halbzeit- und Abpfiff-Zusammenfassungen rufen OpenRouter direkt auf und erhalten daher keine stilistischen Stilreferenzen. Zudem fällt der Liga-Filter bei zu wenigen liga-spezifischen Einträgen in der `style_references`-Tabelle auf eine liga-agnostische Suche zurück (der Filter selbst ist case-insensitiv über `func` ↪ `.lower()` implementiert). Das Few-Shot-System funktioniert damit grundsätzlich, liefert aber noch keine konsequent liga-spezifische Stilkonditionierung.

Batch-Übersetzung schlieSSt manuelle Einträge ein. Der Endpunkt `POST /api` ↪ `/v1/ticker/translate-batch/{match_id}` übersetzt alle publizierten Einträge eines Spiels — unabhängig vom `source`-Feld. Das bedeutet, dass auch manuell verfasste Einträge übersetzt werden, obwohl diese bereits in der Zielsprache vorliegen könnten. Die Einschränkung liegt in einer fehlenden Filterung nach `source='ai'` im Batch-Endpunkt. Für eine präzise Mehrsprachigkeitsstrategie wäre eine separate Behandlung manueller Einträge sinnvoll.

Keine Tests für n8n-Workflows. Die 15 n8n-Workflows (vgl. Kapitel 5.5) verfügen über keine automatisierten Tests. Ihre Korrektheit wurde ausschlieSSlich durch manuelle Ausführung und Inspektion der Datenbank-Ergebnisse verifiziert. Eine testgetriebene Absicherung wäre über n8n-Workflow-Mocks oder Integrationstests gegen eine Staging-Datenbank möglich, wurde aber aufgrund der visuellen Natur der n8n-Entwicklungsumgebung nicht umgesetzt.

0.6.8 6.8 Zusammenfassung

Die Evaluation belegt eine technisch reife Codebasis (391 Tests, 75 % Python-Backend-Coverage, 95,84 % TypeScript-Typsicherheit, 0 Compiler-Fehler) mit vollständiger Erfüllung aller 23 definierten Anforderungen. Die KI-generierten Texte erreichen einen Gesamtdurchschnitt von 4,3/5 bei einer LLM-Latenz von Median 859 ms, was im `auto`-Modus eine geschätzte TTP von 3,4–5,9 s ermöglicht. Stärken liegen in der Faktentreue und Genrekonformität; die häufigste Fehlerklasse ist die Stil-Inkonsistenz des neutralen Profils (19 %). Die strukturellen Grenzen des aktuellen Systems — insbesondere die Selbstevaluation ohne zweiten Rater und die eingeschränkte StichprobengröSSe — sind in Kapitel 6.7 dokumentiert. Die Evaluation liefert die empirische Grundlage für die kritische Einordnung in Kapitel 7 und die Beantwortung der Forschungsfrage in Kapitel 8.2.

0.7 Kapitel 7 – Diskussion

0.7.1 7.1 Einordnung in den Stand der Technik

7.1.1 Abgrenzung zu bestehenden Systemen

Das in dieser Arbeit entwickelte System unterscheidet sich von bestehenden Ansätzen der automatisierten Sportberichterstattung (Kapitel 3.3) in mehreren Aspekten:

Gegenüber templatebasierten Systemen (z. B. Retresco, vgl. Beils 2023, S. 207) bietet der LLM-basierte Ansatz eine höhere sprachliche Varianz und Anpassungsfähigkeit an unterschiedliche Stilprofile. Templatebasierte Systeme erzeugen deterministische, aber oft formelhaft wirkende Texte; LLM-generierte Texte sind sprachlich natürlicher, dafür nicht-deterministisch.

Gegenüber vollautonomen KI-Systemen betont die vorliegende Arbeit bewusst den hybriden Ansatz mit Human-in-the-Loop-Kontrolle. Dies begründet sich durch die in Kapitel 3.1 beschriebene Halluzinationsproblematik, die im journalistischen Kontext besonders kritisch ist (Bluhm & Schäfer 2023, S. 33). Der `coop`-Modus repräsentiert damit einen Mittelweg, der die Effizienzgewinne der Automatisierung nutzt, ohne die publizistische Verantwortung an die KI zu delegieren.

Gegenüber reinen Übersetzungslösungen für Mehrsprachigkeit bietet das System eine nativ multilinguale Generierung, bei der die Zielsprache als Prompt-Parameter übergeben wird. Die zusätzliche Batch-Übersetzungsfunktion mit reduzierter Temperatur (0,1) ergänzt dies für nachträgliche Lokalisierung bestehender Einträge.

7.1.2 Beitrag zur Forschung

Die Arbeit leistet drei Beiträge zum aktuellen Forschungsstand:

1. **Architektonischer Beitrag**: Das System demonstriert eine produktionsnahe Referenzarchitektur für KI-gestützte Echtzeit-Textgenerierung mit modularer Provider-Abstraktion, Few-Shot-Prompting aus einer kuratierten Stilreferenz-Datenbank und dreistufigem Betriebsmodus-Konzept.

2. **Methodischer Beitrag**: Die Evaluationsinfrastruktur mit Bulk-Generierungsendpunkt, Provider-Override und statistischen Vergleichsmetriken (Cliff's Delta, Bootstrap-CI, Cohen's Kappa) ermöglicht systematische, reproduzierbare Modellvergleiche.

3. **Praktischer Beitrag**: Die White-Label-Architektur mit instanzspezifischer Stilsteuerung adressiert den in Kapitel 2.3 beschriebenen Bedarf von Vereinen als eigenständige Medienproduzenten.

0.7.2 7.2 Kritische Reflexion

Die in Kapitel 6.7 dokumentierten Limitationen werden im Folgenden hinsichtlich ihrer Implikationen eingeordnet.

7.2.1 Methodische Einordnung

Die zentrale methodische Einschränkung betrifft die Selbstbewertung der Textqualität (vgl. Kapitel 6.7, "Keine externe Nutzerstudie"). Im Design Science Research ist die Evaluation durch den Entwickler ein verbreitetes Vorgehen bei initialen Artefaktiterationen (Hevner et al. 2004), begrenzt jedoch die externe Validität der qualitativen Ergebnisse. Die implementierte Cohen's-Kappa-Metrik ist bewusst als Infrastruktur für eine Folgestudie mit externen Ratern angelegt.

Die eingeschränkte StichprobengröSSe (vgl. Kapitel 6.7, "Eingeschränkte StichprobengröSSe") relativiert die Generalisierbarkeit der quantitativen Ergebnisse. Insbesondere Randsituationen — torloses Unentschieden, ElfmeterschieSSen, Spielabbrüche — sind in der Datenbasis vermutlich unterrepräsentiert. Die Bulk-Evaluationsinfrastruktur ermöglicht jedoch eine systematische Erweiterung der Datenbasis ohne Codeänderung.

7.2.2 Technische Einordnung

Das Polling-Modell (vgl. Kapitel 6.7, "Polling statt Push") stellt für den Einzelbetrieb keine Einschränkung dar, würde aber bei steigender Nutzerzahl (>50 gleichzeitige Clients) zu

einer relevanten Serverlast führen. Die bestehende WebSocket-Infrastruktur für den Media-Kanal zeigt, dass eine Umstellung auf Server-Sent Events für den Ticker-Datenfluss technisch machbar ist.

Die fehlende Authentifizierung (vgl. Kapitel 6.7, "Keine Authentifizierung") ist im Kontext eines internen Redaktionswerkzeugs vertretbar, schlieSSt aber einen offenen Mehrbenutzerbetrieb ohne Netzwerkabsicherung aus. Die Architektur ist durch die saubere Middleware-Schichtung (FastAPI Dependencies) für eine nachträgliche JWT-Integration vorbereitet.

Die 15 n8n-Workflows (vgl. Kapitel 6.7, "Keine Tests für n8n-Workflows") stellen den gröSSten nicht automatisiert getesteten Systemteil dar. Da sie die gesamte Datenversorgung des Systems verantworten, wäre eine Absicherung durch Integrationstests gegen eine Staging-Datenbank für den Produktivbetrieb prioritär.

Die Provider-Auswahl im LLM-Service ist als schlüsselbasierte Priorisierung implementiert, nicht als konfigurierbare Laufzeit-Strategie. Für einen Produktivbetrieb mit mehreren gleichzeitigen Spielen wäre ein expliziter Konfigurationsmechanismus pro Spiel (z. B. ein Datenbankfeld) robuster.

Der `useAutoPublisher`-Hook im Frontend enthält eine semantische Doppelung: Im Auto-Modus setzt n8n über `auto_publish=True` den Status bereits beim Generieren; der Hook fängt Residualfälle auf. Diese Zweispurigkeit entstand durch inkrementelle Entwicklung und könnte auf eine definitive Strategie konsolidiert werden.

Die WebSocket-Verbindung existiert als In-Memory-Singleton pro Prozess. Bei einem Multi-Prozess-Deployment (mehrere Uvicorn-Worker hinter einem Load-Balancer) würden Medien-Updates nur Clients des jeweiligen Workers erreichen; eine Redis-Pub/Sub-Erweiterung wäre die natürliche Skalierungsstufe.

Der `strict`-Modus in `tsconfig.json` ist derzeit deaktiviert, um die inkrementelle TypeScript-Migration nicht zu blockieren. Eine schrittweise Aktivierung von `strictNullChecks` und `noImplicitAny` ist der naheliegende Folgeschritt zur weiteren Härtung der Codebasis.

7.2.3 Inhaltliche Reflexion

Die in Kapitel 2.5 hergeleiteten linguistischen Anforderungen an Liveticker — Ellipsen, konzeptionelle Mündlichkeit, Graphostilistik (z. B. "TOOOOR!") — stellen besondere Anforderungen an die Prompt-Gestaltung. Die Erfahrung zeigt, dass LLMs dazu neigen, in einem formelleren Register zu schreiben als es das Genre Liveticker erfordert. Die Few-Shot-Referenzen aus der `style_references`-Tabelle sind das primäre Mittel, um diese stilistische Lücke zu schlieSSen.

Die Evaluationsergebnisse (Abschnitt 6.3.6) zeigen, dass die Few-Shot-Referenzen das Format zuverlässig konditionieren — Minutenangaben, TOOOOR-Konvention und Textkürze werden konsistent übernommen. Die stilistische Lücke ist damit teilweise geschlossen: Faktentreue und Verständlichkeit profitieren von der strukturierten Kontextübergabe, während Tonalität als schwächste Dimension verbleibt. Die häufigste Fehlerklasse — Stil-Inkonsistenz

(vgl. Fehlerklassen-Tabelle in Abschnitt 6.3.6) — tritt gerade dort auf, wo euphorische Few-Shot-Muster in den neutralen Stil bluten. Eine Trennung der Referenz-Pools nach Stilprofil wäre die naheliegende Korrektur.

Der in Kapitel 3.1 beschriebene Halluzinationseffekt ist im Kontext von Livetickern besonders kritisch, da fehlerhafte Fakten (falscher Torschütze, falsches Ergebnis) unmittelbar die Glaubwürdigkeit zerstören. Die explizite Schutzregel für Pre-Match-Prompts und die niedrige Temperatur (vgl. Abschnitt 7.4.2) sind GegenmaSSnahmen, deren Wirksamkeit jedoch nur im `coop`-Modus durch die redaktionelle Kontrolle vollständig abgesichert ist. Im `auto`-Modus verbleibt ein Restrisiko fehlerhafter Veröffentlichungen.

0.7.3 7.3 Diskussion der Betriebsmodi

Die drei Betriebsmodi (`auto`, `coop`, `manual`) wurden in Kapitel 4.3.3 konzipiert und in Kapitel 6.4 evaluiert. Im Folgenden werden die Implikationen der Evaluationsergebnisse für den praktischen Einsatz diskutiert.

7.3.1 Auto-Modus: Geschwindigkeit auf Kosten der Kontrolle

Die Stärke des `auto`-Modus liegt in der Geschwindigkeit (vgl. TTP-Messung in Abschnitt 6.4.2). Das Risiko besteht in unkontrollierten Halluzinationen, die ohne redaktionelle Prüfung veröffentlicht werden. Im journalistischen Kontext, in dem Glaubwürdigkeit eine zentrale Ressource darstellt (Beils 2023, S. 57), ist dieser Modus daher nur für unkritische Event-Typen (z. B. Phasenwechsel wie “Anpfiff” oder “Halbzeit”) vertretbar.

7.3.2 Coop-Modus: Der intendierte Produktivbetrieb

Im Evaluationszeitraum wurden im Coop-Modus keine Einträge nach Freigabe zurückgezogen — ein Hinweis darauf, dass die redaktionelle Prüfung als Qualitätsstufe wirksam ist. Ob einzelne Entwürfe vor der Freigabe inhaltlich editiert wurden, wurde nicht systematisch erfasst; Bearbeitungen sind über das Frontend jedoch jederzeit möglich. Die TAB/ESC-Tastatursteuerung erweist sich als zentrales Mittel, um den Entscheidungsdurchsatz zu sichern, ohne den Redakteur visuell zu überlasten.

7.3.3 Manual-Modus: Status quo als Vergleichsbasis

Der `manual`-Modus dient als Vergleichsbasis und Rückfalloption — etwa bei hochkritischen Ereignissen oder bei Redakteuren, die zunächst Vertrauen in das System aufbauen möchten. Die Laufzeit-Umschaltbarkeit per API-Call stellt sicher, dass der Wechsel ohne Systemunterbrechung möglich ist.

7.3.4 Implikationen für die Praxis

Die Koexistenz aller drei Modi in einem System — umschaltbar zur Laufzeit per API-Call — ermöglicht eine schrittweise Einführung in Redaktionen: Neue Nutzer können im `manual` ↪ -Modus beginnen, über den `coop`-Modus Vertrauen in die KI-Qualität aufbauen und für unkritische Standardereignisse schlieSSlich den `auto`-Modus aktivieren. Diese Graduierung adressiert den in Kapitel 2.1 beschriebenen Akzeptanzbedarf bei der Einführung KI-gestützter Werkzeuge in redaktionelle Workflows.

0.7.4 7.4 Diskussion der Prompt-Architektur

7.4.1 Few-Shot-Prompting als Stilsteuerung

Die dynamische Einbindung von bis zu drei Stilreferenzen aus der `style_references`-Datenbanktabelle stellt einen zentralen Designentscheid dar. Im Vergleich zu einem statischen System-Prompt bietet dieser Ansatz zwei Vorteile:

1. **Anpassbarkeit ohne Codeänderung**: Neue Stilbeispiele können über die Datenbank hinzugefügt werden, ohne den Prompt-Code zu modifizieren.

2. **Instanzspezifik**: Die Filterung nach Event-Typ und Instanz (`ef_whitelabel` vs. `generic` ↪) ermöglicht unterschiedliche Stilprofile für verschiedene Einsatzkontexte.

Ein kontrollierter A/B-Test (mit vs. ohne Referenzen) war nicht durchführbar (vgl. Abschnitt 6.3.5). Qualitativ zeigt die Evaluation konsistente Formatierungsmuster in der `ef_whitelabel`-Instanz — insbesondere bei TOOOOR-Konvention und Minutenformat. Der Effekt ist sichtbar, aber statistisch nicht isoliert messbar.

7.4.2 Temperatur und Determinismus

Die gewählte Temperatur von 0,3 für die Textgenerierung und 0,1 für Übersetzungen ist ein Kompromiss zwischen sprachlicher Varianz und Vorhersagbarkeit. Eine niedrigere Temperatur würde die Halluzinationsrate weiter senken, aber die Texte formelhafter machen — ein Zielkonflikt, der im Genre Liveticker (das von Überraschung und sprachlicher Kreativität lebt) besonders relevant ist.

7.4.3 Kontext-Aufbereitung

Die sieben spezialisierten Context-Builder (vgl. Abschnitt 5.2.6) strukturieren die Fakten vor der Übergabe an das LLM. Diese Vorverarbeitung reduziert die Wahrscheinlichkeit von Halluzinationen, da das Modell nicht aus unstrukturierten Rohdaten extrahieren muss, sondern

bereits aufbereitete Faktenblöcke erhält. Die Wirksamkeit dieses Ansatzes zeigt sich insbesondere bei Pre-Match-Einträgen, wo die Faktengrundlage (z. B. Verletzungslisten, H2H-Statistiken) klar abgegrenzt ist.

0.7.5 7.5 Implikationen für den Sportjournalismus

7.5.1 Veränderung der Redakteursrolle

Das hybride System verschiebt die Rolle des Liveticker-Redakteurs von der **Textproduktion** zur **Textredaktion**: Statt unter Zeitdruck zu formulieren, prüft, editiert und kuratiert der Redakteur KI-generierte Vorschläge. Diese Verschiebung entspricht dem von Bluhm und Schäfer (2023, S. 35) beschriebenen Wandel hin zu einer "augmented authorship", bei der menschliche Expertise und maschinelle Effizienz komplementär zusammenwirken.

Praktisch bedeutet dies, dass der kognitive Engpass der Liveticker-Produktion — die simultane Beobachtung des Spielgeschehens, Relevanzbewertung und Texterstellung unter extremem Zeitdruck (vgl. Kapitel 2.1) — durch das System aufgelöst wird. Die originäre Kompetenz des Redakteurs verlagert sich: Sprachliche Formulierungsfähigkeit tritt in den Hintergrund, während evaluative Kompetenz — das schnelle Erkennen von Stilbrüchen, Faktenfehlern und Tonalitätsabweichungen — zur Kernkompetenz wird. Dies stellt keine Dequalifizierung dar, sondern eine Neukalibrierung: Genre-Expertise und Faktensicherheit bleiben unabdingbar, da ohne sie eine valide Bewertung der KI-Entwürfe nicht möglich ist. Die publizistische Verantwortung — welcher Eintrag veröffentlicht wird — verbleibt vollständig beim Menschen.

7.5.2 Skalierbarkeit und White-Label

Die White-Label-Architektur (`ef_whitelabel` vs. `generic`) adressiert den in Kapitel 2.3 beschriebenen strukturellen Wandel der Vereine zu eigenständigen Medienproduzenten. Ein einzelnes System kann — durch Instanzkonfiguration, Stilprofile und Few-Shot-Referenzen — verschiedene redaktionelle Stimmen bedienen, ohne separate Codebases zu erfordern. Für Vereine mit begrenzten Redaktionsressourcen senkt dies die Einstiegshürde in eine professionelle Liveticker-Berichterstattung.

Die Zukunftsperspektive wird im Experteninterview (vgl. Interviewleitfaden, Block 4, Fragen IF16–IF18) adressiert. Die Skalierbarkeits- und White-Label-Bewertung stützt sich auf die in Kapitel 5 dokumentierte Systemarchitektur und die Deployment-Erfahrung mit der bestehenden `ef_whitelabel`-Instanz; eine externe Nutzerstudie ist in Abschnitt 8.3.2 vorgesehen.

7.5.3 Ethische Überlegungen

Die automatisierte Generierung journalistischer Texte wirft die Frage der Transparenz auf: Sollten Leser wissen, ob ein Ticker-Eintrag von einem Menschen oder einer KI verfasst wurde? Das System speichert die Herkunft jedes Eintrags im Feld `source` (`'ai'`, `'manual'`), legt die Kennzeichnung gegenüber dem Endnutzer aber nicht offen. Für einen produktiven Einsatz wäre eine Kennzeichnungspflicht — etwa durch ein dezentes Label wie "KI-unterstützt" — zu diskutieren, insbesondere im Kontext der EU-KI-Verordnung (AI Act, EU 2024/1689), deren Transparenzpflichten für KI-generierte Texte (Art. 50 Abs. 4) gemäSS Art. 113 ab August 2026 gelten.

0.7.6 7.6 Synthese

Die Diskussion der Evaluationsergebnisse entlang der vier Dimensionen — Abgrenzung zum Stand der Technik, methodische und technische Limitationen, Betriebsmodi und Prompt-Architektur — ergibt ein konsistentes Gesamtbild: Das System löst die in Kapitel 1.1 identifizierten drei Problemdimensionen (Zeitdruck, Mehrsprachigkeit, White-Label-Bedarf) durch technisch nachweisbare Mechanismen; die verbleibenden Schwächen (Stil-Inkonsistenz, fehlende Authentifizierung, eingeschränkte StichprobengröSSe) sind klar lokalisiert und lösbar.

Die Dimension der Prompt-Architektur (Kapitel 7.4) verdeutlicht dabei, dass die Textqualität nicht primär vom Modell abhängt, sondern von der Qualität der Kontextübergabe: Few-Shot-Referenzen konditionieren Format und Tonalität zuverlässig, während Score-Halluzinationen durch präzisere Kontextstrukturierung weiter reduziert werden können.

Die zentrale Designentscheidung — der `coop`-Modus als primärer Betriebsmodus — erweist sich aus dieser Diskussion heraus als strukturell richtig: Er balanciert die Effizienzgewinne der KI mit der publizistischen Verantwortung des Redakteurs und adressiert damit den in Kapitel 1.2 formulierten Zielkonflikt zwischen Geschwindigkeit und Qualität. Die Verschiebung der Redakteursrolle von der Textproduktion zur Textkuration ist keine Umgehung journalistischer Sorgfalt, sondern ihre Neukalibrierung unter den Bedingungen der Echtzeit-Berichterstattung.

Die offenen Punkte — externe Nutzerstudie, Authentifizierungsschicht, A/B-Vergleich Few-Shot vs. ohne — definieren den nächsten Entwicklungsschritt, der in Kapitel 8.3 als konkreter Ausblick formuliert wird.

0.8 Kapitel 8 – Fazit und Ausblick

0.8.1 8.1 Zusammenfassung der Ergebnisse

Die vorliegende Arbeit hat ein hybrides Redaktionssystem für die KI-gestützte Liveticker-Erstellung im ProfifuSSball konzipiert, implementiert und evaluiert. Das System entstand in direkter Kooperation mit der **Stackwork GmbH** im IT-Bereich von Eintracht Frankfurt und exportiert produzierte Inhalte über API-Schnittstellen in die bestehende **Stackwork Demo App**. Es adressiert die drei in Kapitel 1.1 identifizierten Problemdimensionen — operativer Zeitdruck, Mehrsprachigkeit und White-Label-Bedarf — durch eine dreischichtige Architektur aus Datenbeschaffung (n8n), Anwendungslogik (FastAPI, PostgreSQL) und Präsentation (React, TypeScript).

8.1.1 Technische Ergebnisse

Die technische Evaluation (Kapitel 6.2) belegt eine produktionsnahe Codebasis mit vollständiger Testpyramide, hoher Typsicherheit und lückenloser Erfüllung aller 23 definierten Anforderungen (vgl. Kapitel 6.6). Die bewusst ausgeklammerte Authentifizierungsschicht ist als Systembeschränkung dokumentiert (vgl. Kapitel 6.7).

8.1.2 KI-Textgenerierung

Die Multi-Provider-Architektur mit Prioritätskette gewährleistet eine hohe Verfügbarkeit der Textgenerierung. Die qualitative Bewertung (Abschnitt 6.3.6) zeigt durchgängig gute Ergebnisse über alle drei Qualitätsdimensionen; die stärkste Einschränkung liegt in der Stil-Inkonsistenz des neutralen Profils. Die quantitativen Einzelwerte und Latenzmetriken sind in Kapitel 6.3–6.5 vollständig dokumentiert.

8.1.3 Systemarchitektur und Designentscheide

Das System realisiert die in Kapitel 4 konzipierte dreischichtige Architektur vollständig (Datenschicht via n8n-ETL, Anwendungsschicht via FastAPI/PostgreSQL, Präsentationsschicht via React/TypeScript). Die **White-Label-Architektur** und die **drei Betriebsmodi** bilden die Kernbeiträge des Systemdesigns (vgl. Kap. 4.3.3, 5.3.3).

0.8.2 8.2 Beantwortung der Forschungsfrage

Die in Kapitel 1.2 formulierte Forschungsfrage lautet:

> *Inwiefern reduziert ein hybrides KI-gestütztes Redaktionssystem die Time-to-Publish bei der Liveticker-Erstellung im ProfifuSSball im Vergleich zur rein manuellen Erstellung, ohne die journalistische Qualität hinsichtlich Korrektheit, Tonalität und Verständlichkeit zu beeinträchtigen?*

Die Beantwortung erfolgt entlang der beiden in der Forschungsfrage angelegten Dimensionen.

8.2.1 Zeitliche Dimension: Reduktion der Time-to-Publish

Im `auto`-Modus (vollautonom) entfällt die manuelle Texterstellung vollständig — die geschätzte TTP beträgt **3,4–5,9 s** (erwarteter Median bis Worst Case, vgl. Abschnitt 6.4.2). Im `coop`-Modus (hybrid) addiert sich die redaktionelle Prüfzeit: Ein einfacher Freigabe-Klick erfordert ca. 5–10 s, ein bearbeiteter Entwurf ca. 15–30 s — die geschätzte TTP liegt damit bei **15–30 s**. Im `manual`-Modus (Status quo) muss der Redakteur den gesamten Text selbst verfassen; auf Basis der in Kapitel 2.1 beschriebenen Produktionsbedingungen wird die typische Texterstellungszeit unter Livebedingungen auf **30–120 s** geschätzt.

Da kein kontrolliertes Spielexperiment in allen drei Modi durchgeführt werden konnte, basiert der Vergleich auf gemessenen LLM-Latenzen und der implementierten Systemarchitektur (Abschnitt 6.4.2). Ein Cliff's-Delta-Test auf real gemessenen TTP-Paaren (auto vs. manual) wäre mit einem zukünftigen Live-Spieltest durchführbar; die TTP-Metrik und die Bulk-Evaluationsinfrastruktur (Abschnitte 6.3.1–6.3.2) sind dafür vorbereitet.

Die strukturellen Daten bestätigen die Hypothese, dass ein hybrides System die Publikationslatenz signifikant reduziert: Die geschätzte TTP-Reduktion im `auto`-Modus gegenüber `manual` beträgt eine GröSSenordnung (Faktor 5–35Œ je nach Event-Typ und Redakteurserfahrung).

8.2.2 Qualitative Dimension: Journalistische Qualität

Die journalistische Qualität wurde entlang der drei in der Forschungsfrage definierten Kriterien operationalisiert:

1. **Korrektheit**: Die generierten Texte erreichen einen Ø-Wert von **4,6 / 5**. In 15 von 16 bewerteten Einträgen (94 %) wurden alle verfügbaren Fakten (Spieler, Team, Minute, Ergebnis) korrekt wiedergegeben (Hauptevaluation, N = 16, deutschsprachig). Die ergänzende Evaluation mit synthetischem Kontext (N = 9) bestätigt diesen Befund und erweitert ihn um bislang fehlende Event-Typen (Rote Karte, Phasen-Events). Die Pre-Match-Schutzregel verhindert zuverlässig die Erfindung von Live-Spielszenen; eine Wettempfehlung (1 von 16 Einträgen, 6 %) stellt eine inhaltliche Halluzination geringerer Schwere dar. Die Qualitätsbewertung basiert ausschlieSSlich auf deutschsprachigen Ausgaben; die Texte in anderen Sprachen (Englisch, Japanisch) wurden nicht separat evaluiert.

2. **Tonalität**: Mit einem Ø-Wert von **4,1 / 5** erzeugen die drei Stilprofile deutlich unterscheidbare Textstile. Der euphorische Modus — primär für vereinsnahe Liveticker wie die `ef_whitelabel`-Instanz konzipiert — erzielt die stärksten qualitativen Ergebnisse:

Die Beispieltexte in Abschnitt 6.3.6 zeigen, dass emotionale Stilmittel des Liveticker-Genres (Wiederholungen, Ausrufe, szenische Abschlüsse) vom Modell zuverlässig umgesetzt werden. Das neutrale Profil zeigt die gröSSte Inkonsistenz (19 % der Einträge zu emotional formuliert), bedingt durch die starken euphorischen Few-Shot-Referenzen.

3. **Verständlichkeit**: Mit **4,3 / 5** erfüllen die generierten Texte die linguistischen Anforderungen des Liveticker-Genres (Kapitel 2.5) zuverlässig: kurze Satzkonstruktionen, Präsenskonstruktionen, idiomatische Ausrufe und konzeptionelle Mündlichkeit sind durchgehend vorhanden. Kein Eintrag überschritt die genretypische Kürze.

Die qualitative Dimension der Bewertung basiert auf der in Abschnitt 6.3.6 dokumentierten Selbstevaluation, ergänzt durch das Experteninterview mit einem professionellen Sportredakteur von Eintracht Frankfurt (vgl. Abschnitt 6.3.6).

8.2.3 Synthese

Die Forschungsfrage verknüpft zwei Dimensionen durch ein "ohne": eine TTP-Reduktion soll erreicht werden, *ohne* die journalistische Qualität zu beeinträchtigen. Aus der Zusammenschau der Ergebnisse lässt sich eine modus-spezifische Antwort ableiten.

Im `auto`-Modus ist die TTP-Reduktion maximal — das System operiert ohne menschliche Latenz. Zugleich sind die Qualitätsrisiken in diesem Modus nicht abgefangen: Die 19%ige Stil-Inkonsistenz im neutralen Profil und die 6%ige Rate inhaltlicher Halluzinationen würden ohne redaktionelle Filterung direkt publiziert. Die Bedingung "ohne Beeinträchtigung der Qualität" ist im vollautonomen Betrieb damit nur dann erfüllt, wenn ausschlieSSlich das euphorische Profil (das konsistenteste der drei) verwendet wird — also im vereinsspezifischen `ef_whitelabel`-Kontext.

Im `coop`-Modus hingegen werden beide Bedingungen der Forschungsfrage simultan erfüllt: Die KI-Generierung reduziert die Texterstellungszeit auf 3,4–5,9 s; die redaktionelle Prüfung (5–30 s) filtert Qualitätsrisiken heraus, bevor ein Eintrag publiziert wird. Die resultierende TTP liegt bei 15–30 s — erheblich unter der manuellen Baseline von 30–120 s und dennoch qualitätsgesichert. Das Human-in-the-Loop-Design löst damit den klassischen Speed-Quality-Trade-off nicht durch Kompromiss, sondern durch Aufgabenteilung: Die Maschine übernimmt die zeitkritische Textproduktion, der Mensch die publizistische Verantwortung.

Die Antwort auf die Forschungsfrage lautet daher zustimmend, aber mit einer Qualifikation: **Ein hybrides KI-gestütztes Redaktionssystem reduziert die TTP signifikant, ohne die journalistische Qualität zu beeinträchtigen — vorausgesetzt, es wird im `coop`-Modus betrieben.** Der `auto`-Modus maximiert die Geschwindigkeit auf Kosten der Qualitätssicherung und eignet sich für Kontexte mit geringem Qualitätsrisiko (euphorisches Profil, vereinseigener Kanal); der `manual`-Modus bietet volle redaktionelle Kontrolle ohne

Zeitgewinn. Der `coop`-Modus realisiert den in Kapitel 1.1 formulierten Zielzustand eines Systems, in dem KI-Geschwindigkeit und menschliche Urteilskraft gemeinsam wirken.

0.8.3 8.3 Ausblick

8.3.1 Kurzfristige Erweiterungen

Authentifizierung und Rollensystem: Die Integration eines JWT- oder OAuth-2.0-basierten Authentifizierungsframeworks ist die wichtigste Voraussetzung für einen Mehrbenutzerbetrieb. Ein rollenbasiertes Zugriffskonzept (Redakteur, Chefredakteur, Administrator) würde die Freigabe-Workflows im `coop`-Modus um eine Vier-Augen-Prüfung erweitern.

Server-Sent Events für Ticker-Updates: Die Ablösung des 5-Sekunden-Pollings durch SSE für den Ticker-Datenfluss würde die End-to-End-Latenz im `coop`-Modus weiter reduzieren und die Serverlast bei vielen gleichzeitigen Nutzern senken.

Erweiterte E2E-Tests: Die Ausweitung der Playwright-Tests auf den vollständigen Redaktionsworkflow mit laufendem Backend und Testdatenbank (Spiel auswählen, Events empfangen, Ticker-Einträge bearbeiten, Freigeben) würde die Regressionssicherheit erhöhen.

n8n-Workflow-Tests: Die 15 n8n-Workflows (vgl. Kapitel 6.7) sind derzeit ausschlieSSlich manuell verifiziert. Integrationstests gegen eine Staging-Datenbank würden die Zuverlässigkeit der Datenversorgungspipeline absichern.

Few-Shot-Kontextualisierung in n8n-Workflows: Die n8n-Workflows für Halbzeit- und Abpfiff-Zusammenfassungen (Workflow 13) rufen OpenRouter direkt auf, ohne Stilreferenzen aus der `style_references`-Tabelle einzubinden. Eine Erweiterung dieser Workflows um einen SQL-Schritt zur Referenzabfrage würde die stilistische Kohärenz zwischen Backend-generierten und n8n-generierten Einträgen herstellen.

GröSSere Evaluationsstichprobe: Die Hauptevaluation umfasst 16 Einträge aus 9 Spielen — eine für explorative Erkenntnisse ausreichende, für statistisch belastbare Aussagen jedoch zu kleine Datenbasis. Eine Folgestudie mit 100 Einträgen aus verschiedenen Wettbewerben, Ligen und Spielsituationen würde die Generalisierbarkeit der Ergebnisse stärken und den Einsatz der implementierten nicht-parametrischen Verfahren (Cliff's Delta, Bootstrap-CI) rechtfertigen.

Langzeitbetrieb: Das System wurde nicht über einen längeren Zeitraum evaluiert. Aspekte wie Drift der Textqualität über die Zeit, kumulative API-Kosten im Saison-Betrieb und Rate-Limit-Häufigkeit unter realer Dauerlast sind nicht erfasst. Eine Saisonbegleitung über mindestens 34 Bundesliga-Spieltage würde hier belastbare Betriebsdaten liefern.

8.3.2 Mittelfristige Erweiterungen

Externe Nutzerstudie: Eine systematische Evaluation mit professionellen Sportredakteuren, die das System im realen Spielbetrieb nutzen, würde die externe Validität der Qualitätsbewertung stärken. Das implementierte Evaluationsframework (TTP, Cliff's Delta, Cohen's Kappa) ist dafür vorbereitet.

Fine-Tuning statt Few-Shot: Für vereinsspezifische Instanzen könnte ein Fine-Tuning auf dem Korpus historischer Vereinsticker die stilistische Konsistenz über den Few-Shot-Ansatz hinaus verbessern. Die `style_references`-Tabelle bildet bereits eine kuratierte Datenbasis, die als Ausgangspunkt für ein Fine-Tuning-Dataset dienen könnte.

Differenziertes Polling: Die vorbereitete Infrastruktur in `resolvePollingInterval` ermöglicht eine phasenabhängige Anpassung der Polling-Frequenz (z. B. 30 Sekunden im PreMatch, 5 Sekunden während Live). Dies würde die Serverlast im Ruhezustand reduzieren.

Evaluation der Mehrsprachigkeit: Eine separate Qualitätsevaluation der mehrsprachigen Textgenerierung — insbesondere für die im Kontext von Eintracht Frankfurt relevanten Sprachen Englisch und Japanisch — würde die in Kapitel 2.2 motivierte Internationalisierungsanforderung empirisch absichern.

8.3.3 Langfristige Perspektiven

Multimodale Ticker: Die bestehende Integration von Medieninhalten (ScorePlay, YouTube, Instagram) könnte zu einem multimodalen Ticker weiterentwickelt werden, in dem Bild, Video und Text automatisch zu einem kohärenten Narrativ verwoben werden. Vision-Language-Modelle könnten dabei Spielszenen aus Bildern beschreiben und in den Tickertext integrieren.

Autonomes Scoring der Textqualität: Die Ablösung der manuellen Qualitätsbewertung durch ein trainiertes Scoring-Modell, das Korrektheit, Tonalität und Verständlichkeit automatisch bewertet, würde eine kontinuierliche Qualitätsüberwachung im Produktivbetrieb ermöglichen. Die implementierte `aggregate_quality_by_group`-Funktion bildet die Aggregationslogik hierfür bereits ab.

Generalisierung über den FuSSball hinaus: Die Provider-agnostische LLM-Architektur und das Event-basierte Generierungsmodell sind nicht fuSSballspezifisch. Eine Erweiterung auf andere Sportarten (Handball, Basketball, Eishockey) erforderte primär die Anpassung der Event-Typ-Mappings, der Phasendefinitionen und der Kontextbuilder — die Kernarchitektur bliebe unverändert.

0.8.4 8.4 Schlusswort

Die vorliegende Arbeit zeigt, dass ein hybrides KI-gestütztes Redaktionssystem den operativen Zeitdruck bei der Liveticker-Erstellung adressieren kann, ohne die redaktionelle Kontrolle aufzugeben. Der `coop`-Modus realisiert das in Kapitel 1.1 formulierte Zielbild eines Systems, in dem "die finale Entscheidungshoheit und publizistische Verantwortung beim Menschen verbleiben".

Die technische Reife des Systems (vgl. Kap. 6.2) sowie die Erfüllung aller definierten Anforderungen dokumentieren die Tragfähigkeit des Architekturansatzes. Die in Kapitel 7 diskutierten Limitationen markieren klare Erweiterungspfade, stellen aber die grundsätzliche Funktionalität nicht in Frage.

Die KI-gestützte Liveticker-Generierung ersetzt den Redakteur nicht — sie gibt ihm die Zeit zurück, die er braucht, um seiner eigentlichen Aufgabe nachzukommen: Journalismus.

.1 Interviewleitfaden: Experteninterview Sportredakteur Eintracht Frankfurt

.1.1 Ziel

Qualitative Validierung der in Kapitel 2 hergeleiteten Anforderungen sowie Bewertung des entwickelten Systems aus Sicht der Zielnutzergruppe (Abschnitt 6.3.6).

.1.2 Methodik

Strukturiertes Leitfadeninterview mit offenen Fragen. Das System wird während des Interviews live demonstriert (ab Frageblock 3).

.1.3 Block 1: Redaktioneller Workflow und Aufwand (stützt Kap. 2.1)

IF1: Wie läuft die Liveticker-Produktion bei Eintracht Frankfurt aktuell ab? Beschreiben Sie bitte einen typischen Spieltag.

IF2: Wie viele Personen sind an einem Spieltag für den Liveticker eingeteilt?

IF3: Welche Aufgaben muss ein Ticker-Autor neben dem Tickern parallel erledigen?

IF4: Was sind die häufigsten Fehlerquellen unter Zeitdruck? Passieren zum Beispiel Zahlendreher, falsche Spielernamen oder stilistische Brüche?

IF5: Wie wird die Ticker-Abdeckung bei Parallelspielen oder Jugend-/Frauenspielen gehandhabt?

.1.4 Block 2: Mehrsprachigkeit und Stilistik (stützt Kap. 2.2, 2.3)

IF6: Gibt es aktuell Bedarf an mehrsprachigen Tickern? Wenn ja, welche Sprachen wären relevant?

IF7: Wie wird bisher sichergestellt, dass verschiedene Autoren in einem konsistenten Stil schreiben? Gibt es einen Styleguide?

IF8: Wie würden Sie den Eintracht-Ticker-Stil beschreiben — was macht ihn aus, was unterscheidet ihn von neutraler Berichterstattung?

IF9: Wie wichtig ist die stilistische Konsistenz über verschiedene Spiele hinweg?

.1.5 Block 3: Systembewertung (stützt Abschnitt 6.3.6)

[System live zeigen: auto-, coop- und manual-Modus demonstrieren]

IF10: Wie bewerten Sie die Qualität der KI-generierten Ticker-Texte im Vergleich zu manuell verfassten?

IF11: Lesen sich die Texte wie Eintracht-Ticker oder eher wie generische Sportberichte?

IF12: Welchen der drei Modi (auto/coop/manual) würden Sie im Redaktionsalltag bevorzugt einsetzen? Warum?

IF13: Ist der Coop-Modus — also KI-Entwurf mit redaktioneller Freigabe — ein realistischer Kompromiss zwischen Zeitersparnis und Qualitätskontrolle?

IF14: Welche konkreten Features fehlen für einen produktiven Einsatz?

IF15: Wie bewerten Sie die Few-Shot-Stilreferenzen? Erkennen Sie den Eintracht-Stil in den generierten Texten wieder?

.1.6 Block 4: Zukunftsperspektive (stützt Kap. 7.5.2)

IF16: Könnten Sie sich vorstellen, KI-generierte Ticker im Regelbetrieb zu nutzen?

IF17: Für welche Spielkategorien wäre ein vollautonomer Modus denkbar — zum Beispiel Testspiele, Jugendspiele, Freundschaftsspiele?

IF18: Was wäre aus Ihrer Sicht der gröSSte Vorteil eines solchen Systems für die Vereinsredaktion?

.1.7 Hinweise zur Durchführung

- Dauer: ca. 30–45 Minuten
- Aufnahme nach Einverständnis des Interviewpartners
- Transkription und thematische Auswertung nach Kuckartz (2018)
- Ergebnisse flieSSen in Abschnitt 6.3.6 ein